



A K T U

B.Tech II-Year

CS IT & CS Allied / MCA

DATA STRUCTURE

Unit-5 : Graphs

in ONE SHOT

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified



Pdf Notes



AKTU PYQs



Imp. Questions



AKTU



Gateway Classes



CS IT & CS Allied

Gateway Udaan Batch



Sem-IV

Validity Till
Sem Exam

COMBO of 5 Subjects

- ✓ Video Lectures
- ✓ Pdf Notes
- ✓ AKTU PYQs
- ✓ Most Imp. Questions



Helpline No-7819 0058 53





AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-5 Lecture-1

Today's Target

- Introduction to Graph Data structure
- Graph Representations
 - Adjacency Matrix
 - Adjacency List
- Graph Traversal
 - Depth First Search(DFS)
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**



CS IT & CS Allied III-Sem

Subject : DATA STRUCTURE



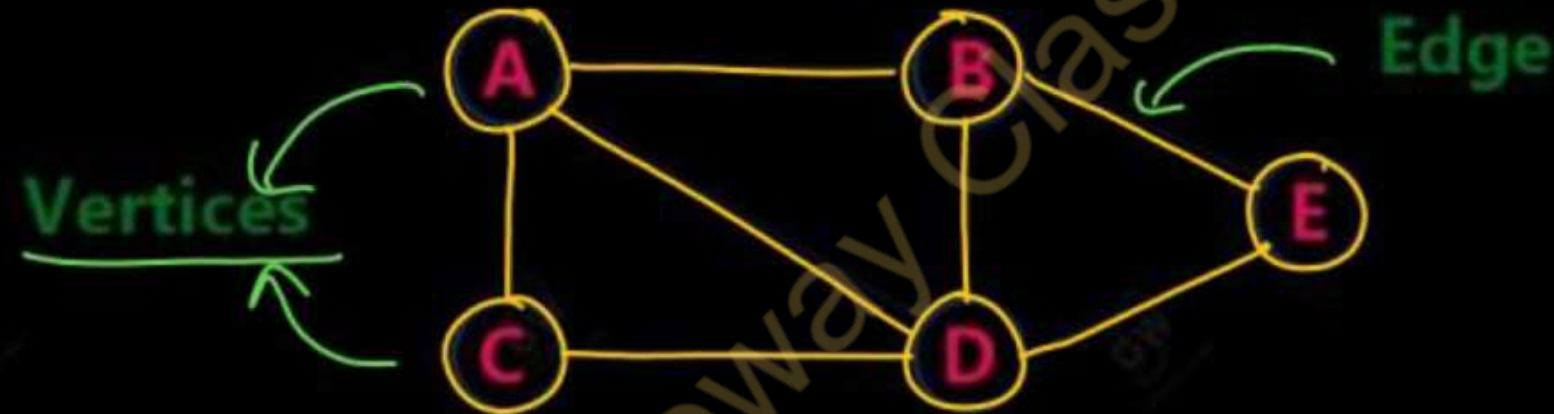
Unit-5 : Syllabus

Graphs: Terminology used with Graph, Data Structure for Graph Representations: Adjacency Matrices, Adjacency List, Adjacency. Graph Traversal: Depth First Search and Breadth First Search, Connected Component, Spanning Trees, Minimum Cost Spanning Trees: Prims and Kruskal algorithm. Transitive Closure and Shortest Path algorithm: Warshal Algorithm and Dijikstra Algorithm.

Graphs

- Graph is a non-linear data structure.
- It contains a set of points known as nodes (or vertices) and a set of links known as edges (or Arcs).
- Here edges are used to connect the vertices.

Graph is a collection of vertices(nodes) and edges in which vertices are connected with edges



Generally, a graph G is represented as $G = (V, E)$, where V is set of vertices and E is set of edges.

Example: The following is a graph with 5 vertices and ~~6~~⁷ edges.

Where $V = \{A, B, C, D, E\}$ and

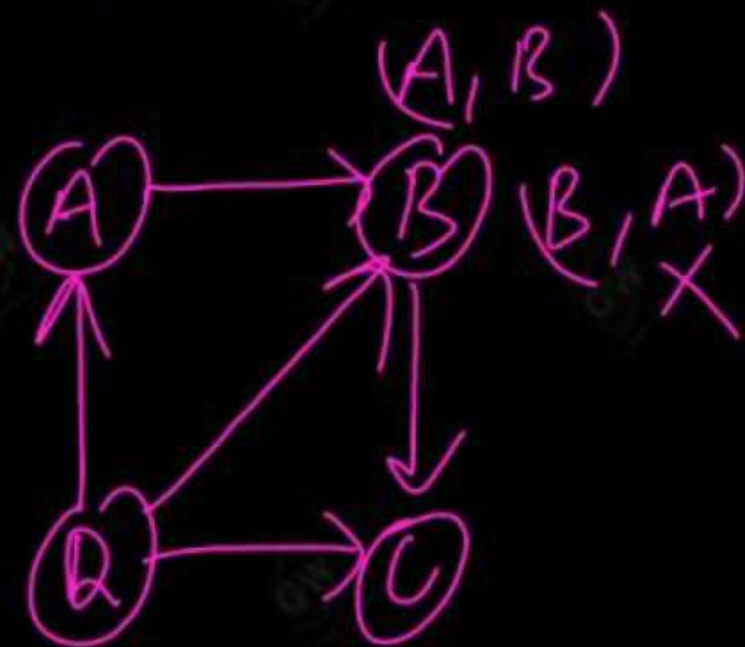
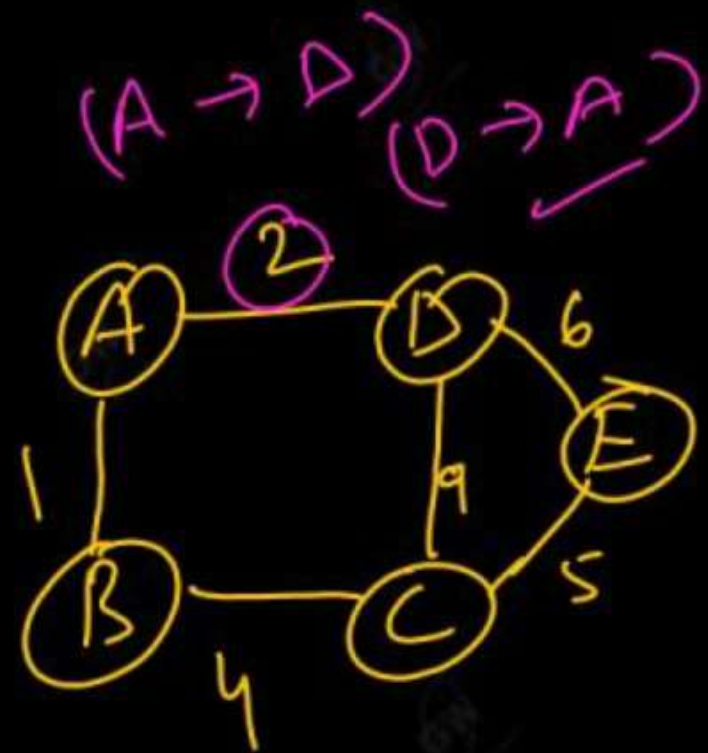
$$E = \{(\underline{A, B}), (\underline{A, C}), (A, D), (B, D), (C, D), (B, E), (E, D)\}.$$

Graph Terminology

Vertex: Individual data element of a graph, also known as node.

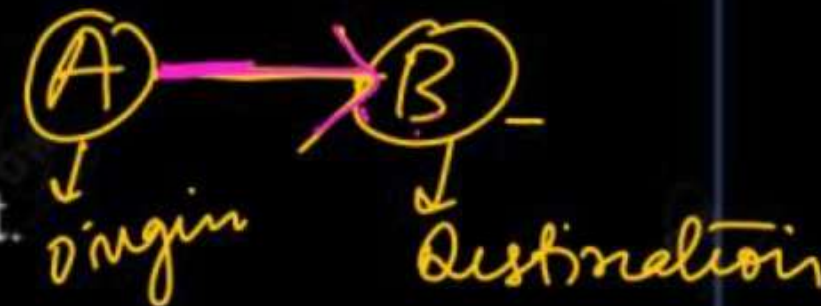
Edge: An edge(arc) is a connecting link between two vertices. An edge is represented as (startingVertex, endingVertex).

1. **Undirected Edge** - A bidirectional edge. If there is undirected edge between vertices A and B then edge (A, B) is equal to edge (B, A).
2. **Directed Edge** - A unidirectional edge. If there is directed edge between vertices A and B, edge (A, B) is not equal to edge (B, A).
3. **Weighted Edge** - A weighted edge is a edge with value (cost) on it.

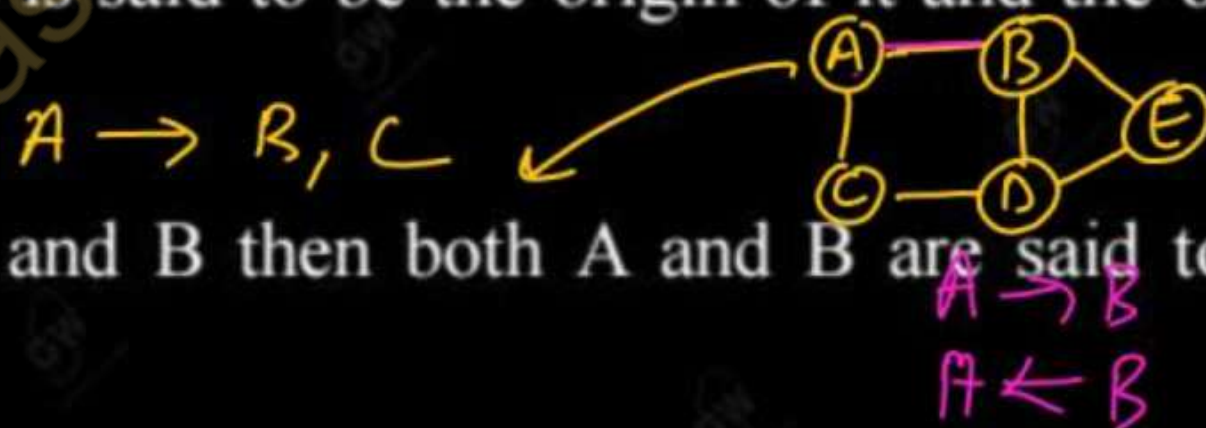


End vertices or Endpoints: The two vertices joined by edge are called end vertices (or endpoints) of that edge.

Origin: If an edge is directed, its first endpoint is said to be the origin of it.



Destination: If an edge is directed, its first endpoint is said to be the origin of it and the other endpoint is said to be the destination of that edge.



Adjacent: If there is an edge between vertices A and B then both A and B are said to be adjacent.

Incident: Edge is said to be incident on a vertex if vertex is one of the endpoints of that edge.

Outgoing Edge: A directed edge is said to be outgoing edge on its origin vertex.

Incoming Edge: A directed edge is said to be incoming edge on its destination vertex.

Degree: Total number of edges connected to a vertex is said to be degree of that vertex.

Indegree: Total number of incoming edges connected to a vertex is said to be indegree of that vertex.



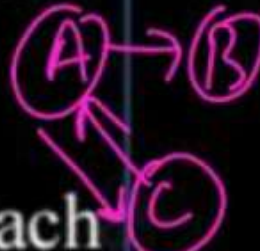
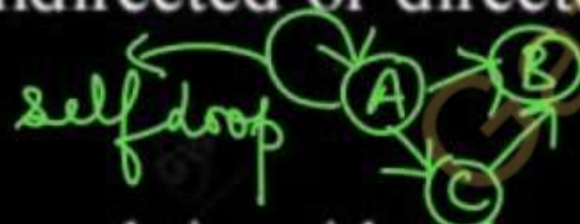
Outdegree: Total number of outgoing edges connected to a vertex is said to be outdegree of that vertex.



Parallel edges or Multiple edges: If there are two undirected edges with same end vertices and two directed edges with same origin and destination, such edges are called parallel edges or multiple edges.

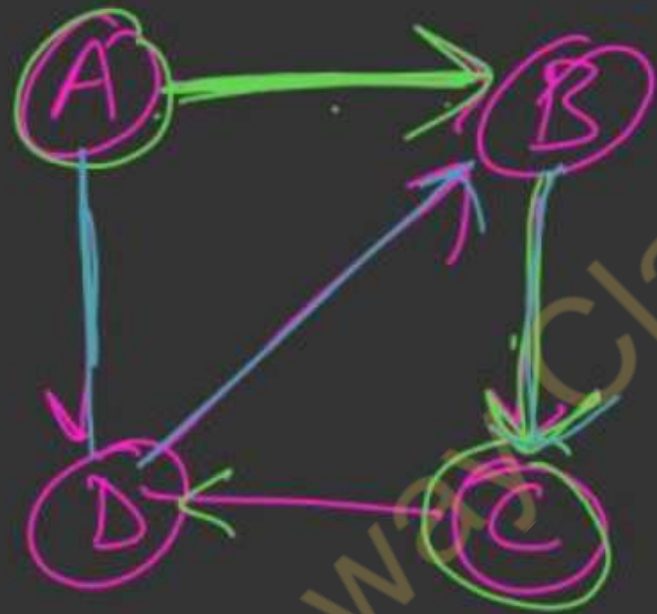


Self-loop: Edge (undirected or directed) is a self-loop if its two endpoints coincide with each other.



Simple Graph: A graph is said to be simple if there are no parallel and self-loop edges.

Path: A path is a sequence of alternate vertices and edges that starts at a vertex and ends at other vertex such that each edge is incident to its predecessor and successor vertex.

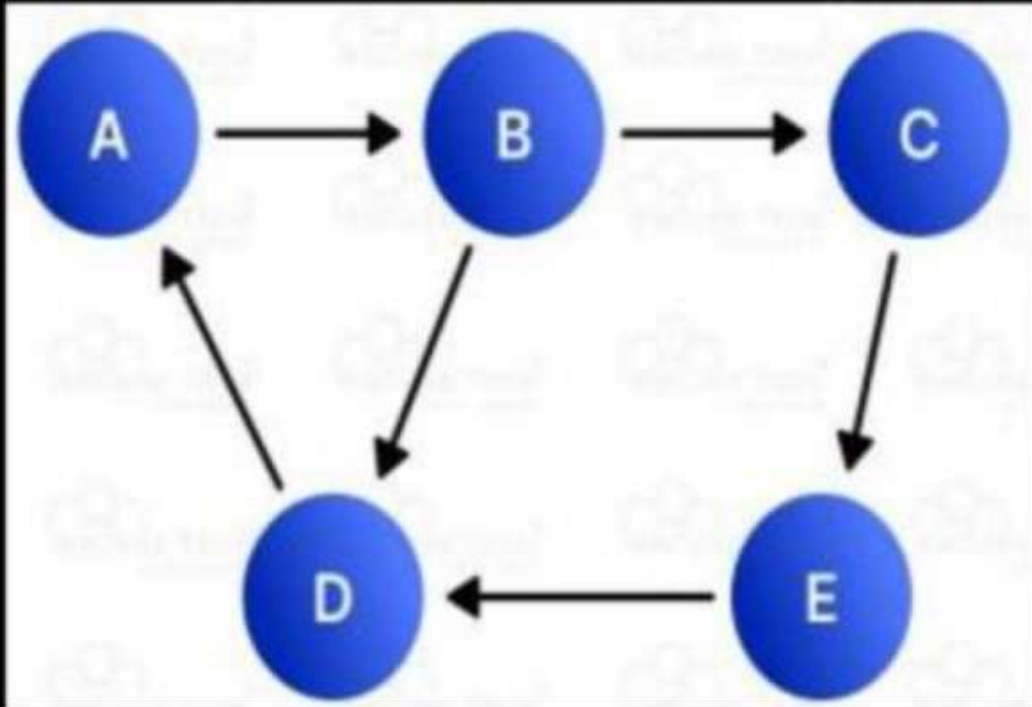


$A \rightarrow B \rightarrow C$

$A \rightarrow D \rightarrow B \rightarrow C$

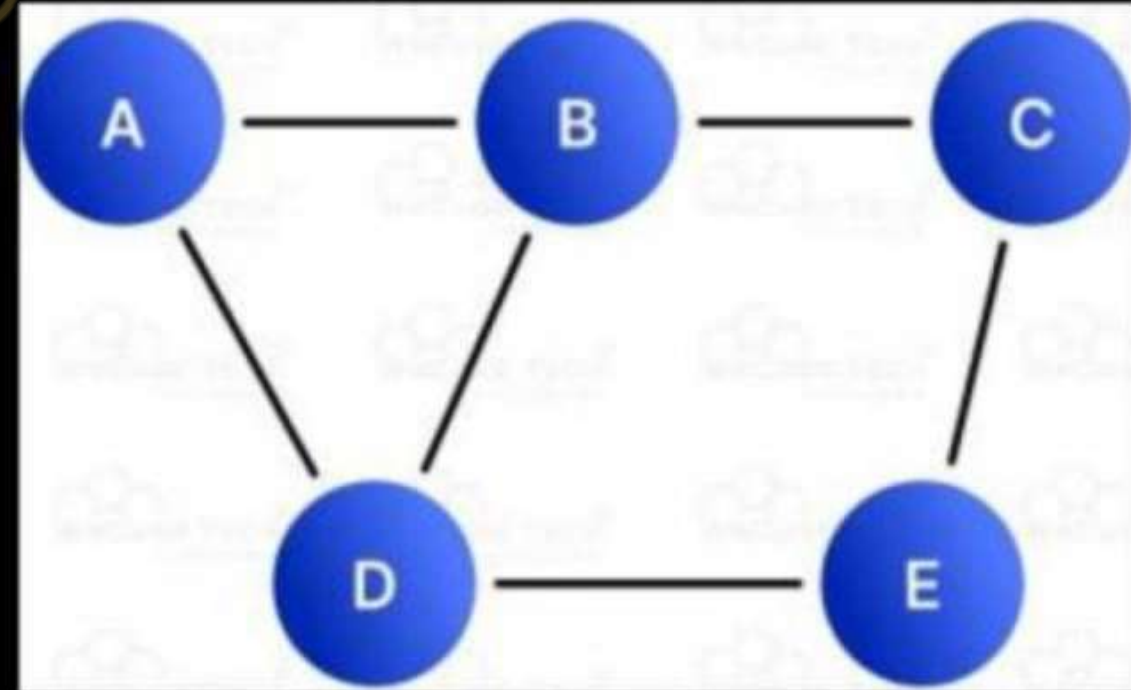
1. Directed Graph:

Edges have a direction, they go from one node to another in a specific way.



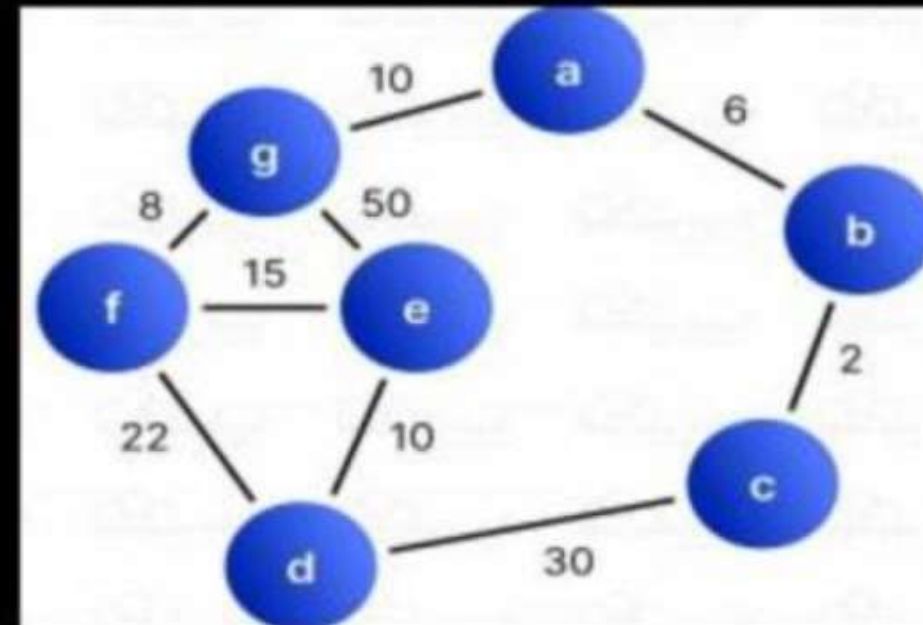
2. Undirected Graph

Edges do not have a direction. They simply connect two nodes without any particular order.



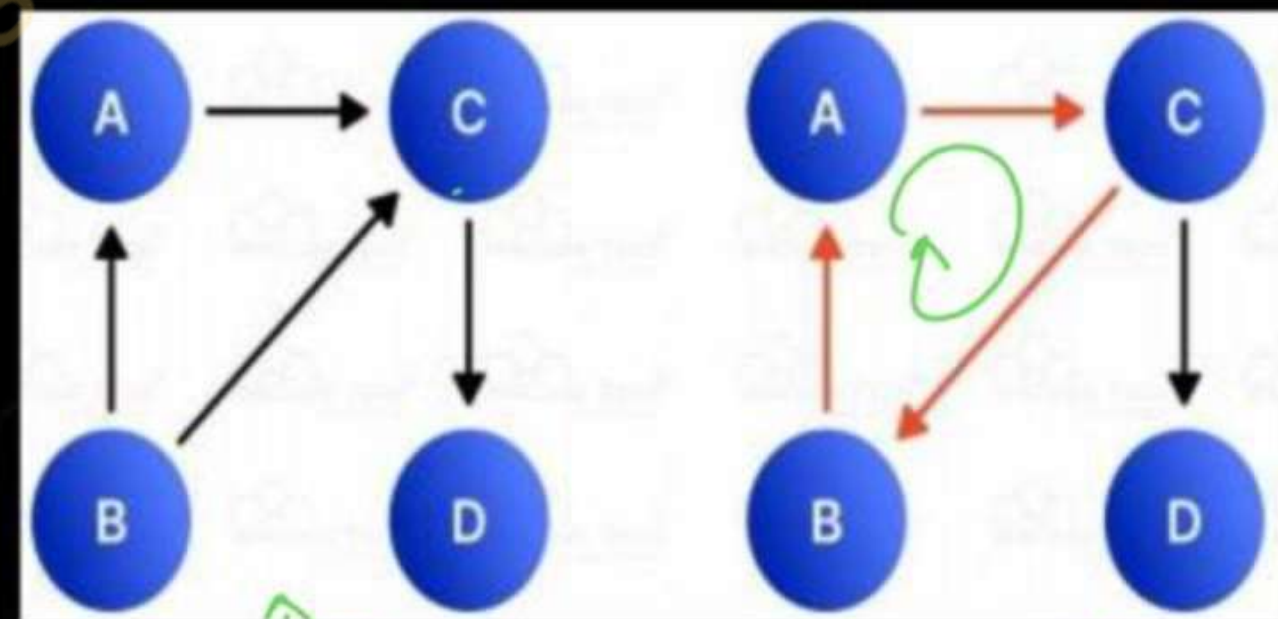
3. Weighted Graph

Edges have weights or costs associated with them. These weights can represent distances, costs, or any other metric.



4. Cyclic Graph

A cyclic graph contains at least one cycle, you can start at a node and follow a path that leads back to the same node



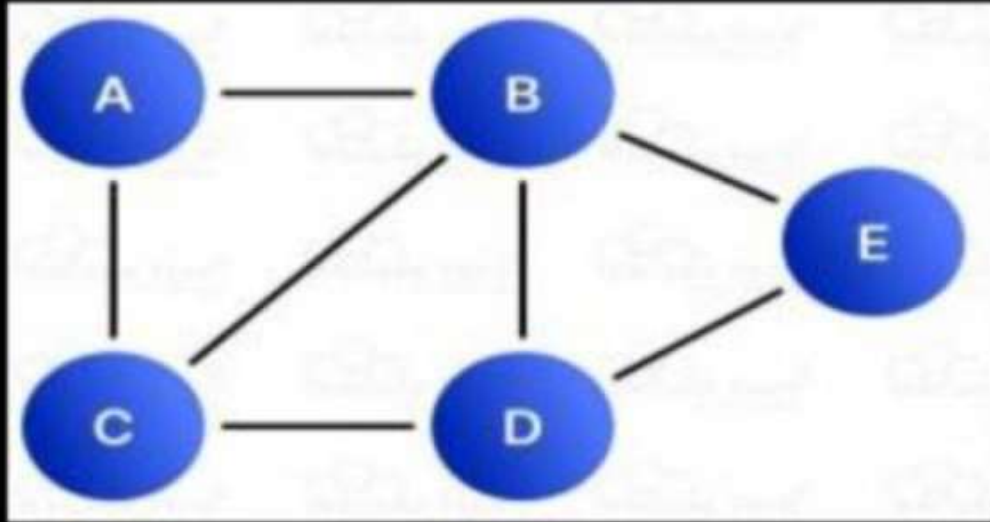
Acyclic graph

cyclic graph

5. Acyclic Graph: does not contain any cycles.

6. Connected Graph

A connected graph has a path between every pair of nodes.



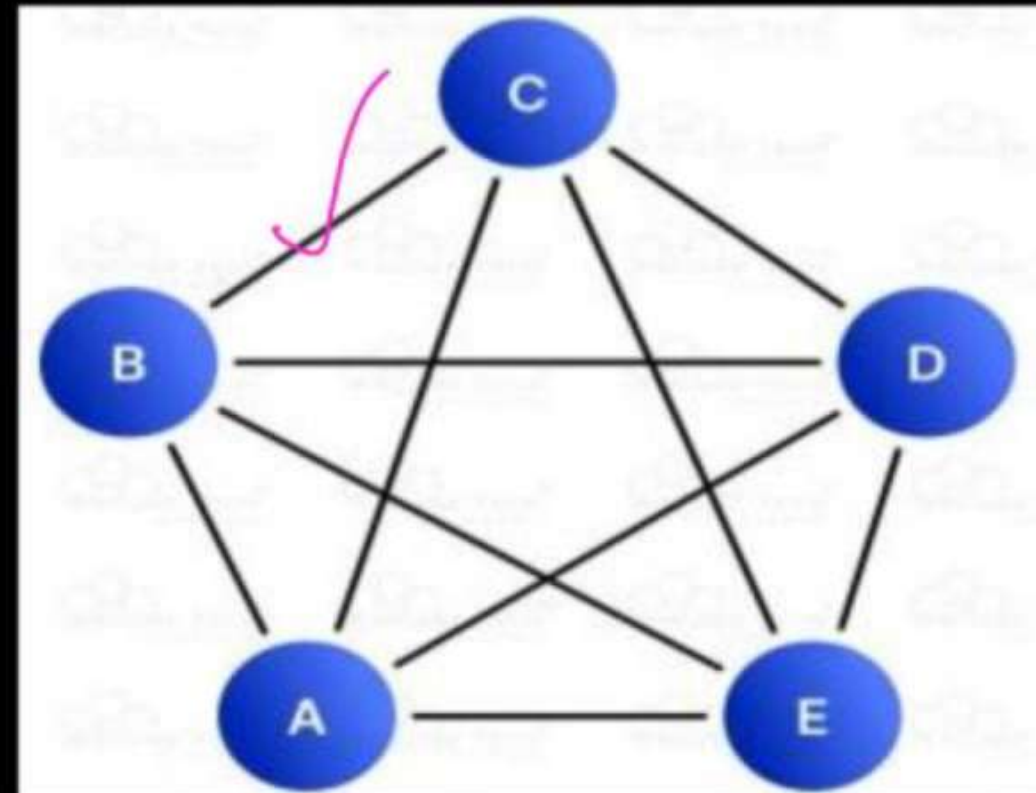
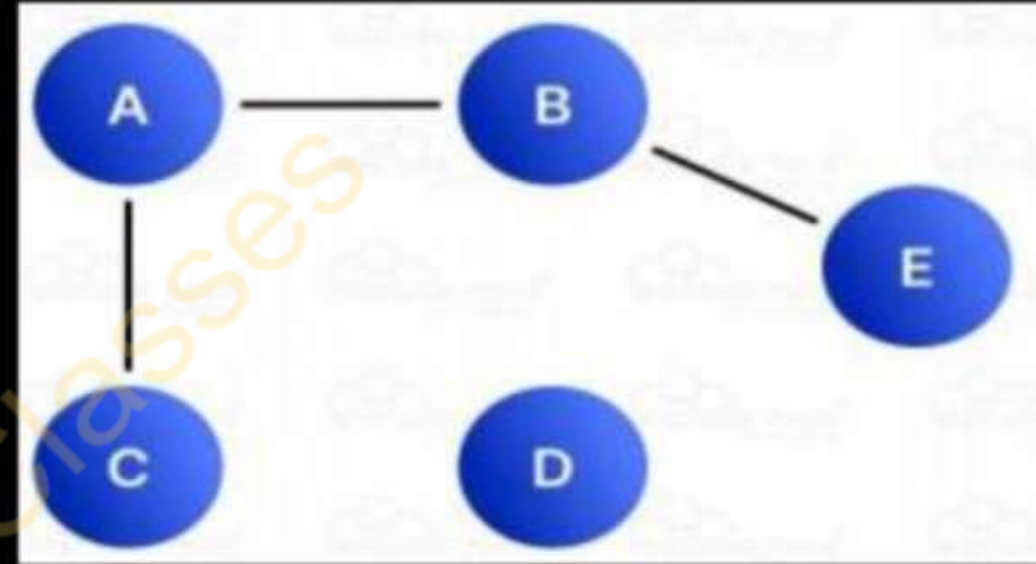
8. Complete Graph

In a complete graph, there is a unique edge between every pair of nodes.

$n = 5$
 $5 \times 4 = 20$
 $\frac{20}{2} = 10$
 No. of edges = $\frac{n(n-1)}{2}$

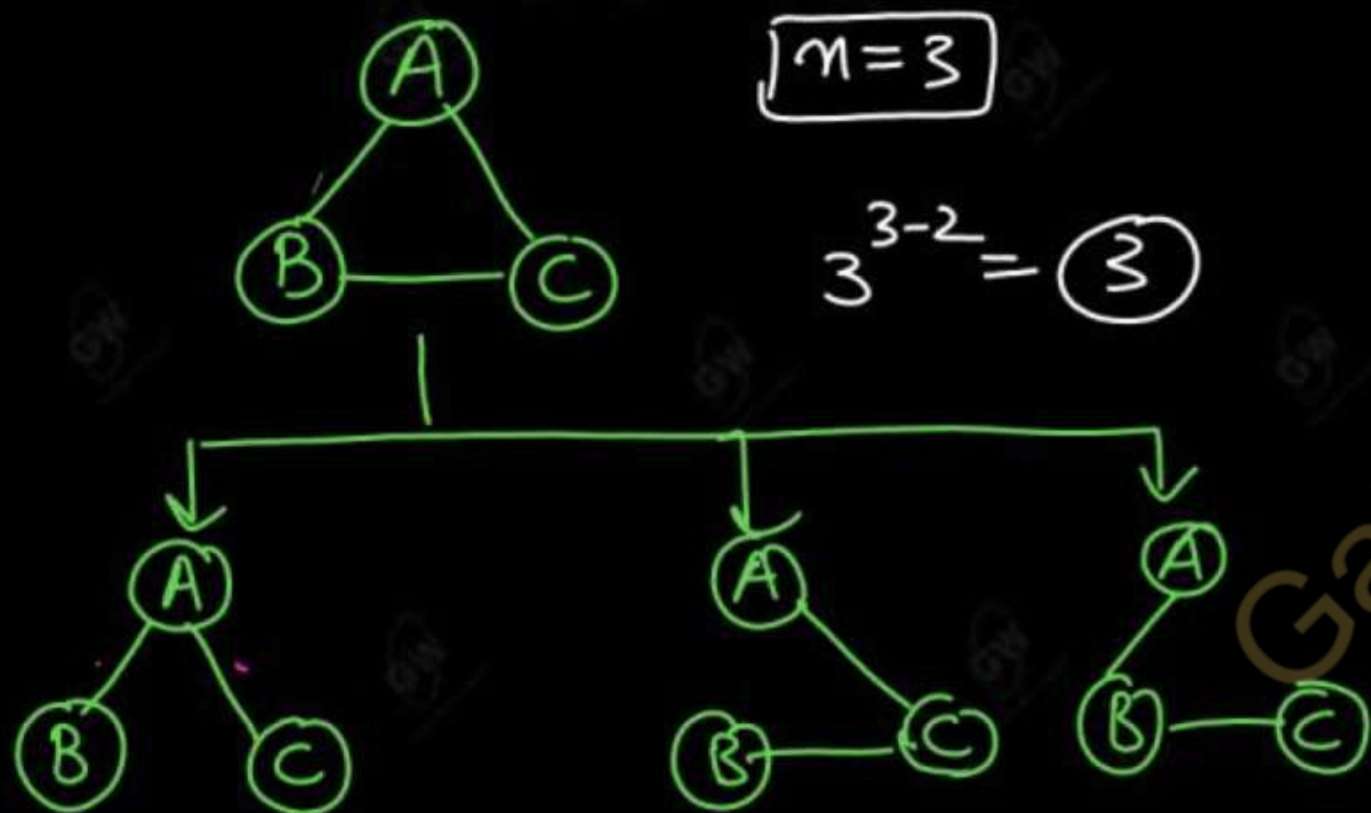
7. Disconnected Graph

A disconnected graph has at least one pair of nodes with no path between them.



$\frac{n \times (n-1)}{2}$

- A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..
- Every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.



- A complete undirected graph can have maximum n^{n-2} number of spanning trees, where n is the number of nodes.
- Here n is 3, hence $3^{3-2} = 3$ spanning trees are possible.

1. Adjacency Matrix Representation

2. Adjacency List Representation

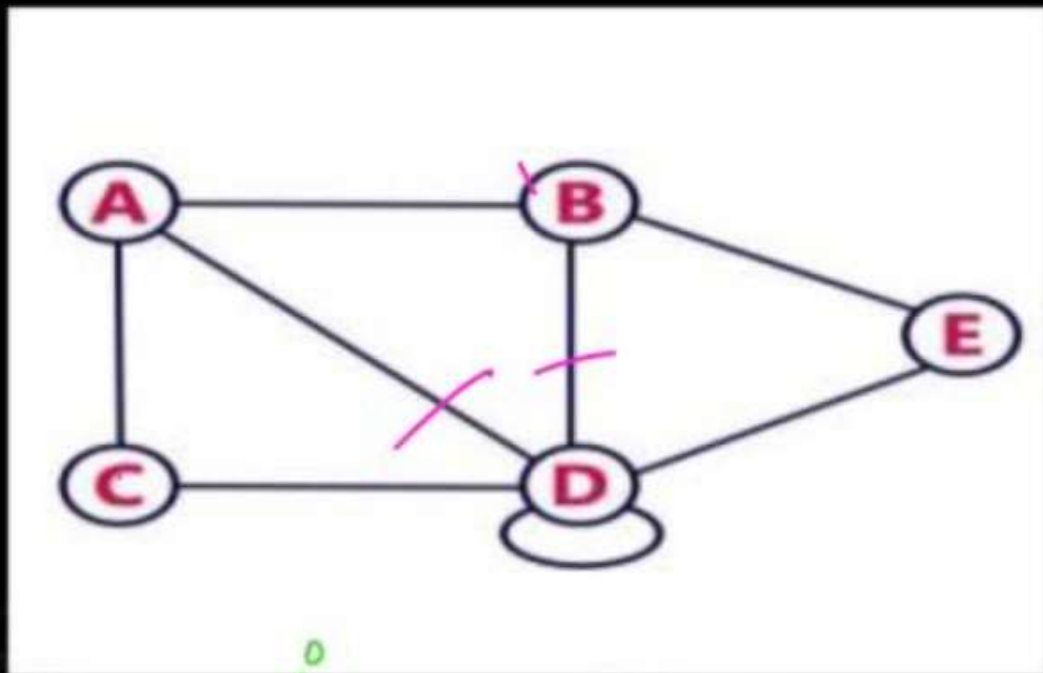
Adjacency Matrix

In this, graph is represented using a matrix of size total number of vertices by $V \times V$ where V is total number of vertices. This matrix is filled with either 1 or 0. Here, 1 represents that there is an edge from row vertex to column vertex and 0 represents that there is no edge from row vertex to column vertex.

- An **adjacency matrix** is a **2D array** where each cell $[i][j]$ indicates whether there is an edge between vertex i and vertex j .

Structure: For a graph with V vertices, we use an $V \times V$ matrix:

- $\text{matrix}[i][j] = 1$ (or weight w) if there is an edge from i to j
- $\text{matrix}[i][j] = 0$ if there is no edge



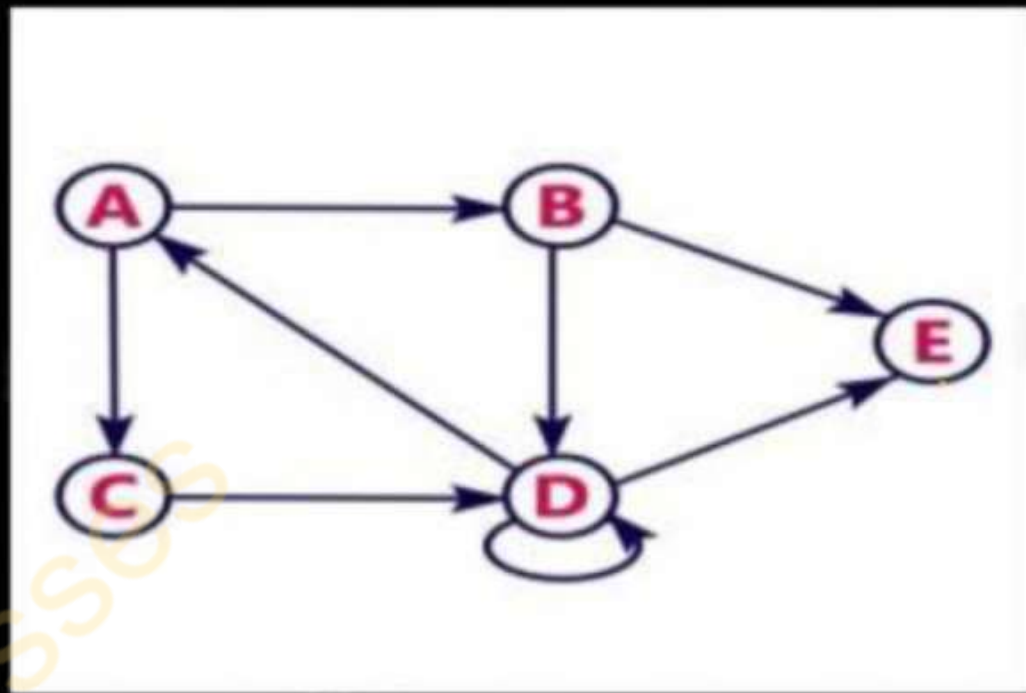
$V \rightarrow 5$
 $=$

(5×5)

Adj $\Rightarrow$
Matrix

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	1	1
E	0	1	0	1	0

5×5



	A	B	C	D	E
A	0	1	1	0	0
B	0	0	0	1	1
C	0	0	0	1	0
D	1	0	0	1	1
E	0	0	0	0	0

Space Complexity: $O(V \times V)$ ✓

Advantages of Adjacency Matrix:

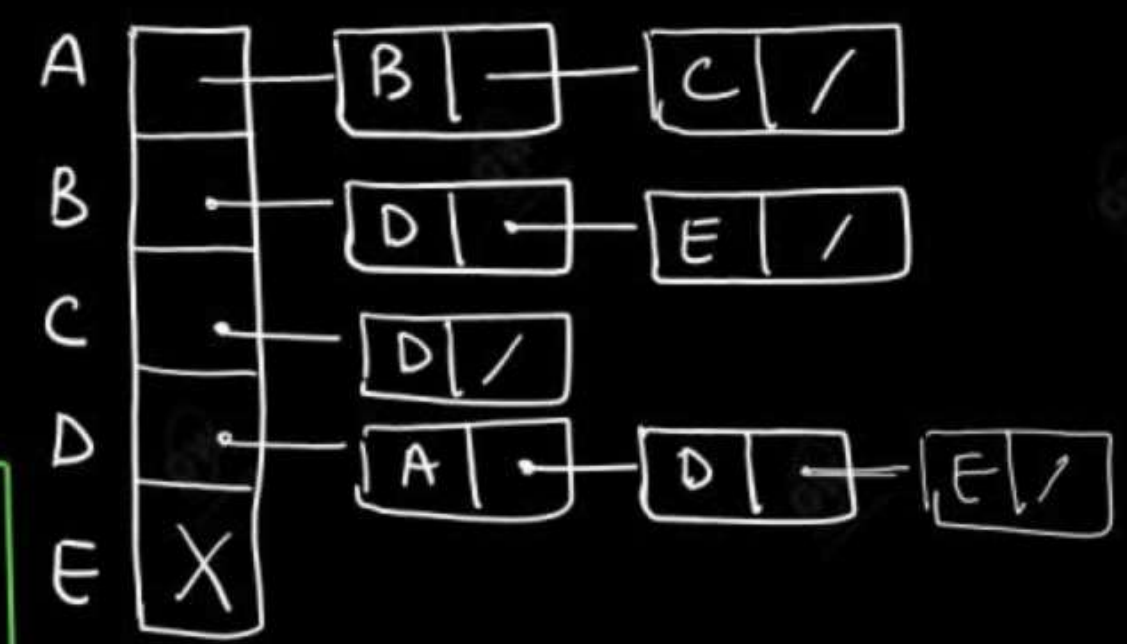
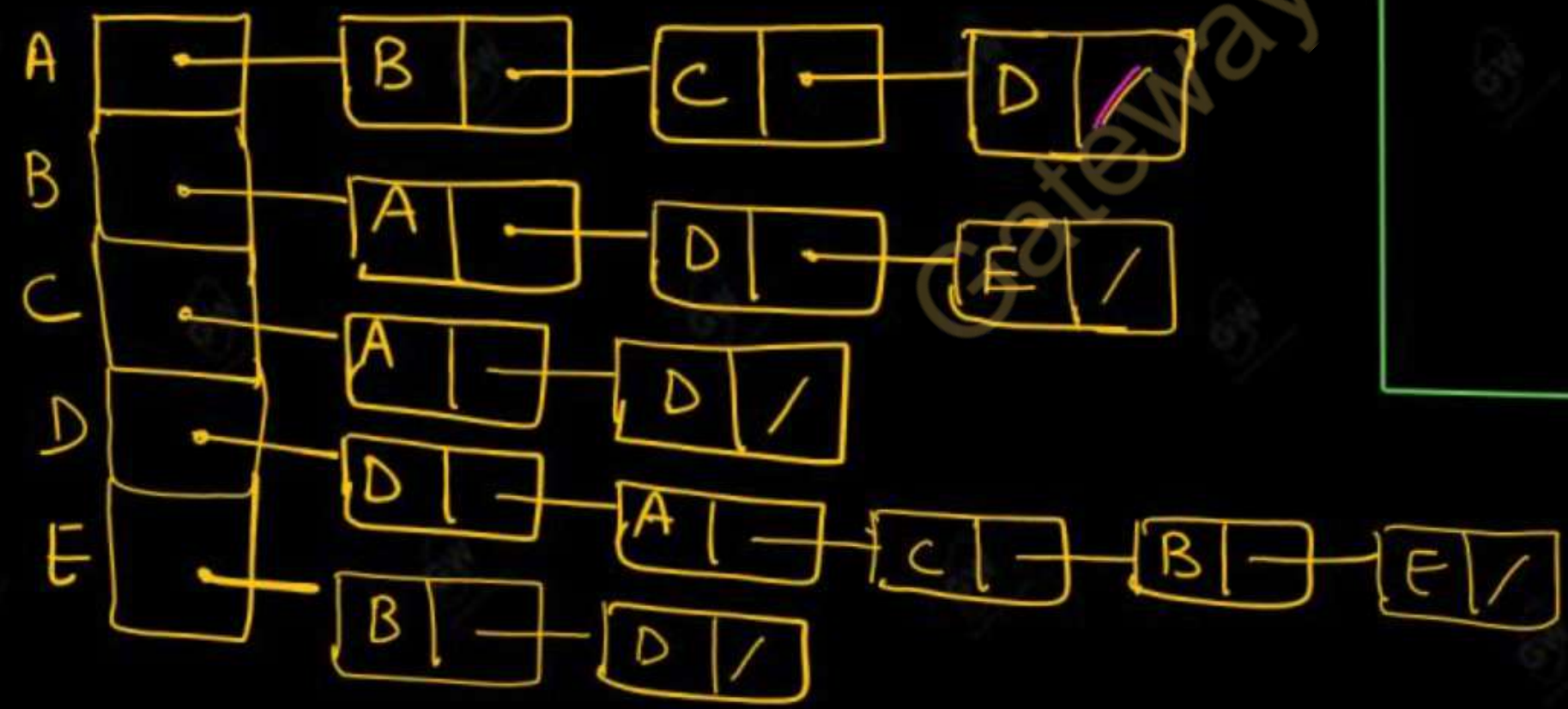
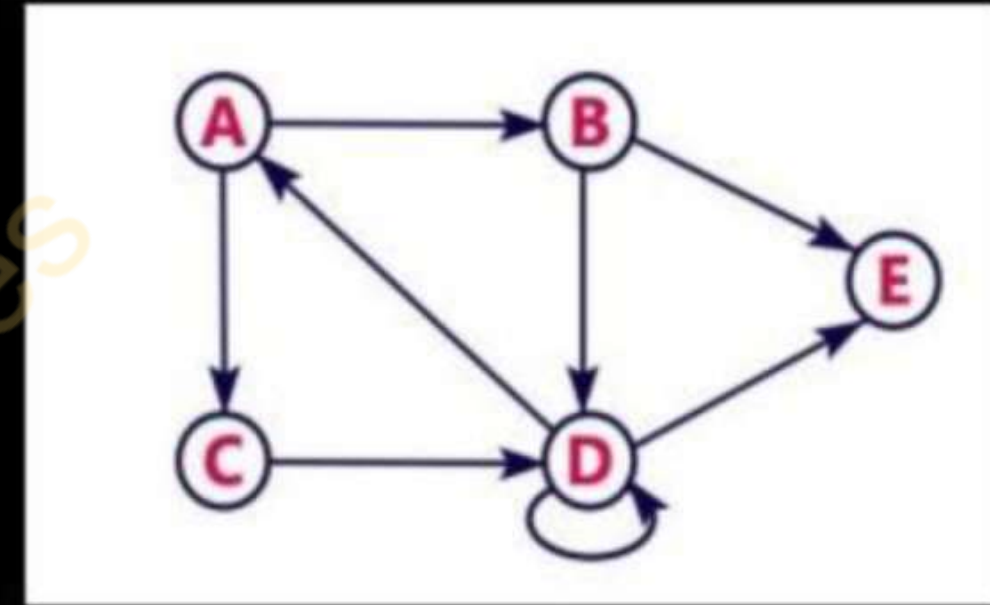
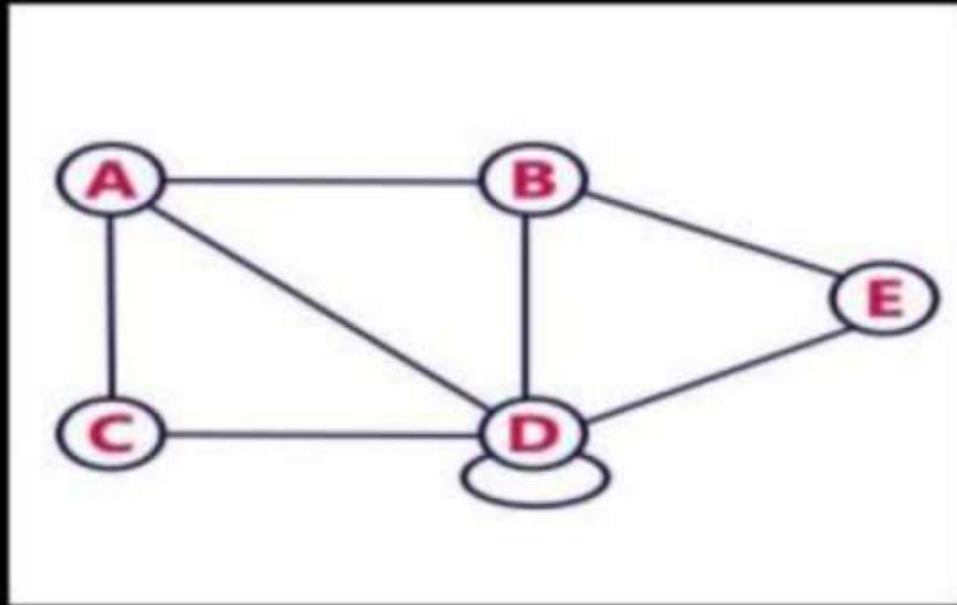
1. Simple and Easy to implement.
2. Space efficient when the graph is dense as it requires VXV space to represent the entire graph.
3. Allows constant time(faster) look up to check whether there exists an edge between a pair of vertices.

Disadvantages of Adjacency Matrix:

1. Space inefficient for Sparse Graphs as it takes up VXV space even if the graph is sparse.
2. Adding or deleting a vertex can be costly as it requires resizing the matrix.
3. Slow Traversals: Graph traversals like DFS, BFS takes $O(\underline{VXV})$ time to visit all the vertices whereas Adjacency List takes only $O(V + E)$.

Adjacency List Representation

In this representation, every vertex of a graph contains list of its adjacent vertices.



➤ In the worst case, if a graph is connected $O(V)$ is required for a vertex and $O(E)$ is required for storing neighbours corresponding to every vertex.

➤ Overall space complexity is $O(|V|+|E|)$.

$V \times V$

Advantages of using an Adjacency list:

1. Requires less space compared to adjacency matrix for sparse graphs.
2. Easy to traverse through all edges of a graph.
3. Adding an vertex is easier compared to adjacency matrix representation.
4. Most of the graph algorithms are implemented faster with this representation.

Disadvantages of using an Adjacency list:

1. Checking if there is an edge between two vertices is costly as we have to traverse the adjacency list.
2. Not suitable for dense graphs.

- To traverse a Graph means to start in one vertex, and go along the edges to visit other vertices until all vertices, or as many as possible, have been visited.
- The graph traversal is also used to decide the order of vertices in which they are visited.
- In graph traversal, we visit all the vertices of the graph without getting into looping path.

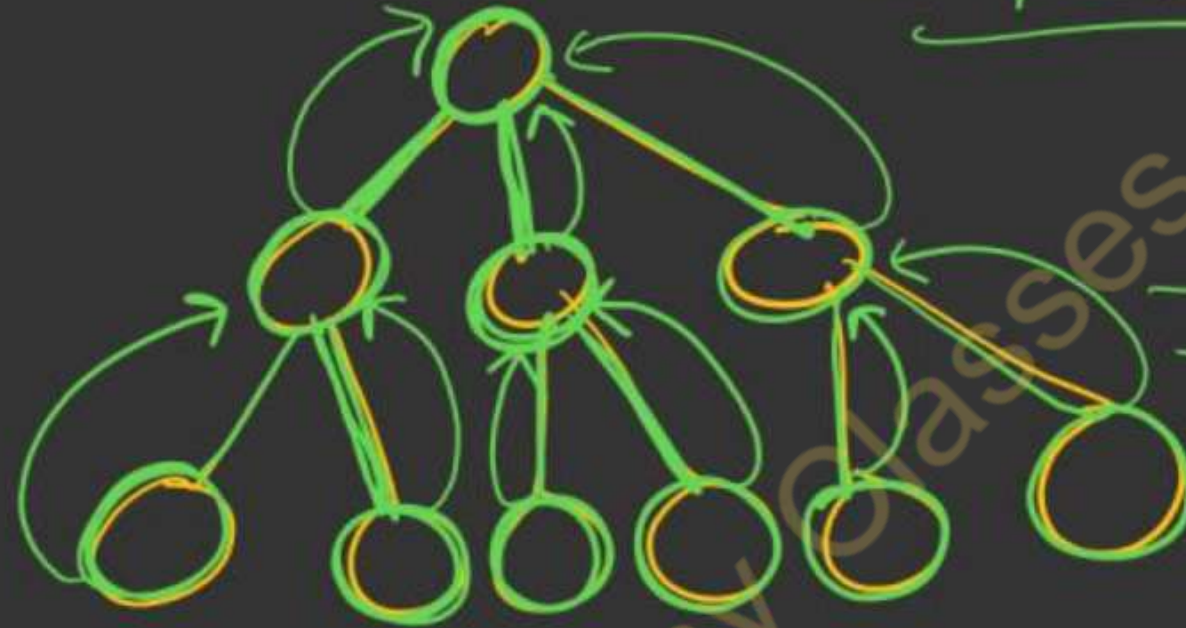
There are two graph traversal techniques and they are as follows...

1. DFS (Depth First Search) ✓

2. BFS (Breadth First Search) ✓

DFS

(Depth)



Stack is used

BFS



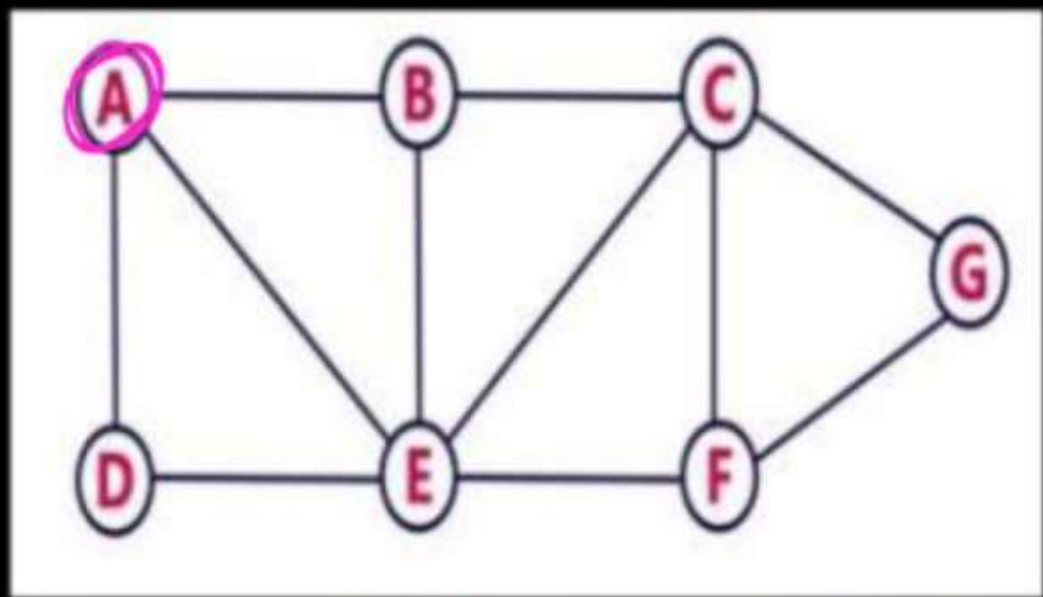
Queue is used



We use **Stack data structure** with maximum size of total number of vertices in the graph

Steps to implement DFS traversal..

- Step 1 - Define a Stack of size total number of vertices in the graph.
- Step 2 - Select any vertex as **starting point** for traversal. Visit that vertex and push it on to the Stack.
- Step 3 - Visit any one of the non-visited **adjacent** vertices of a vertex which is at top of stack and push it on to the stack.
- Step 4 - Repeat step 3 until there is no new vertex to be visited from vertex which is at top of stack.
- Step 5 - When there is no new vertex to visit then use **back tracking** and pop one vertex from stack.
- Step 6 - Repeat steps 3, 4 and 5 until stack becomes Empty.
- Step 7 - When stack becomes Empty, then produce final spanning tree by removing unused edges from graph



1)

A	B	C	D	E	F	G

status → visited/Not visited

2) Stack ⇒ size equal to
no. of nodes (7)

3) Output → showing the order
of traversal

Step-1.

Selecting A as starting vertex



output → A

Status -

A	B	C	D	E	F	G
1						

Step-2

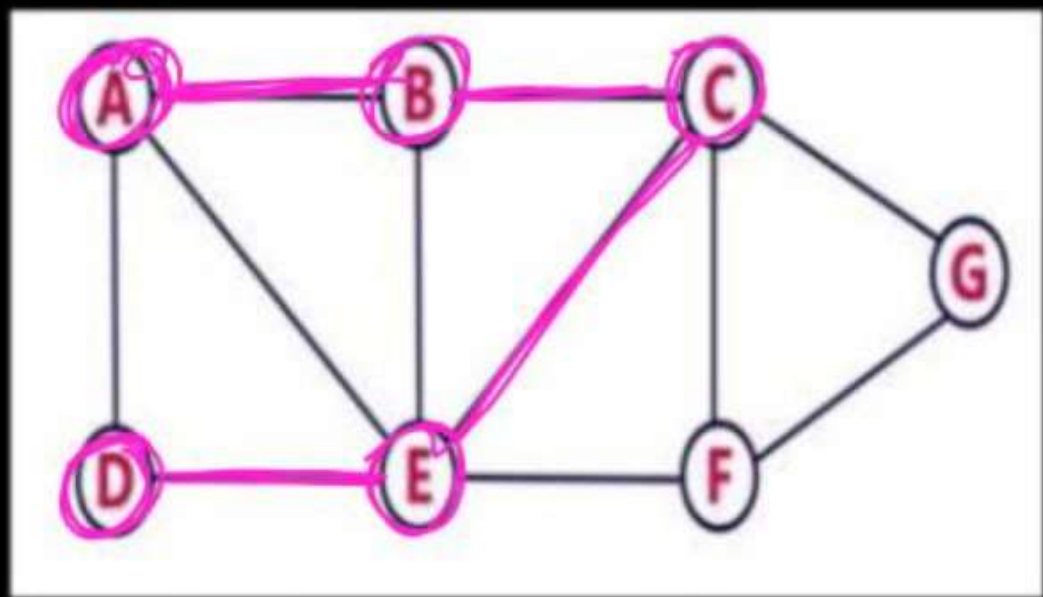
→ visit any one unvisited adjacent vertex of A and push it into the stack:



output :- A B

Status →

A	B	C	D	E	F	G
1	1					



Step-3

visit any unvisited adjacent vertex of (B)



C
B
A

output:- A B C

status-

A	B	C	D	E	F	G
1	1	1				

Step 4

E
C
B
A



output → ABCE

status -

A	B	C	D	E	F	G
1	1	1		1		

Step 5

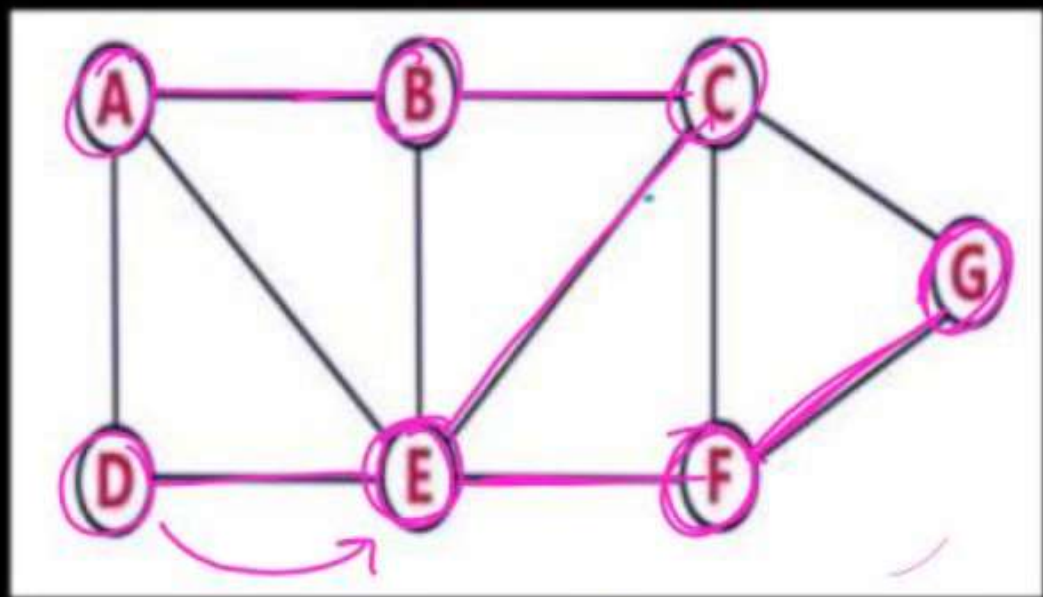
D
E
C
B
A



output:- ABCED

status →

A	B	C	D	E	F	G
1	1	1	1	1	1	



Step-6 → Now there is no unvisited adj vertex remaining of (D) so pop it from the stack ⇒ Back-Track

E
C
B
A

output:- A B C E D

status-

A	B	C	D	E	F	G
1	1	1	1	1	1	1

Step-7 - Now explore any unvisited adj vertex of (E) ⇒ (F)

F
E
C
B
A

output - A B C E D F

A	B	C	D	E	F	G
1	1	1	1	1	1	1

Step-8

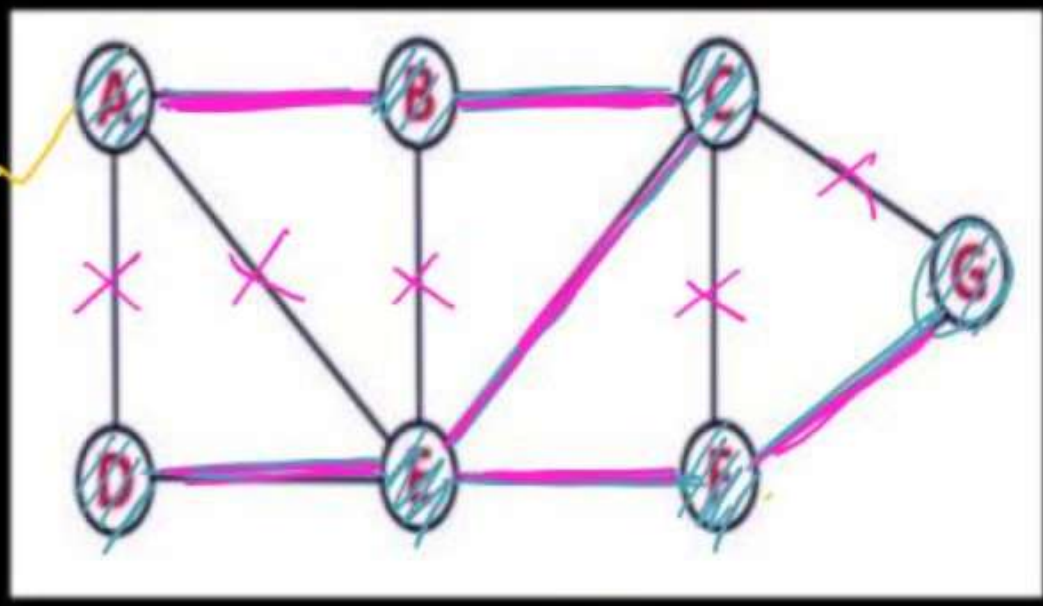
F → G

G
F
E
C
B
A

output:- A B C E D F G

status →

A	B	C	D	E	F	G
1	1	1	1	1	1	1



Step-6 →



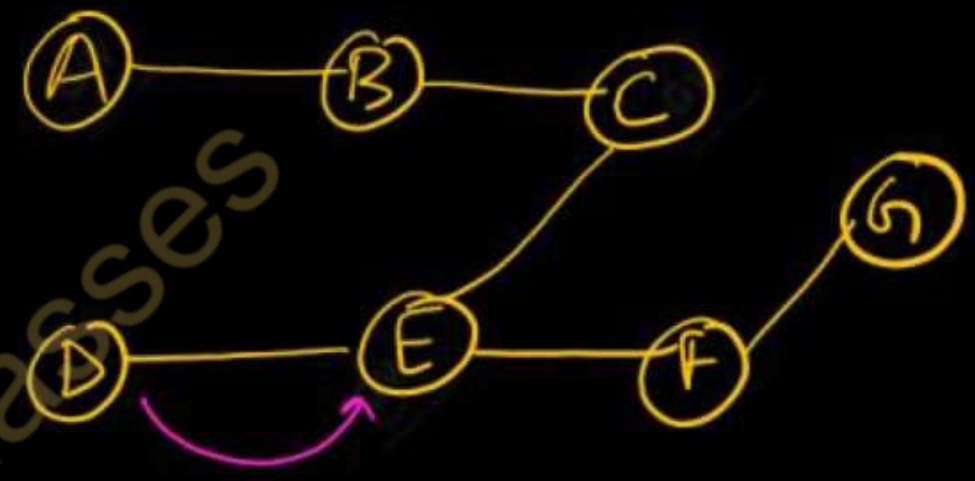
output

A B C E D F G

status

A	B	C	D	E	F	G
1	1	1	1	1	1	1

DFS Tree



Sequence

A B C E D F G

Unit-5 DPP-1

1. Explain the following graph terminologies with examples: (AKTU 2024-25)
(i) Graph (ii) Weighted Graph (iii) Degree of a Vertex (iv) Connected and Disconnected Graph
(v) Cycle in a Graph (vi) Directed and Undirected Graph (vii) MST
2. Compare adjacency matrix and adjacency list representations of graph. (AKTU 2019-20)
3. Explain two ways to represent a graph in memory and compare their advantages: (i) Adjacency matrix
(ii) Adjacency List (AKTU 2024-25, 2021-22, 2018-19)
4. Explain Depth-First Search (DFS) algorithm with help of an example graph. (AKTU 2024-25, 2018-19)
5. Define connected graph and complete graph with example. (AKTU 2018-19)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-5 Lecture-2

Today's Target

- Graph Traversal
 - Breadth First Search(BFS) ✓
- Finding Connected Components in a graph ✓
- Finding Transitive closure ✓
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

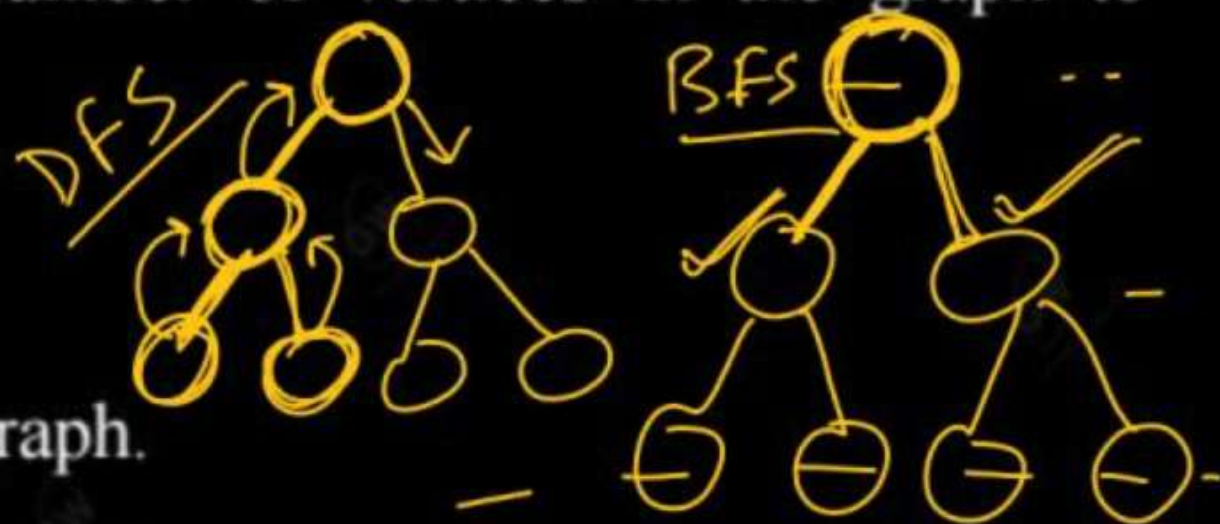
Ph.D (CSE)

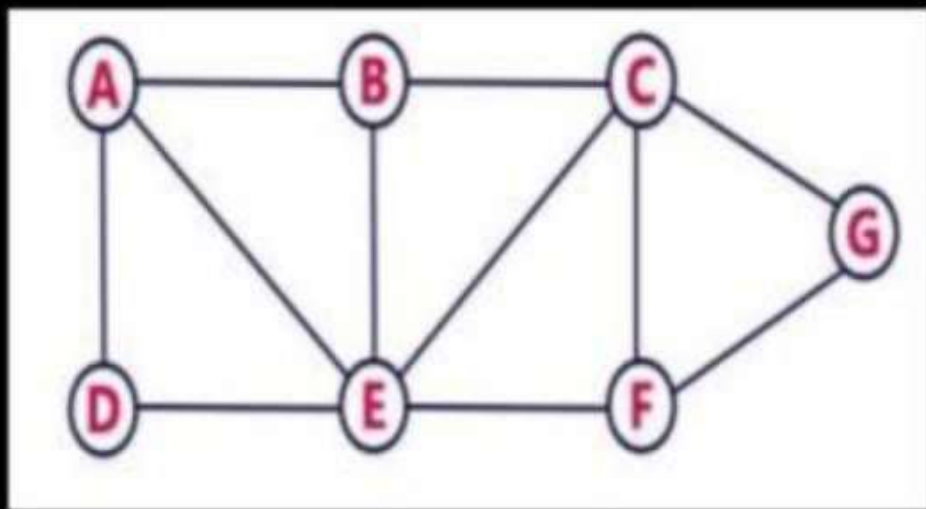
- M.Tech Gold Medalist
- Net Qualified

We use Queue data structure with maximum size of total number of vertices in the graph to implement BFS traversal.

Steps to implement BFS traversal..

- Step 1 - Define a Queue of size total number of vertices in the graph.
- Step 2 - Select any vertex as **starting point**. Visit that vertex and insert it into the Queue.
- Step 3 - Visit all non-visited **adjacent** vertices of front vertex and insert them into the Queue.
- Step 4 - When there is no new vertex to be visited from vertex which is at front of Queue then delete that vertex.
- Step 5 - Repeat steps 3 and 4 until queue becomes empty.
- Step 6 - When queue becomes empty, then produce final spanning tree by removing unused edges from graph





visited Array:-

A	B	C	D	E	F	G
0	0	0	0	0	0	0

Queue -

--	--	--	--	--	--	--

Result:- order in which keys (stored in nodes) are displayed.

Parent:-

A	B	C	D	E	F	G

Step-1 lets select A as starting vertex.

Queue -

A						
---	--	--	--	--	--	--

visited -

A	B	C	D	E	F	G
1						

Result =

Step-2 Delete A and insert all its unvisited adjacent vertices in Queue

Queue -

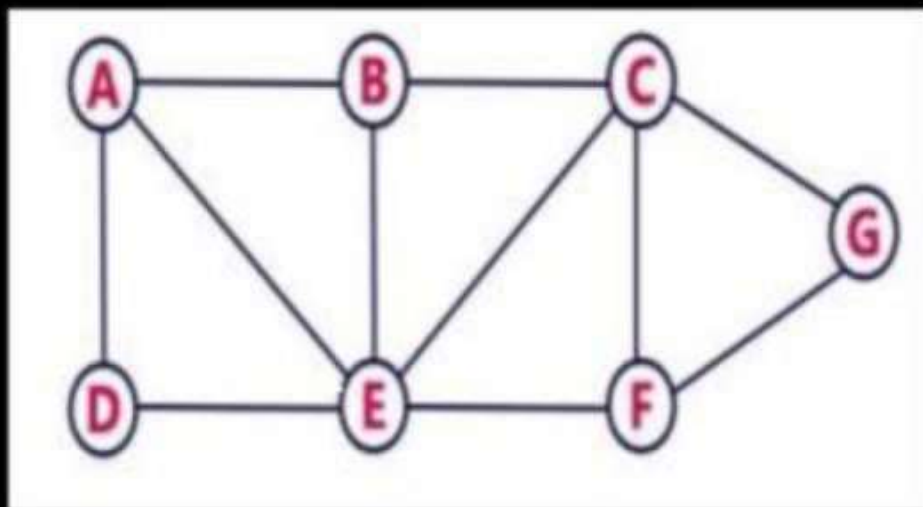
A	D	E	B			
--------------	---	---	---	--	--	--

visited -

A	B	C	D	E	F	G
1	1		1	1		

Result - A

→ used if directed graph is given.



Step-3. D has no unvisited adj. neighbour.

Queue -

A	D	E	B			
--------------	--------------	---	---	--	--	--

visited -

A	B	C	D	E	F	G
1	1		1	1		

Result $\Rightarrow$ A D

Step-4.

Queue -

A	D	E	B	C	F	
--------------	--------------	--------------	---	---	---	--

visited -

A	B	C	D	E	F	G
1	1	1	1	1	1	

Result - A D E

Step-5. B has no unvisited adj node

Queue -

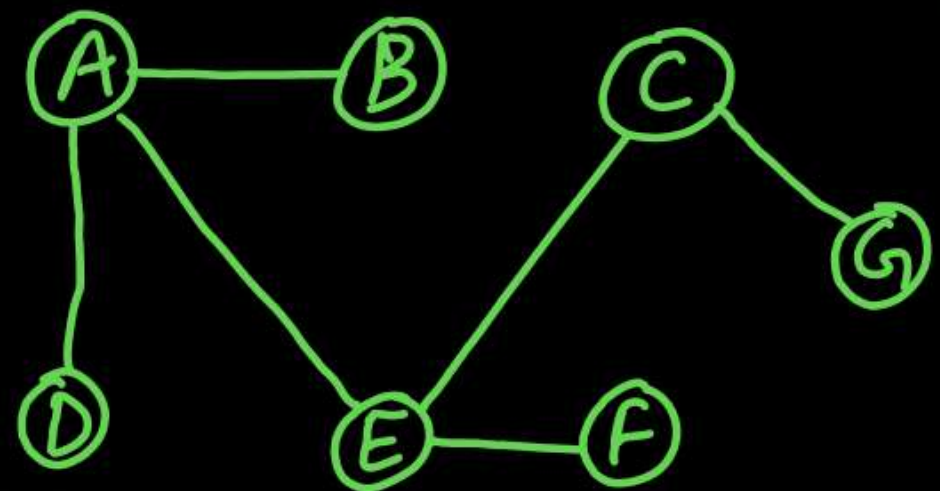
A	D	E	B	C	F	
--------------	--------------	--------------	--------------	---	---	--

visited -

A	B	C	D	E	F	G
1	1	1	1	1	1	

Result - A D E B





Step-6 C has G as unvisited adj. node

Queue -

A	D	E	B	C	F	G
--------------	--------------	--------------	--------------	--------------	---	---

visited -

A	B	C	D	E	F	G
1	1	1	1	1	1	1

Result $\Rightarrow$ A D E B C

Step-7

Queue -

A	D	E	B	C	F	G
--------------	--------------	--------------	--------------	--------------	--------------	--------------

visited -

A	B	C	D	E	F	G
1	1	1	1	1	1	1

Result - A D E B C F

Step-8

Queue -

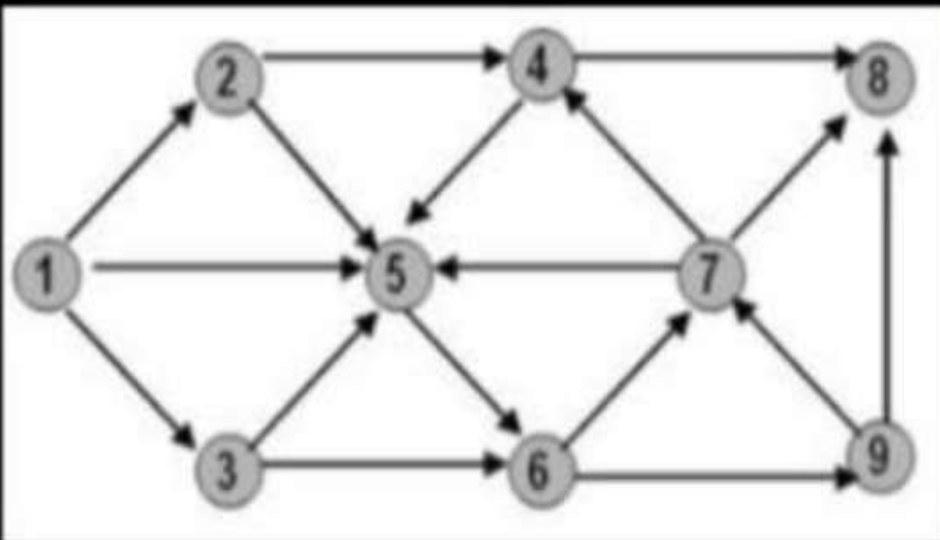
A	D	E	B	C	F	G
--------------	--------------	--------------	--------------	--------------	--------------	--------------

visited -

A	B	C	D	E	F	G
1	1	1	1	1	1	1

Result - A D E B C F G

Q: Draw the breadth First Tree for the graph. (AKTU 2021-22)



Step-1. Lets start from node ①

Queue -

1	2	3	5					
--------------	---	---	---	--	--	--	--	--

Result - 1

visited -

1	2	3	4	5	6	7	8	9
1	1	1		1				

parent -

1	2	3	4	5	6	7	8	9
0	1	1		1				

Step-2

Queue -

1	2	3	5	4	1	1	1	
--------------	--------------	---	---	---	---	---	---	--

Result - 1 2

visited -

1	2	3	4	5	6	7	8	9
1	1	1	1	1				

parent -

1	2	3	4	5	6	7	8	9
0	1	1	2	1				

Step-3

Queue -

1	2	3	5	4	6			
--------------	--------------	--------------	---	---	---	--	--	--

Result - 1 2 3

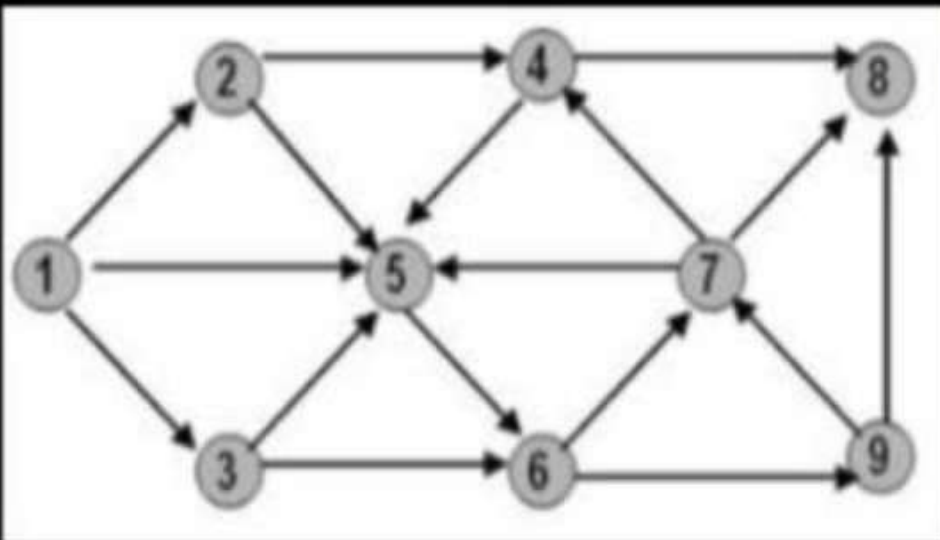
visited -

1	2	3	4	5	6	7	8	9
1	1	1	1	1	1			

parent -

1	2	3	4	5	6	7	8	9
0	1	1	2	1	3			

Q: Draw the breadth First Tree for the graph. (AKTU 2021-22)



Step-4

Queue -

1	2	3	5	4	6	8		
--------------	--------------	--------------	--------------	--------------	---	---	--	--

Result - 1 2 3 5 4

visited -

1	1	1		1	1	1		1
---	---	---	--	---	---	---	--	---

parent -

0	1	1	2	1	3		4	
---	---	---	---	---	---	--	---	--

Step-5

Queue -

1	2	3	5	4	6	8	7	9
--------------	--------------	--------------	--------------	--------------	--------------	---	---	---

Result - 1 2 3 5 4 6

visited -

1	2	3	4	5	6	7	8	9
1	1	1	1	1	1	1	1	1

parent -

1	2	3	4	5	6	7	8	9
0	1	1	2	1	3	6	4	6

Step-3

Queue -

1	2	3	5	4	6	8	7	9
--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------

Result - 1 2 3 5 4 6 8 7 9

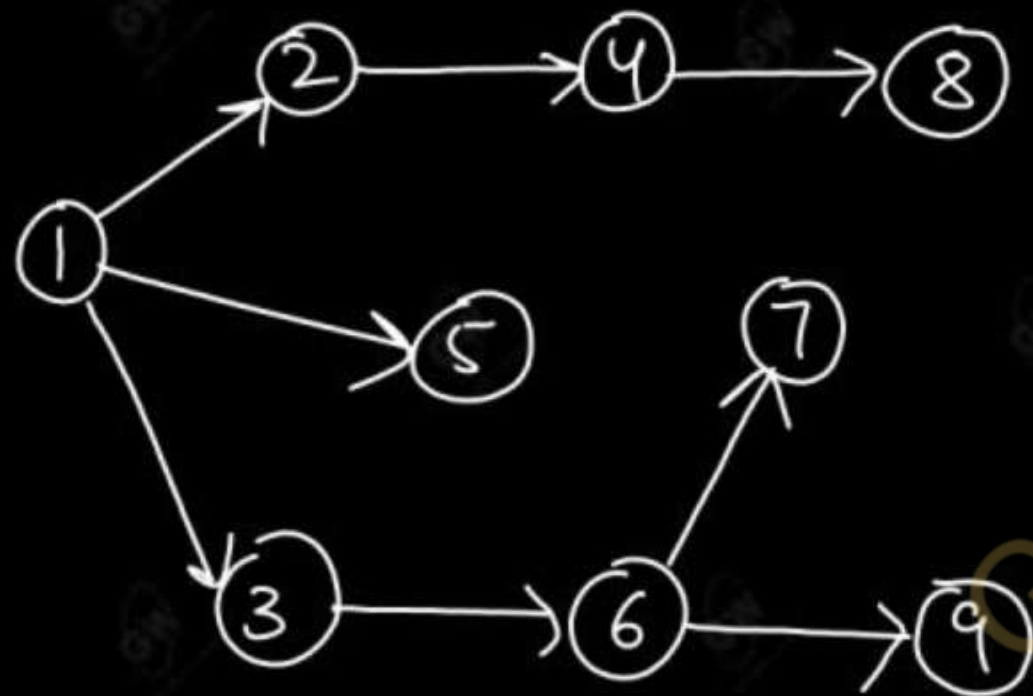
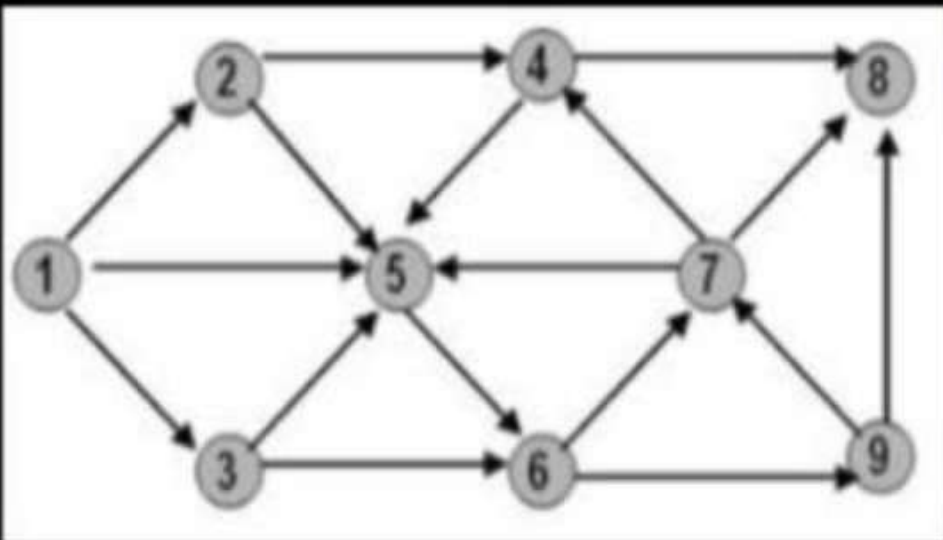
visited -

1	2	3	4	5	6	7	8	9
1	1	1	1	1	1	1	1	1

parent -

1	2	3	4	5	6	7	8	9
0	1	1	2	1	3	6	4	6

Q: Draw the breadth First Tree for the graph. (AKTU 2021-22)



BFS Tree

Queue -

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

Result -

1 2 3 5 4 6 8 7 9

visited -

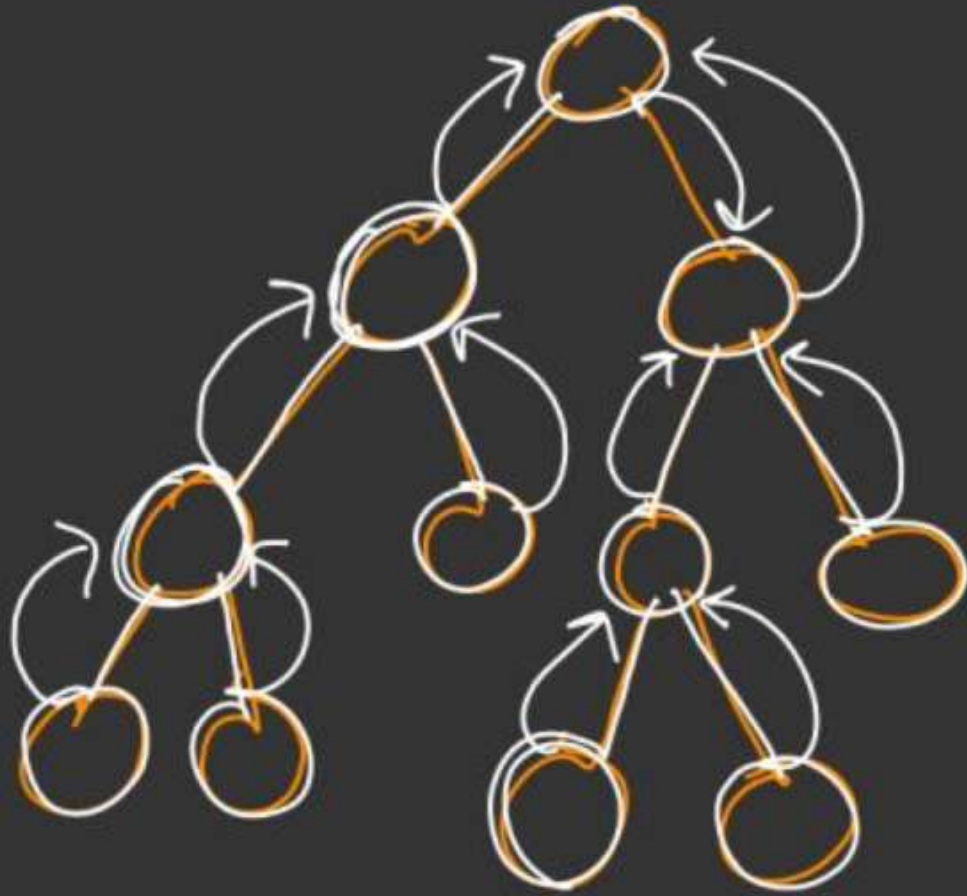
1	2	3	4	5	6	7	8	9
1	1	1	1	1	1	1	1	1

parent -

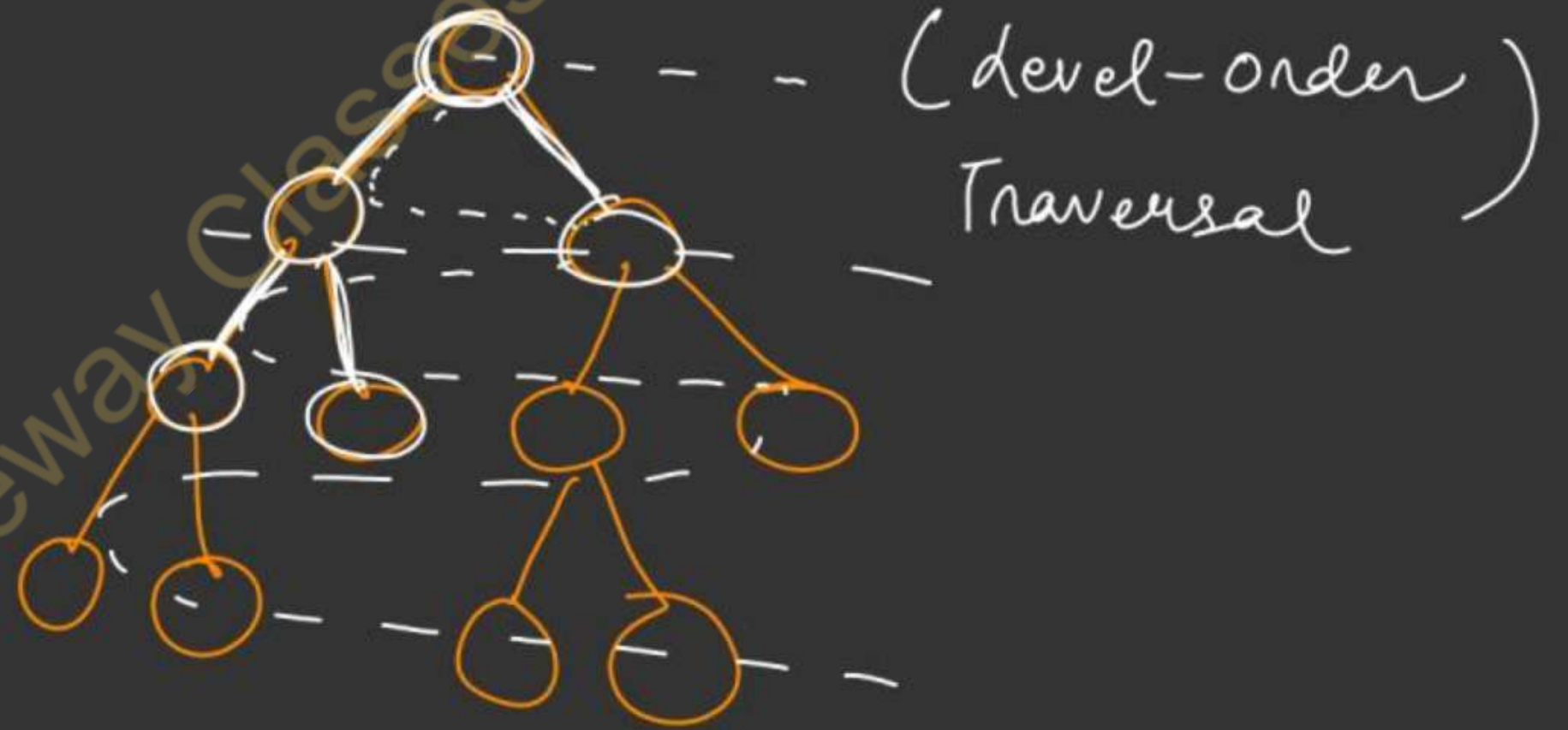
1	2	3	4	5	6	7	8	9
0	1	1	2	1	3	6	4	6

Difference b/w DFS and BFS Traversal

DFS (uses stack)



BFS (uses Queue)

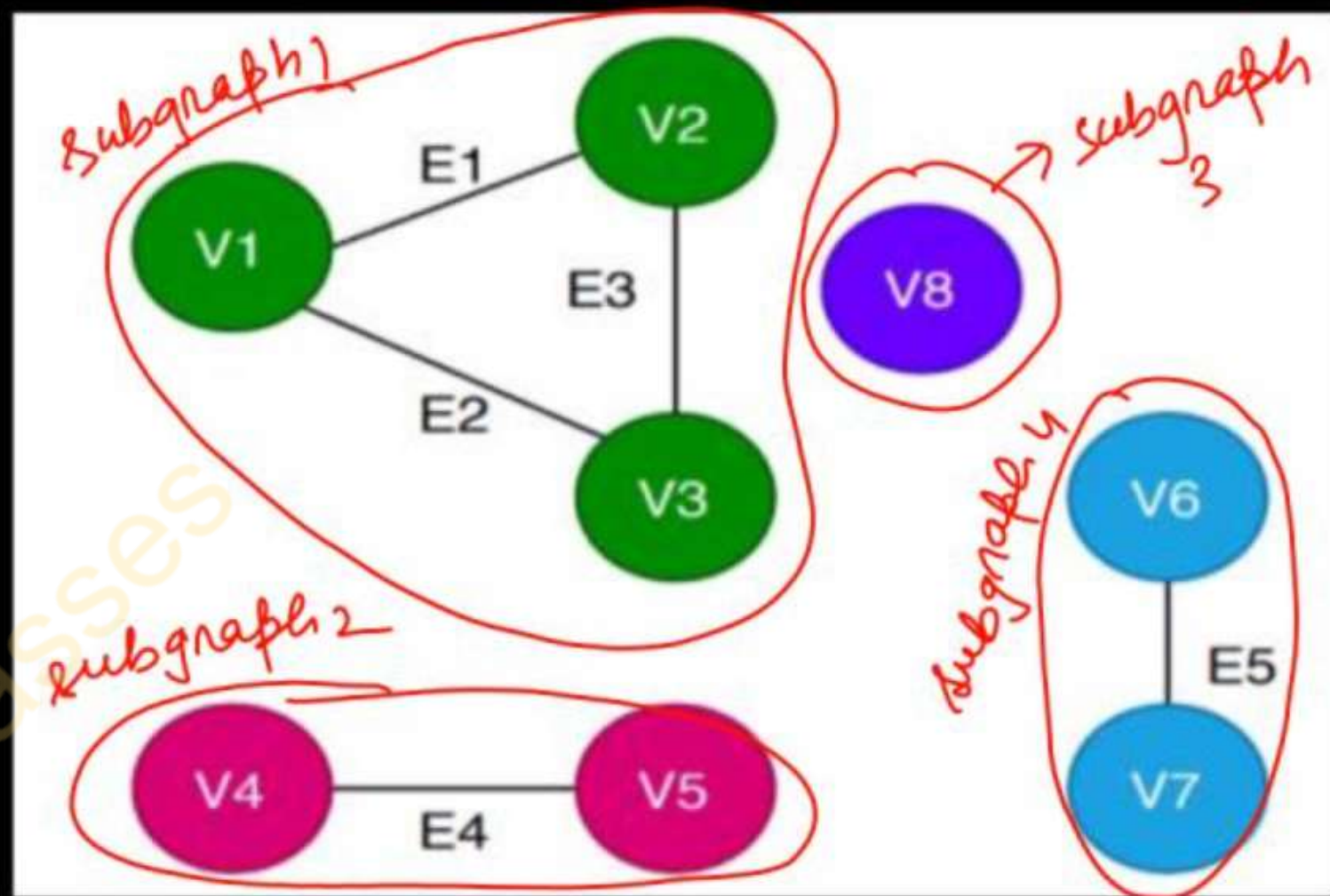


Difference between BFS and DFS (Imp)

Parameters	BFS(Breadth First Search)	DFS(Depth First Search)
Data Structure	Uses Queue for finding shortest path.	Uses Stack data structure.
Definition	We first walk through all nodes on the same level before moving on to the next level.	Traversing begins at root node and proceeds through nodes as far as possible until we reach node with no unvisited nearby nodes.
Conceptual Difference	BFS builds the tree level by level.	DFS builds the tree sub-tree by sub-tree.
Approach	It works on the concept of <u>FIFO</u>	It works on the concept of LIFO
Suitable for	More suitable for searching vertices closer to the given source.	More suitable when there are solutions away from source.
Applications	bipartite graphs, shortest paths. If weight of every edge is same, then BFS gives shortest path from source to every other vertex.	acyclic graphs and finding strongly connected components etc.

Connected Component

- A connected component of an undirected graph is a subgraph in which each pair of nodes is connected with each other via a path.
- A set of nodes forms a connected component in an undirected graph if any node from the set of nodes can reach any other node by traversing edges.
- In connected components, all the nodes are always reachable from each other.



$G(V, E)$

$V \Rightarrow V_1, V_2, V_3, V_4, V_5, V_6, V_7, V_8$

$E \Rightarrow E_1, E_2, E_3, E_4, E_5$

Pseudocode for finding Connected Components

Connected_Components(G)

$G(V, E)$

$N=6$

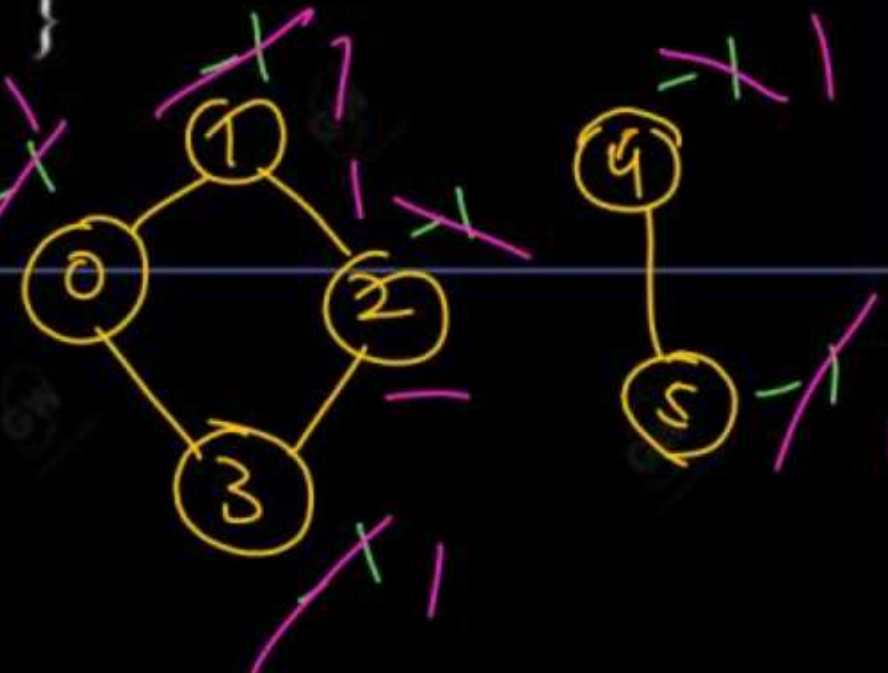
```
{
  for each vertex  $v \in N$ 
  {
    flag[v] = -1
  } // Mark all vertices as unvisited
  count = 0 // count will store required no. of
  for ( $v = 0$ ;  $v < N$ ;  $v++$ )
  {
    if (flag[v] == -1)
    {
      DFS(v, flag)
      count++
    }
  }
  print(count)
}
```

Handwritten notes:

- $N=6$ in example
- No. of vertices $\rightarrow 6$
- for all vertices
- count = 0
- count = 0 1 2 3 4
- output $\rightarrow 0 1 2 3 4$

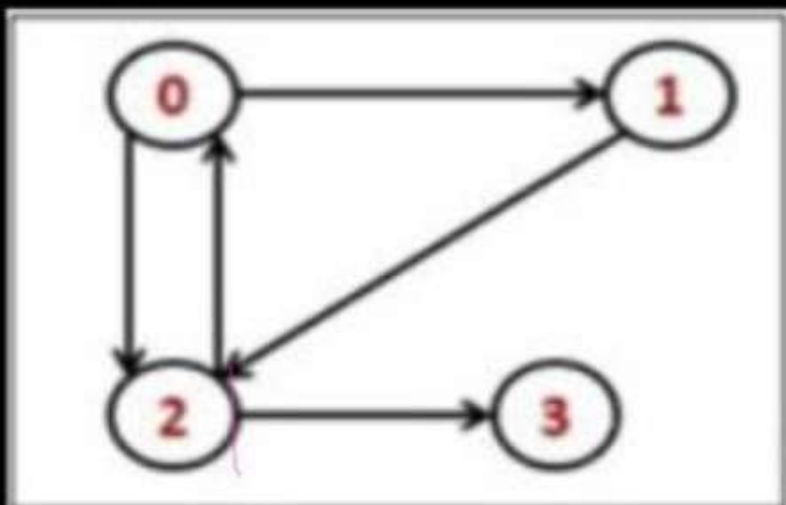
DFS(int v, int flag)

```
{
  flag[v] = 1
  print v
  for each adjacent node u of v
  {
    if (flag[u] == -1)
    {
      DFS(u, flag)
    }
  }
}
```



Transitive Closure

- Transitive Closure is the reachability matrix to reach from vertex u to vertex v of a graph.
- One graph is given, we have to find a vertex v which is reachable from another vertex u , for all vertex pairs (u, v) .
- The final matrix is the Boolean type. When there is a value 1 for vertex u to vertex v , it means that there is at least one path from u to v .



Adjacency Matrix

	0	1	2	3
0	0	1	1	0
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0

Transitive closure

Reachability Matrix

	0	1	2	3
0	1	1	1	1
1	1	1	1	1
2	1	1	1	1
3	0	0	0	1

Input: The given graph.

Output: Transitive Closure matrix.

Begin

copy the adjacency matrix into another matrix named transMat

for any vertex k in the graph, do

for each vertex i in the graph, do

for each vertex j in the graph, do

transMat[i, j] := transMat[i, j] OR (transMat[i, k] AND transMat[k, j])

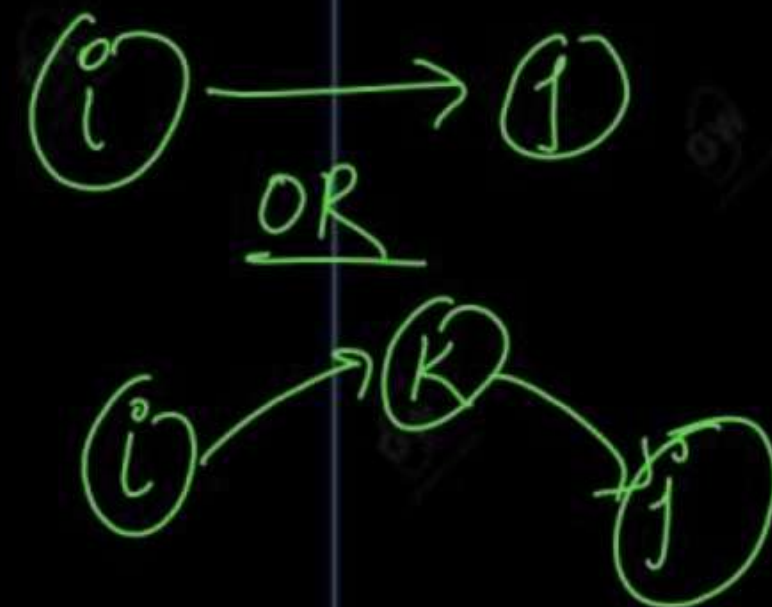
done

done

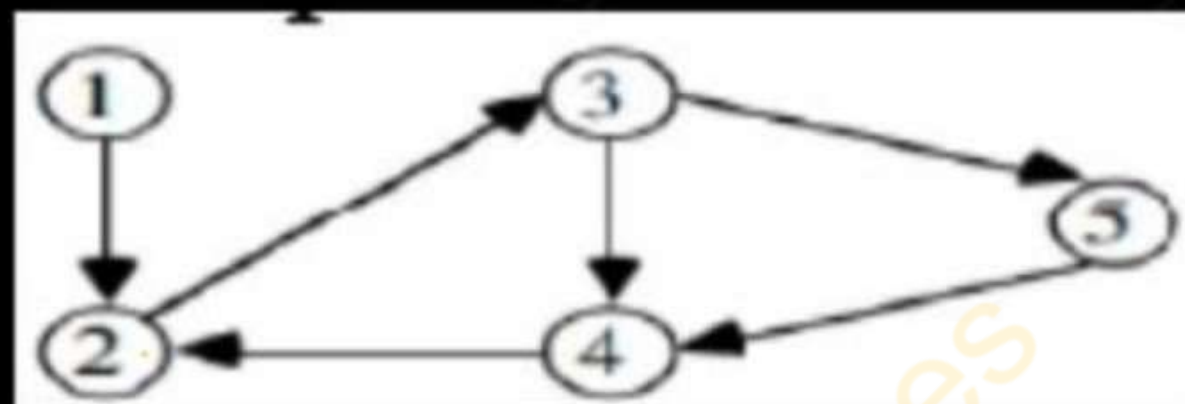
done

Display the transMat

End



Q: Compute the Transitive closure of following graph. (AKTU 2020-21)

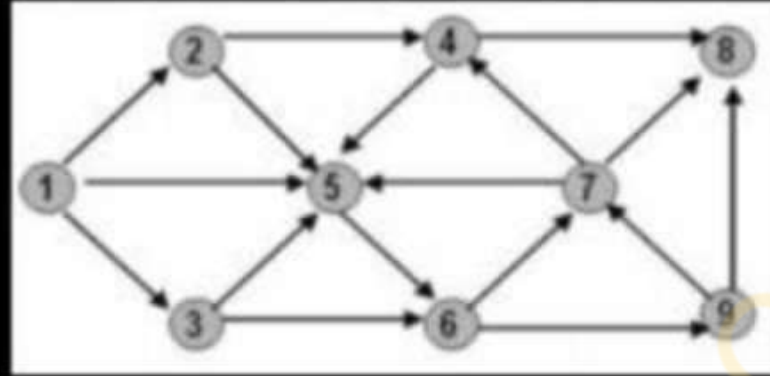


$$\begin{matrix}
 & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\
 \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix}
 1 & 1 & 1 & 1 & 1 \\
 0 & 1 & 1 & 1 & 1 \\
 0 & 1 & 1 & 1 & 1 \\
 0 & 1 & 1 & 1 & 1 \\
 0 & 1 & 1 & 1 & 1
 \end{bmatrix}
 \end{matrix}$$

Data Structure

Unit-5 DPP-2

1. Write an algorithm for Breadth First Search (BFS) traversal of a graph and explain with example. (AKTU 2022-23, 2020-21)
2. Differentiate between DFS and BFS. Draw the breadth First Tree for the graph. (AKTU 2021-22)



3. Write an algorithm to obtain the connected components of a graph. (AKTU 2023-24)
4. Compute the Transitive closure of following graph. (AKTU 2020-21)





AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-5 Lecture-3

Today's Target

- Minimum Spanning Tree
 - Prims Algorithm ✓
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

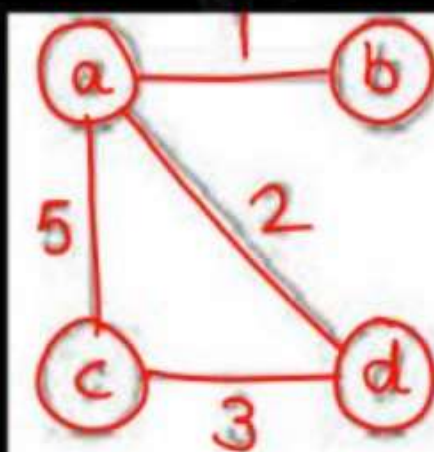
Minimum cost spanning trees

A **spanning tree** of a connected graph is its connected acyclic subgraph (i.e., a tree) that contains all the vertices of the graph.

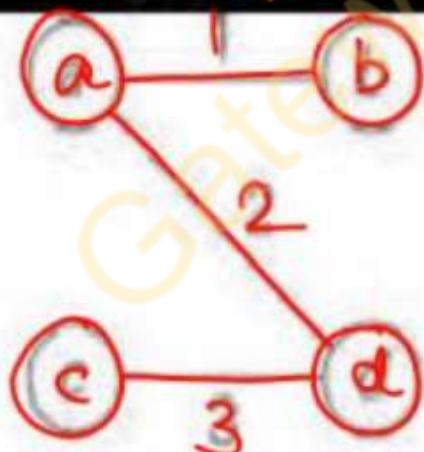
A **minimum spanning tree** of a weighted connected graph is its spanning tree of the smallest weight, where the weight of a tree is defined as the sum of the **weights** on all its edges.

The **minimum spanning tree problem** is the problem of finding a minimum spanning tree for a given weighted connected graph.

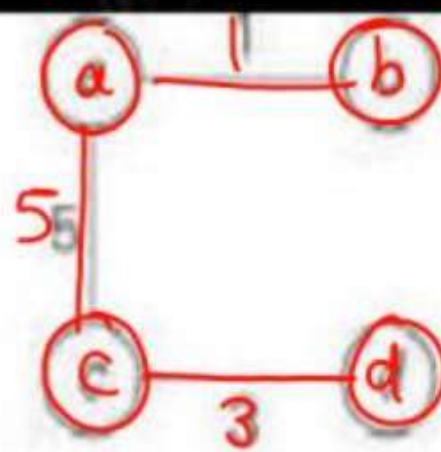
minimum cost spanning tree



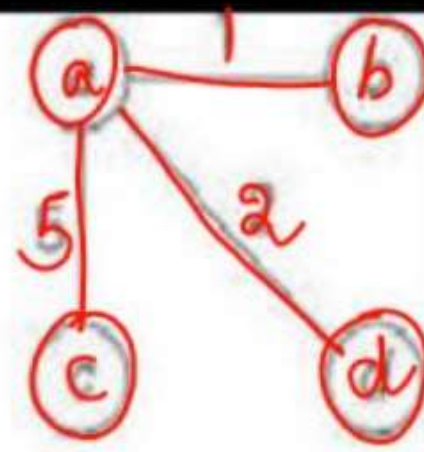
graph



$$w(T_1) = \underline{6}$$



$$w(T_2) = 9$$



$$w(T_3) = 8$$

Prim's Algorithm

- Prim's algorithm constructs a minimum spanning tree through a sequence of expanding sub-trees.
- The initial subtree consists of a single vertex selected arbitrarily from the set V of the graph's vertices.
- On each iteration it expands the current tree in the greedy manner by simply attaching to it the nearest vertex not in that tree.
- The algorithm stops after all the graph's vertices have been included in the tree being constructed.
- Since the algorithm expands a tree by exactly one vertex on each of its iterations, the total number of such iterations is $n - 1$, where n is the number of vertices in the graph.

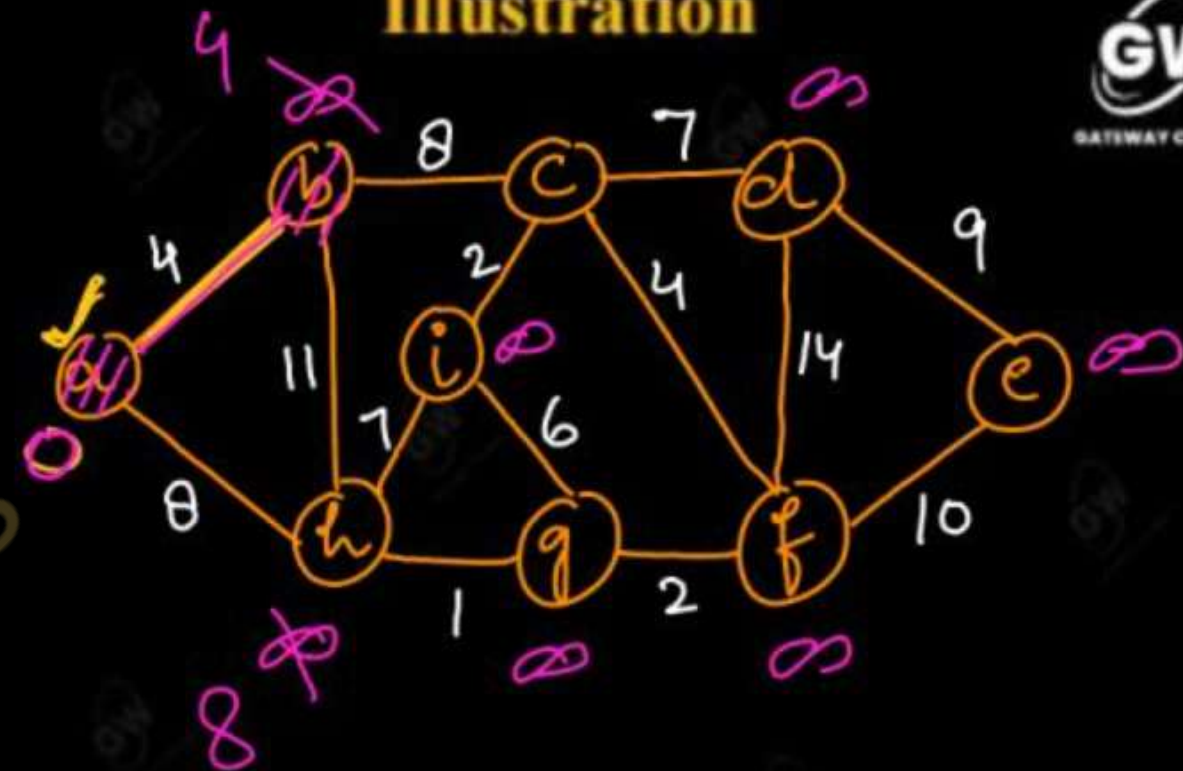
Remaining Vertices in Q

$a(-, 0)$	$e(-, \infty)$
$b(-, \infty)$	$f(-, \infty)$
$c(-, \infty)$	$g(-, \infty)$
$d(-, \infty)$	$h(-, \infty)$
	$i(-, \infty)$

$a(-, 0)$

$b(a, 4)$	$e(-, \infty)$
$c(-, \infty)$	$f(-, \infty)$
$d(-, \infty)$	<u>$h(a, 8)$</u>
$g(-, \infty)$	$i(-, \infty)$

Illustration



vertex (parent, cost)

Remaining Vertices in Q

$C(b, 8)$
 $d(-, \infty)$
 $e(-, \infty)$
 $h(a, 8)$
 $g(-, \infty)$
 $i(-, \infty)$

$C(\underline{b}, 8)$

$d(c, 7)$
 $h(a, 8)$

$e(-, \infty)$
 $g(-, \infty)$

$f(c, 4)$
 $i(\underline{c}, 2)$

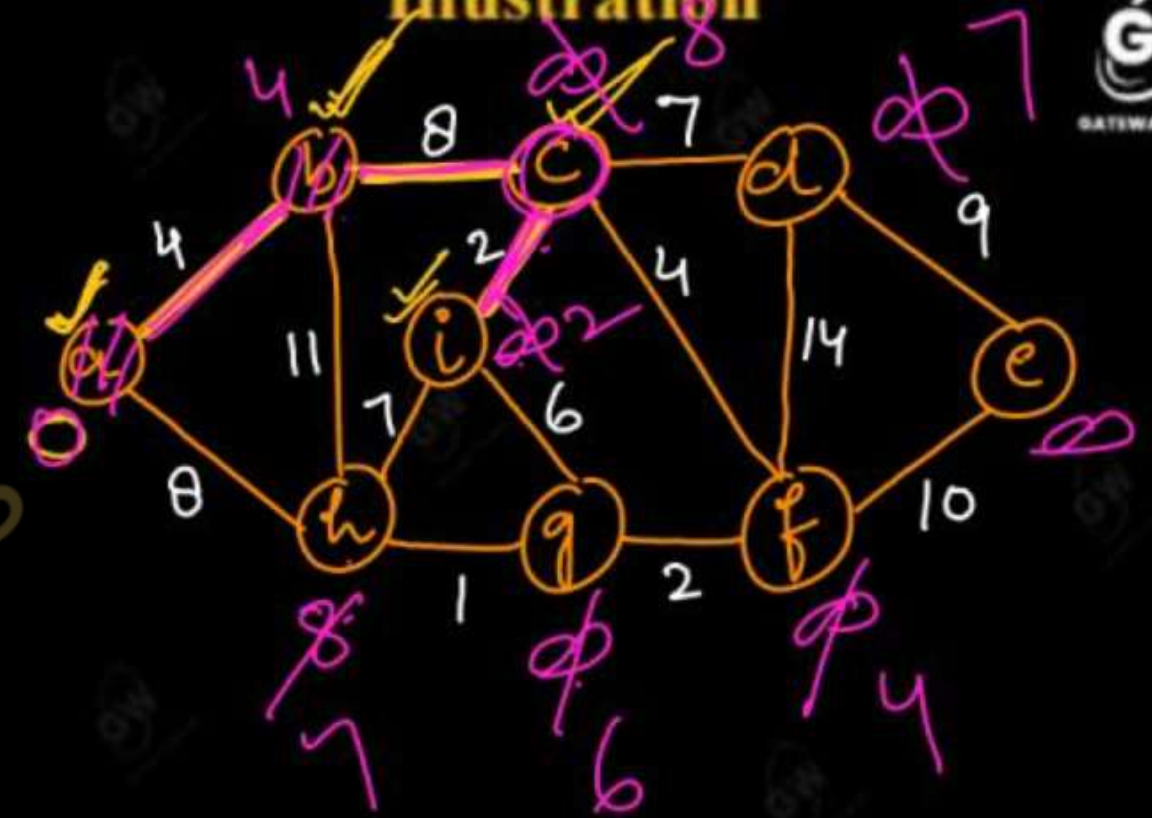
$i(c, 2)$

$d(c, 7)$
 $h(i, 7)$

$e(-, \infty)$
 $g(i, 6)$

$f(c, 4)$

Illustration



$f(c, 4)$

to $\uparrow$ from $\uparrow$
 $g(f, 2)$

$h(g, 1)$

$d(c, 7)$

$e(d, 9)$

Remaining Vertices in Q

$d(c, 7)$

$h(i, 7)$

$e(f, 10)$

$g(f, 2)$

$d(c, 7)$

$h(g, 1)$

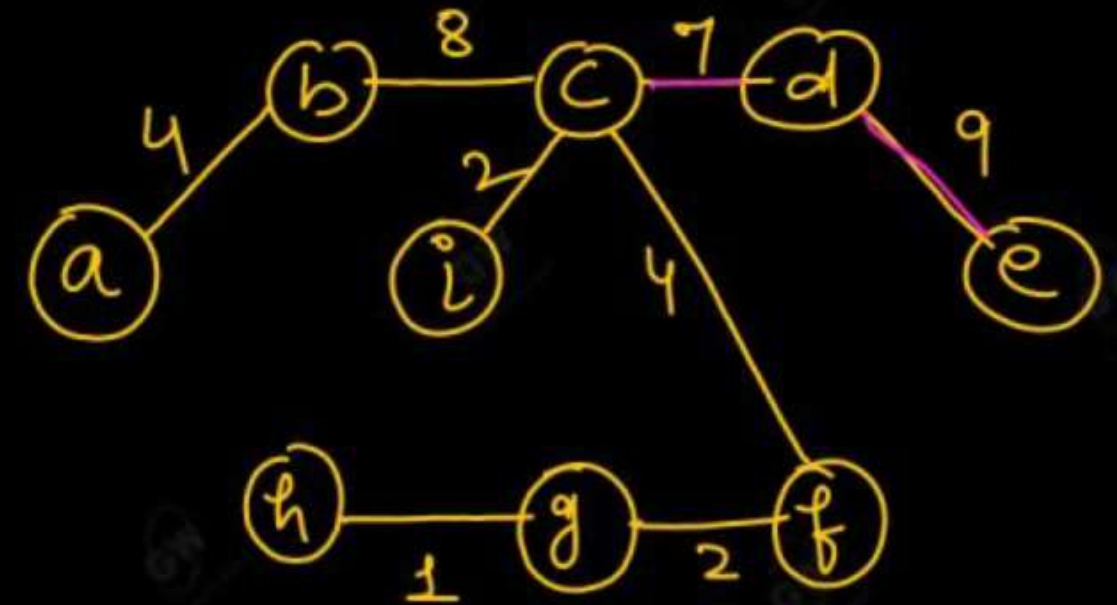
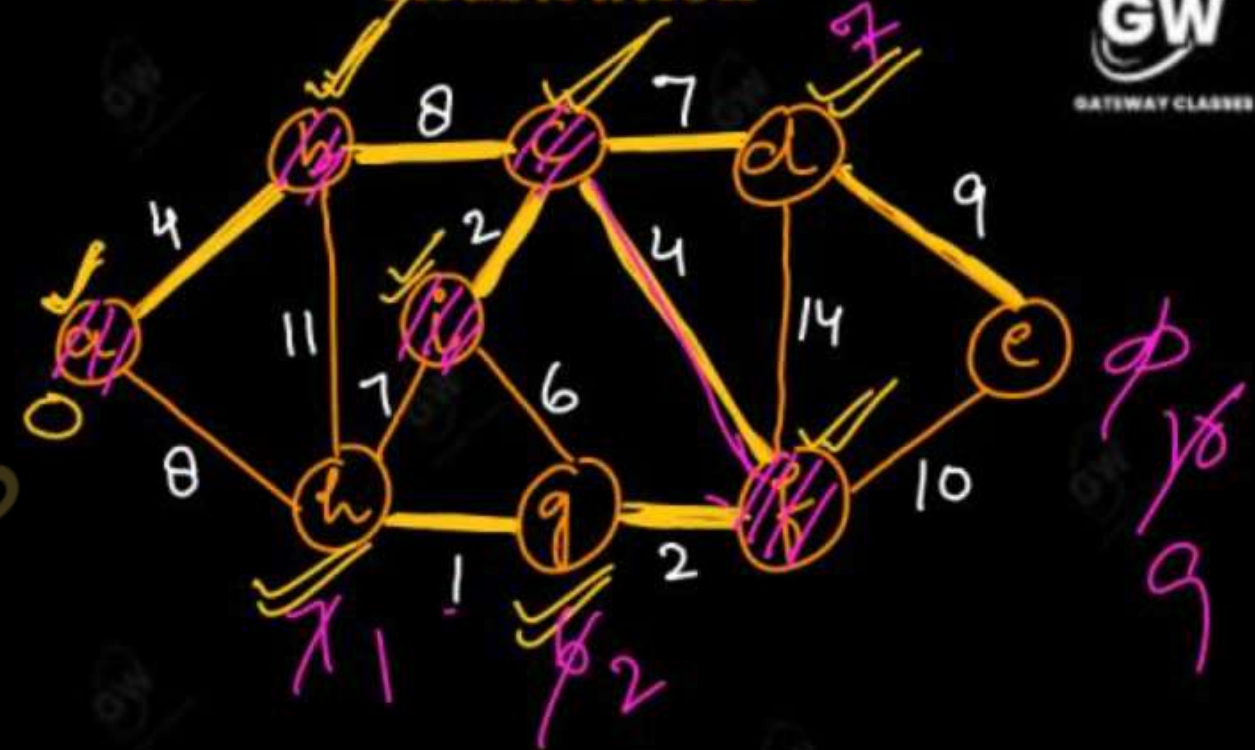
$e(f, 10)$

$d(c, 7)$

$e(f, 10)$

$e(d, 9)$

Illustration



Total cost = $4 + 8 + 7 + 9 + 2 + 4 + 1 + 2$
of MST

Q: Find the minimum spanning tree using Prim's algorithm for the graph shown below: -

(AKTU 2023-24)

Tree Vertices

Remaining Vertices in Q

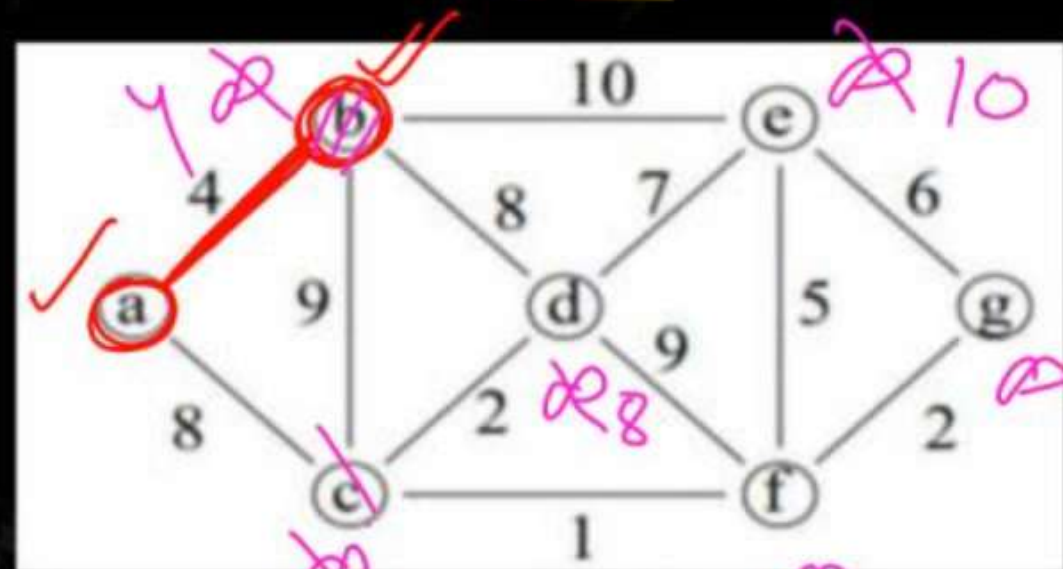
$a(-, 0)$	$e(-, \infty)$
$b(-, \infty)$	$f(-, \infty)$
$c(-, \infty)$	$g(-, \infty)$
$d(-, \infty)$	

$a(-, 0)$

$b(a, 4)$	$e(-, \infty)$
$c(a, 8)$	$f(-, \infty)$
$d(-, \infty)$	$g(-, \infty)$

$b(a, 4)$

$c(a, 8)$	$e(b, 10)$	$g(-, \infty)$
$d(b, 8)$	$f(-, \infty)$	



Q: Find the minimum spanning tree using Prim's algorithm for the graph shown below: -

(AKTU 2023-24)

Tree Vertices

$c(a, 8)$

$f(c, 1)$

$d(c, 2)$

$g(f, 2)$

$e(f, 5)$

Remaining Vertices in Q

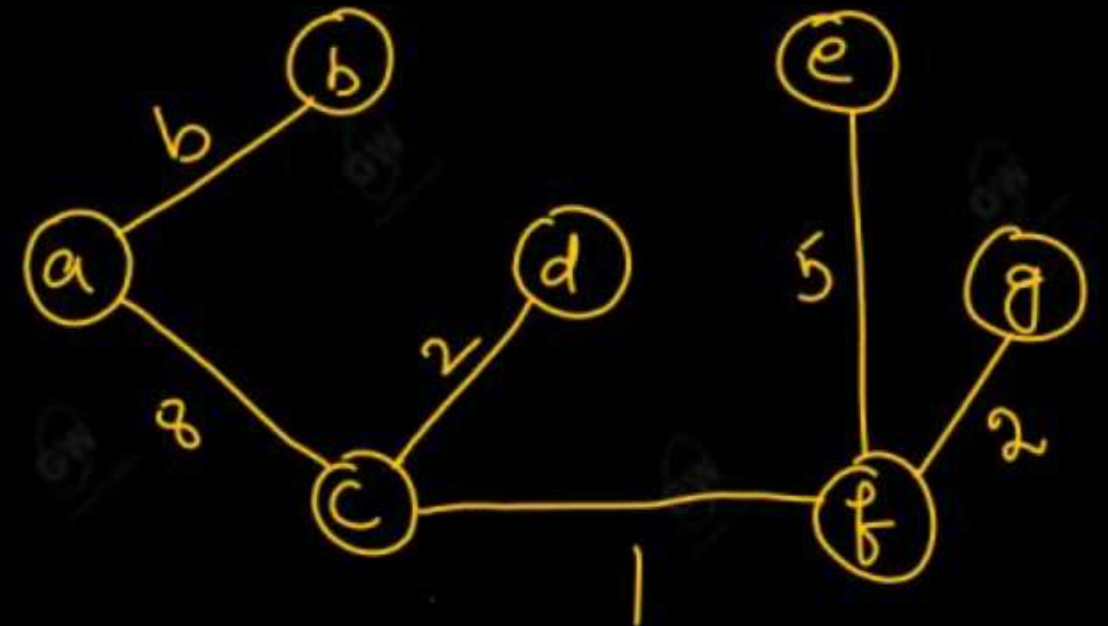
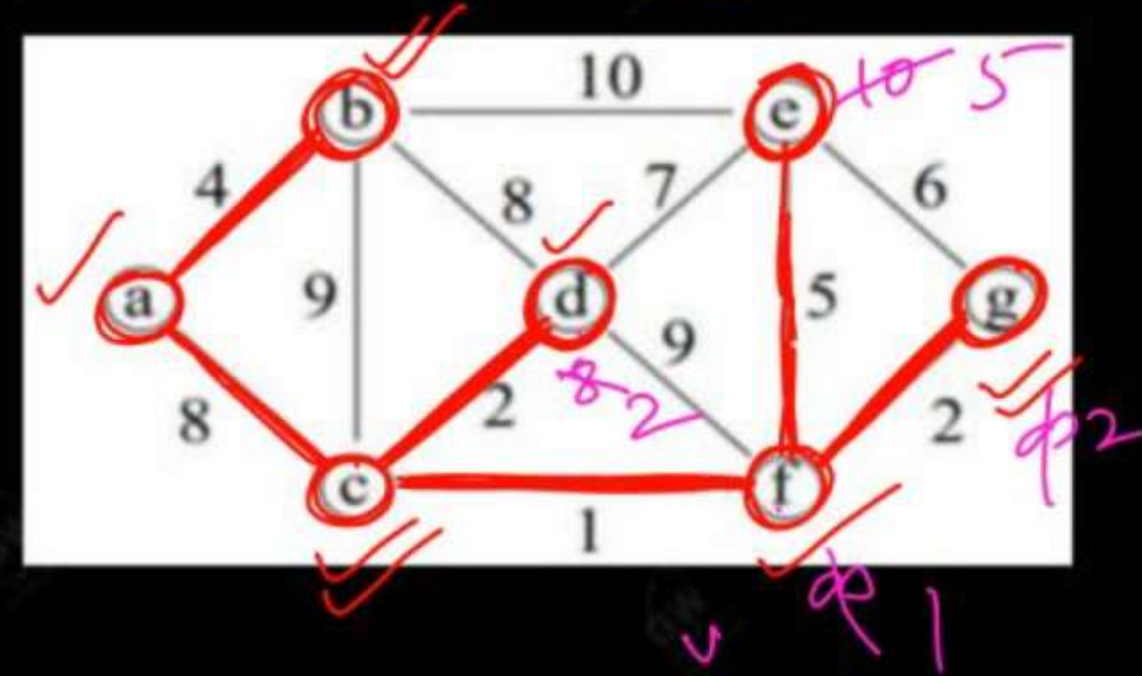
$d(c, 2)$ $f(c, 1)$
 $e(b, 10)$ $g(-, \infty)$

$d(c, 2)$ $g(f, 2)$

$e(f, 5)$

$g(f, 2)$ $e(f, 5)$

$e(f, 5)$



MST-PRIM(G, w, r) → graph → initial vertex

```

1  for each  $u \in V[G]$  // for all vertices in a graph
2  do  $key[u] \leftarrow \infty$  // set cost of each vertex  $\Rightarrow \infty$ 
3  parent of u  $\leftarrow \pi[u] \leftarrow \text{NIL}$  // set parent of each vertex  $\Rightarrow \text{Nil}$ 
4   $key[r] \leftarrow 0$  // set cost of initial vertex  $\Rightarrow 0$ 
5   $Q \leftarrow V[G]$  // Add all the vertices in a priority Queue.
6  while  $Q \neq \emptyset$  // Repeat untill Q becomes empty
7  do  $u \leftarrow \text{EXTRACT-MIN}(Q)$  // extract min-cost vertex from Q
8  for each  $v \in \text{Adj}[u]$  // for all adj vertices
9  do if  $v \in Q$  and  $w(u, v) < key[v]$ 
10 vertices still in Q then  $\pi[v] \leftarrow u$ 
11  $key[v] \leftarrow w(u, v)$ 

```



Correctness: Prim's algorithm always yields a minimum spanning tree.

Time Complexity: $O(|E| \log |V|)$

Q: What is spanning tree? Write down the Prim's algorithm to obtain minimum cost spanning tree. Use Prim's algorithm to find the minimum cost spanning tree in the following graph:

(AKTU 2022-23)

Tree vertices

Remaining Vertices in Q

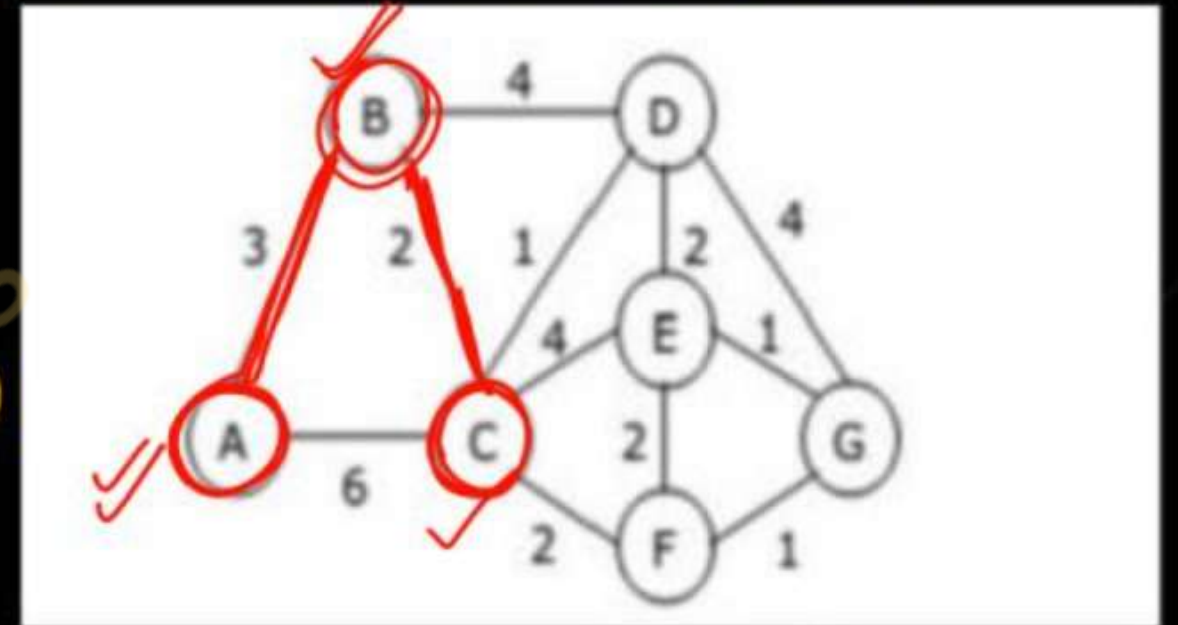
$A(-, 0)$	$D(-, \infty)$	$G(-, \infty)$
$B(-, \infty)$	$E(-, \infty)$	
$C(-, \infty)$	$F(-, \infty)$	

$A(-, 0)$

$B(A, 3)$	$D(-, \infty)$	$F(-, \infty)$
$C(A, 6)$	$E(-, \infty)$	$G(-, \infty)$

$B(A, 3)$

$C(B, 2)$	$E(-, \infty)$	$G(-, \infty)$
$D(B, 4)$	$F(-, \infty)$	



Q: What is spanning tree? Write down the Prim's algorithm to obtain minimum cost spanning tree. Use Prim's algorithm to find the minimum cost spanning tree in the following graph:

(AKTU 2022-23)

Tree vertices

$C(B, 2)$

$D(C, 1)$

$E(D, 2)$

$G(E, 1)$

$F(G, 1)$

Remaining Vertices in Q

$D(C, 1)$

$E(C, 4)$

$E(D, 2)$

$F(C, 2)$

$F(G, 1)$

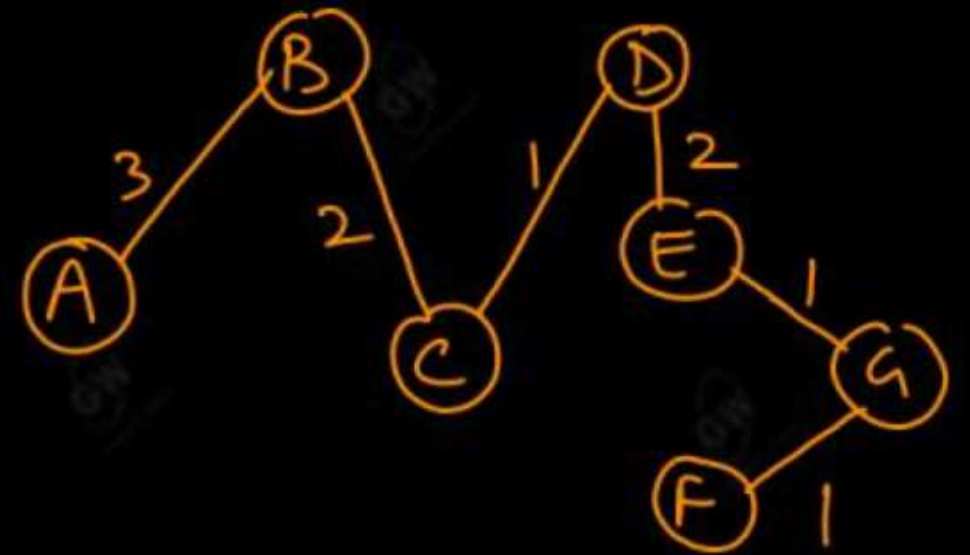
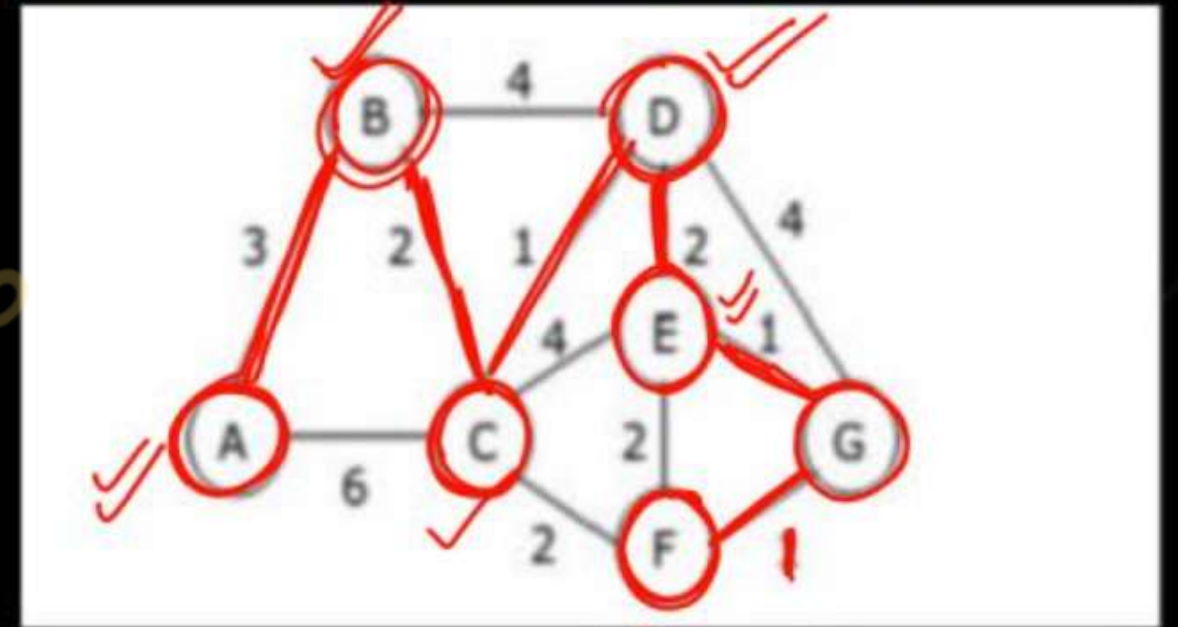
$F(C, 2)$

$G(-, \infty)$

$F(C, 2)$

$G(E, 1)$

$G(D, 4)$



Q: Construct minimum spanning tree for the below given graph using Prim's algorithm

(Source node - a).

(AKTU 2017-18)

Tree Vertices

Remaining vertices in Q

$a(-, 0)$ $c(-, \infty)$ $e(-, \infty)$
 $b(-, \infty)$ $d(-, \infty)$

$a(-, 0)$

$b(a, 3)$ $d(a, 5)$
 $c(a, 1)$ $e(a, 8)$

$c(a, 1)$

$b(a, 3)$ $d(c, 4)$ $e(c, 2)$

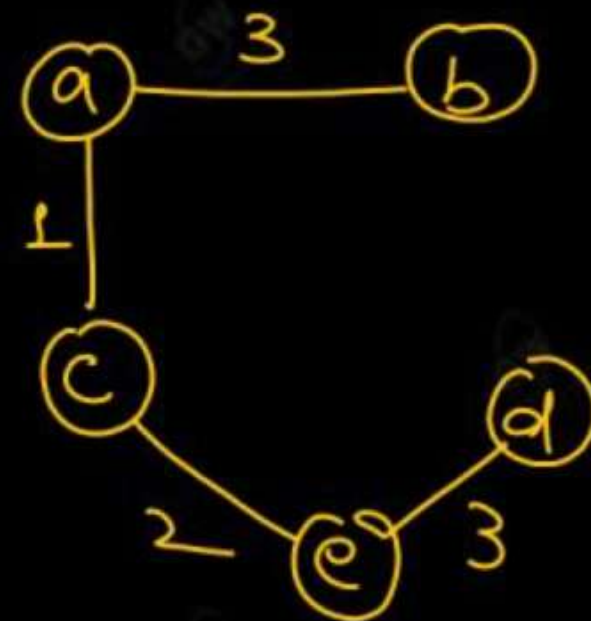
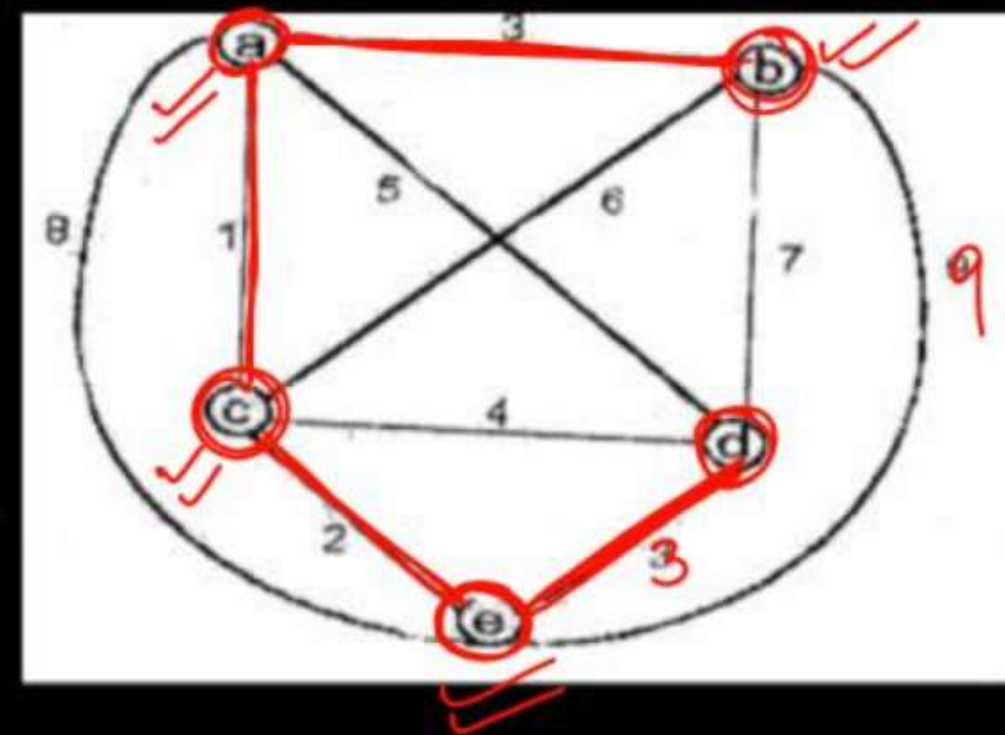
$e(c, 2)$

$b(a, 3)$ $d(e, 3)$

$b(a, 3)$

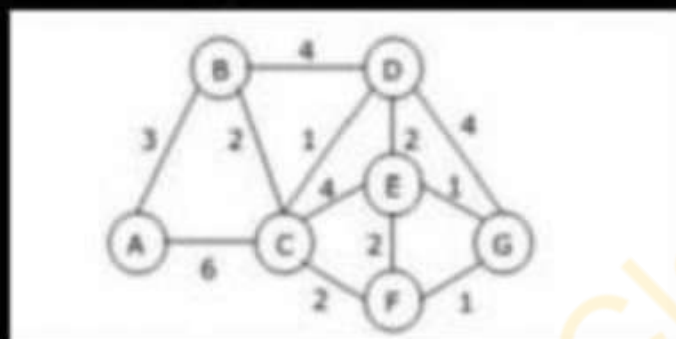
$d(e, 3)$

$d(e, 3)$

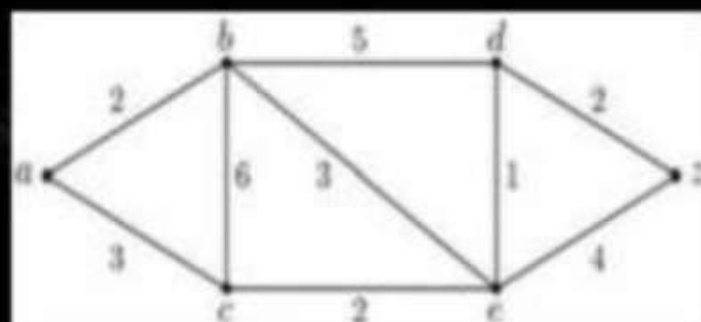


1. What is spanning tree? Write down the Prim's algorithm to obtain minimum cost spanning tree. Use Prim's algorithm to find the minimum cost spanning tree in the following graph:

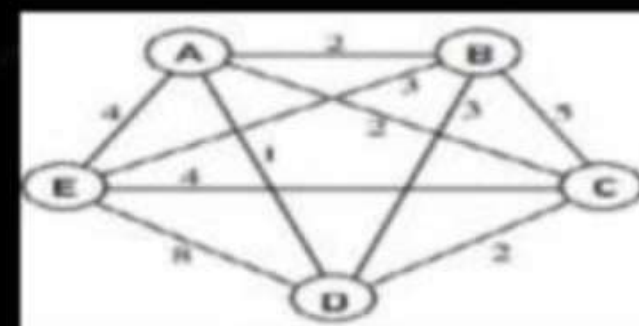
(AKTU 2022-23)



2. Apply Prim's algorithm to find a minimum spanning tree. (AKTU 2021-22)



3. Describe Prim's algo and find cost of minimum spanning tree using Prim's Algorithm (AKTU 2020-21)





AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-5 Lecture-4

Today's Target

- Minimum Spanning Tree
 - Kruskal Algorithm
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

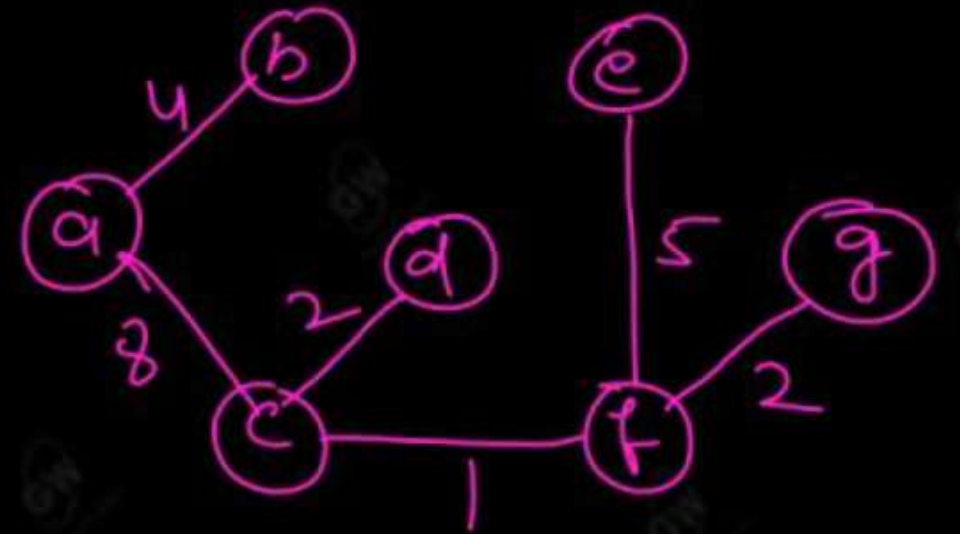
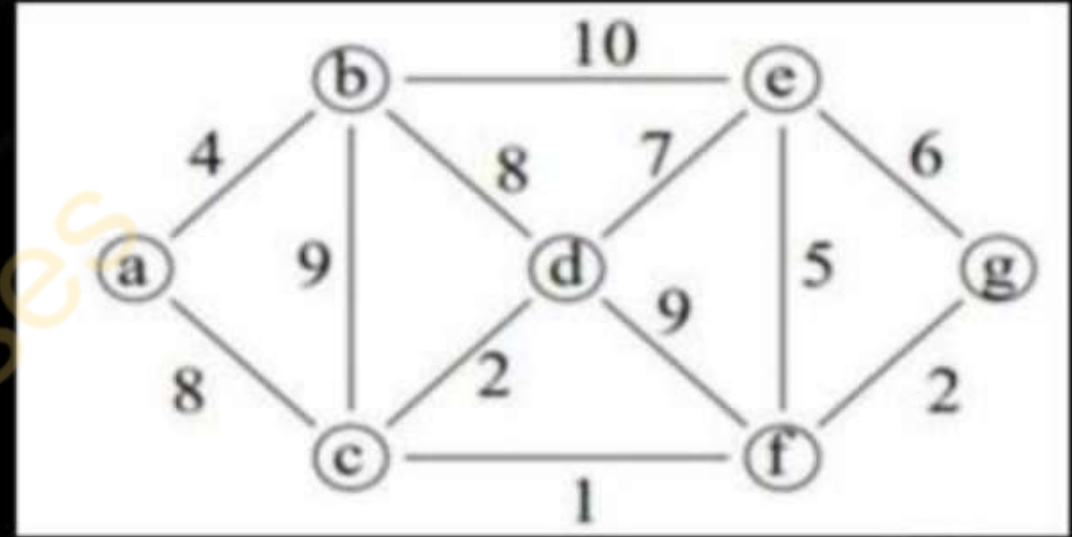
Kruskal's Algorithm

- It is a greedy algorithm for the MST problem that always yields an optimal solution.
- To apply Kruskal's algorithm, the given graph must be weighted, connected and undirected.
- It begins by **sorting** the graph's edges in **non-decreasing** order of their **weights**.
- Starting with the empty subgraph, it adds the next edge to the current sub graph if it **does not create a cycle or skip the edge otherwise**.
- On each iteration, the algorithm takes the next edge (u, v) from the sorted list of the graph's edges, finds the trees containing the vertices u and v , and, if these trees are not the same, unites them in a larger tree by adding the edge (u, v) .

Q: Find the minimum spanning tree using Kruskal's algorithm for the graph shown below: -
(AKTU 2023-24, 2019-20)

set of vertices

$\{a\} \{b\} \{c\} \{d\} \{e\} \{f\} \{g\}$
 $\{a\} \{b\} \{c, f\} \{d\} \{e\} \{g\}$
 $\{a\} \{b\} \{c, f, d\} \{e\} \{g\}$
 $\{a\} \{b\} \{c, f, d, g\} \{e\}$
 $\{a, b\} \{c, f, d, g\} \{e\}$
 $\{a, b\} \{c, f, d, g, e\}$



cost of MST $\Rightarrow 4 + 8 + 2 + 1 + 5 + 2$
 $= 22$

vertices are not present in disjoint sets
 so we do not consider these edges.

$\{a, b, c, f, d, g, e\}$

edge cost.

$\checkmark (c, f) \rightarrow 1$

$\checkmark (c, d) \rightarrow 2$

$\checkmark (f, g) \rightarrow 2$

$\checkmark (a, b) \rightarrow 4$

$\checkmark (e, f) \rightarrow 5$

$\times (e, g) \rightarrow 6$

$\times (d, e) \rightarrow 7$

$\checkmark (a, c) \rightarrow 8$

$\times (b, d) \rightarrow 8$

$\times (b, c) \rightarrow 9$

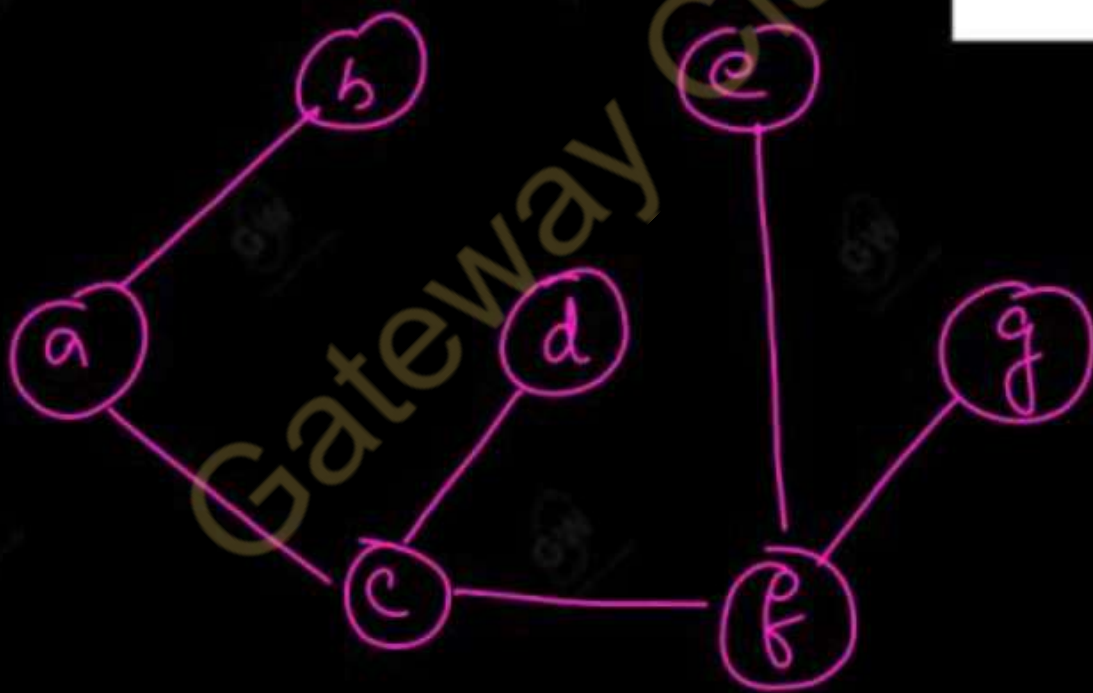
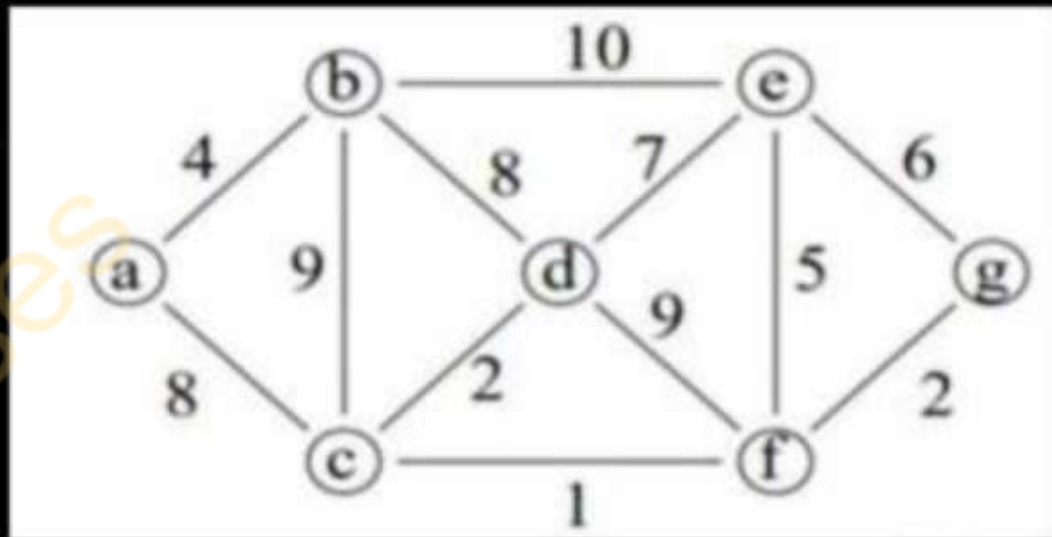
$\times (b, e) \rightarrow 10$

Q: Find the minimum spanning tree using Kruskal's algorithm for the graph shown below: -

(AKTU 2023-24, 2019-20)

edge cost.

direct Method



- ✓ $(c, f) \rightarrow 1$
- ✓ $(c, d) \rightarrow 2$
- ✓ $(f, g) \rightarrow 2$
- ✓ $(a, b) \rightarrow 4$
- ✓ $(e, f) \rightarrow 5$
- ✗ $(e, g) \rightarrow 6$
- ✗ $(d, e) \rightarrow 7$
- ✓ $(a, c) \rightarrow 8$
- ✗ $(b, d) \rightarrow 8$
- ✗ $(b, c) \rightarrow 9$
- ✗ $(b, e) \rightarrow 10$

MST-KRUSKAL(G, w)

```

1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$  // for each vertex in graph
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) // if  $u$  and  $v$  belongs to diff. sets
7           $A = A \cup \{(u, v)\}$  // include this edge  $(u, v)$  into MST
8          UNION( $u, v$ ) // perform union.
9  return  $A$ 

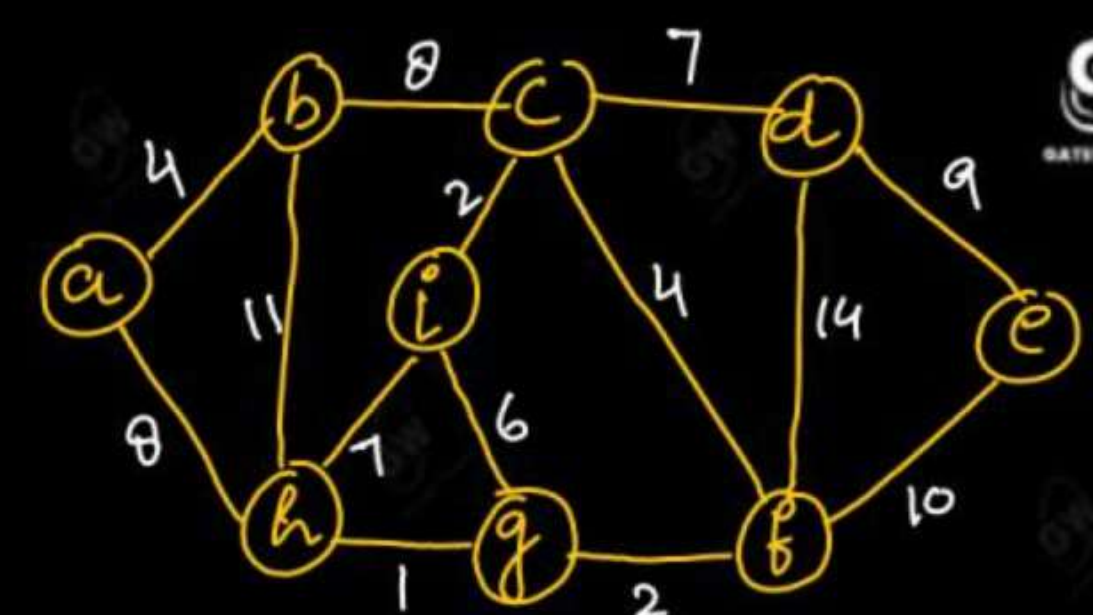
```

Kruskal's

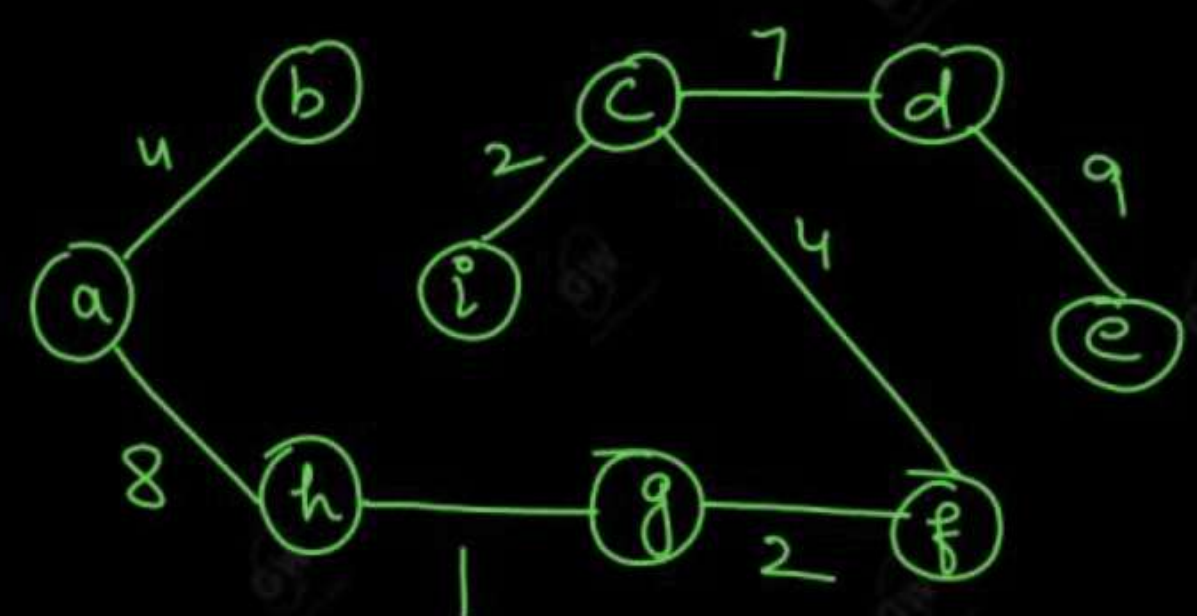
Correctness: ~~Prim's~~ algorithm always yields a minimum spanning tree.

Time Complexity: $O(|E| \log |E|)$ or $O(|E| \log |V|)$

<u>Edges</u>	<u>cost</u>	
✓ (h, g) → 1		{a}{b}{c}{d}{e}{f}{g}{h}{i}
✓ (g, f) → 2		{a}{b}{c}{d}{e}{f}{g, h}{i}
✓ (i, c) → 2		{a}{b}{c}{d}{e}{f, g, h}{i}
✓ (a, b) → 4		{a, b}{c, i}{d}{e}{f, g, h}
✓ (c, f) → 4		{a, b}{c, i}{d}{e}{f, g, h}
X (g, i) → 6	X	{a, b}{c, i, f, g, h}{d}{e}
✓ (c, d) → 7		{a, b}{c, i, f, g, h, d}{e}
X (h, i) → 7	X	{a, b, c, i, f, g, h, d}{e}
✓ (a, h) → 8		{a, b, c, i, f, g, h, d}{e}
X (b, c) → 8	X	{a, b, c, i, f, g, h, d, e}
✓ (d, e) → 9		
X (e, f) → 10	X	
X (b, h) → 11	X	
X (d, f) → 14	X	



AKTU PYQ



$$\text{cost (MST)} = 4 + 8 + 2 + 1 + 2 + 4 + 7 + 9$$

$$= \boxed{37} \checkmark$$

Kruskal's Algorithm is preferred when-

- The graph is sparse. ✓
- There are less number of edges in the graph like $E = O(V)$
- The edges are already sorted or can be sorted in linear time.

Prim's Algorithm is preferred when-

- The graph is dense.
- There are large number of edges in the graph like $E = O(V^2)$.

Difference Between Prim's and Kruskal's Algorithm (AKTU PQQ 2021-22)

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Vertex-based, grows the MST one vertex at a time	Edge-based, adds edges in increasing order of weight
Graph Representation	Adjacency matrix	Edge list
Initialization	Starts from an arbitrary vertex	Starts with all vertices as separate trees (forest)
Edge Selection	Chooses the minimum weight edge from the connected vertices	Chooses the minimum weight edge from all edges
Suitable for	Dense graphs	Sparse graphs
Starting Point	Requires a starting vertex	No specific starting point, operates on global edges

Data Structure

Unit-5 DPP-4

1. Differentiate between Prim's algorithm and Kruskal's Algorithm. (AKTU 2021-22)
2. Explain Kruskal's algorithm with pseudocode and working example.

Gateway Classes



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-5 Lecture-5

Today's Target

- Single Source Shortest path problem
 - Dijkstra's Algorithm ✓
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified



AKTU



Gateway Classes



CS IT & CS Allied

Gateway Udaan Batch



Sem-IV

Validity Till
Sem Exam

COMBO of 5 Subjects

- ✓ Video Lectures
- ✓ Pdf Notes
- ✓ AKTU PYQs
- ✓ Most Imp. Questions



Helpline No-7819 0058 53



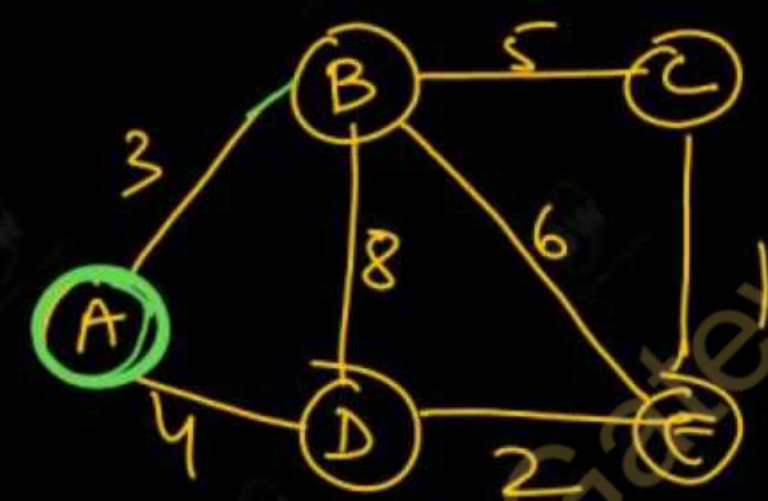
Dijkstra's Algorithm

- Solves single source shortest path problem on a weighted and directed/undirected graph

$G=(V,E)$ where all edge weights are non-negative.

- Ensures optimal solution.

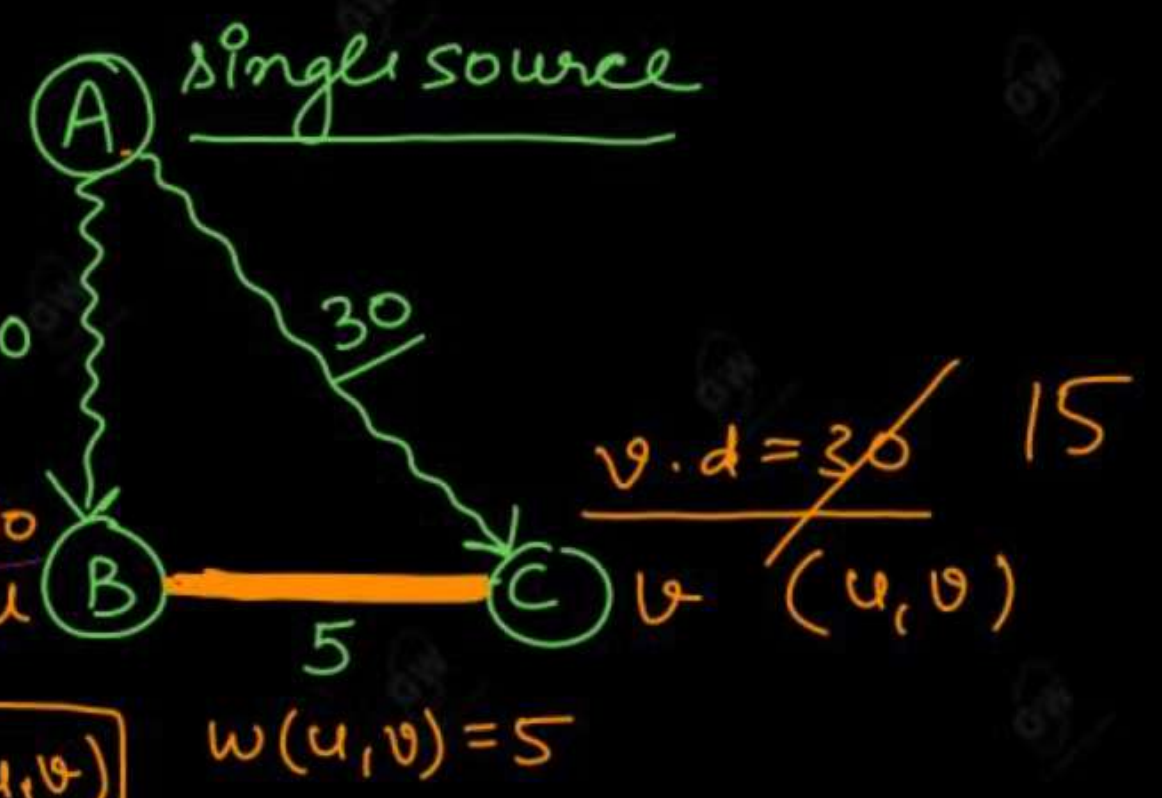
- $A \rightarrow B$
- $A \rightarrow C$
- $A \rightarrow D$
- $A \rightarrow E$



Relax (u, v, w)
 $\{$
 $v.d > u.d + w(u, v) \quad 10$
 $30 > 10 + 5$
 $\}$

$u.d = 10$
 u

$v.d = u.d + w(u, v)$



DIJKSTRA($\underline{G}, \underline{w}, \underline{s}$) *source vertex*

1 INITIALIZE-SINGLE-SOURCE($\underline{G}, \underline{s}$)

2 $\underline{S} = \emptyset$, $\Rightarrow$ *solution set initially empty (will include all vertices one by one)*

3 $\underline{Q} = \underline{G.V}$

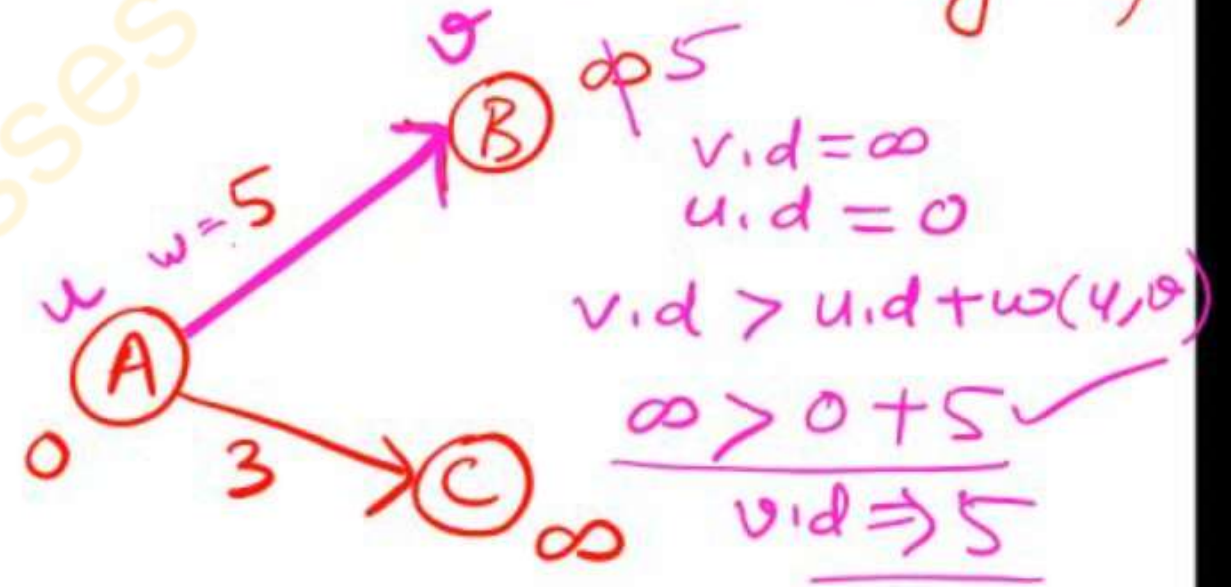
4 **while** $\underline{Q} \neq \emptyset$ // *untill Q becomes empty*

5 $\underline{u} = \text{EXTRACT-MIN}(\underline{Q})$

6 $\underline{S} = \underline{S} \cup \{u\}$

7 **for each** vertex $\underline{v} \in \underline{G.Adj}[u]$

8 $\underline{\text{RELAX}}(u, v, w)$



RELAX(u, v, w)

1 **if** $v.d > u.d + w(u, v)$

2 $v.d = u.d + w(u, v)$

3 $v.\pi = u$

INITIALIZE-SINGLE-SOURCE($\underline{G}, \underline{s}$)

1 **for each** vertex $\underline{v} \in \underline{G.V}$

2 $v.d = \infty$

3 $v.\pi = \text{NIL}$

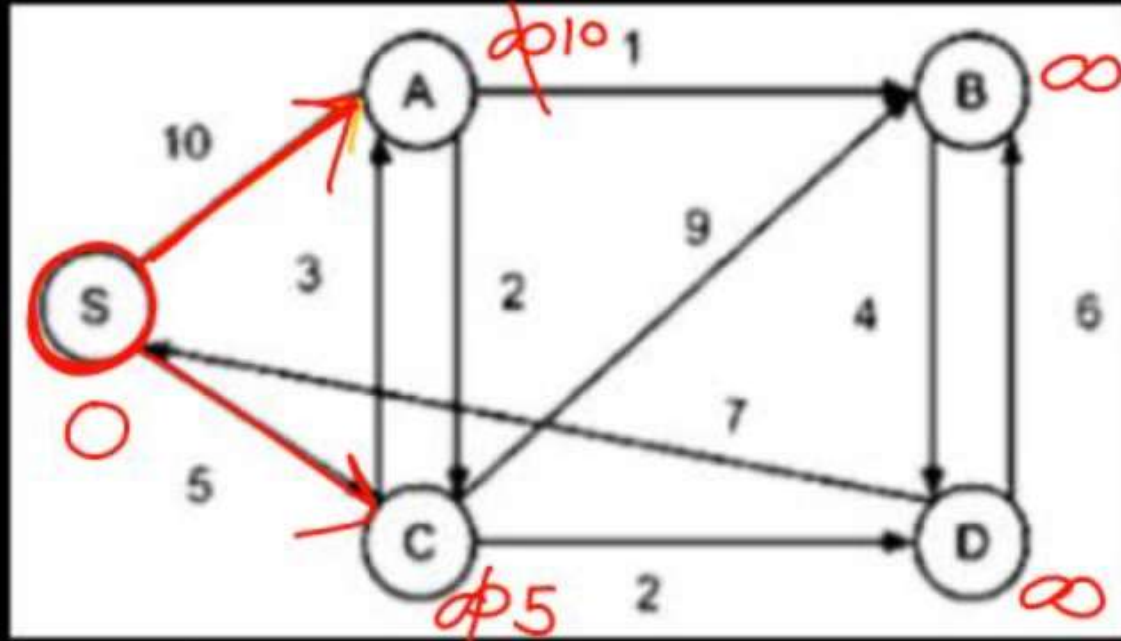
4 $s.d = 0$

Time complexity of Dijkstra Algo

$E \log V$

Q: Consider the following graph and using Dijkstra Algorithm find the shortest path.

(AKTU 2023-24, 2019-20)



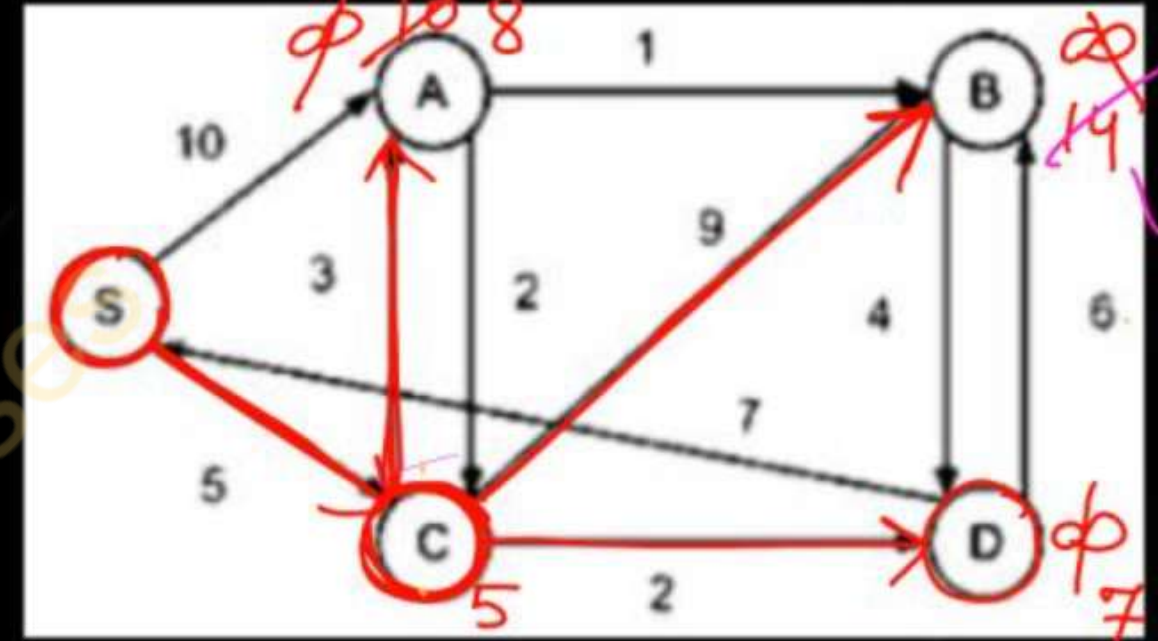
Extracted
vertex

Vertices in Q

$S(-, 0)$ $B(-, \infty)$ $D(-, \infty)$
 $A(-, \infty)$ $C(-, \infty)$

$S(-, 0)$ $A(S, 0+10)$ $B(-, \infty)$ $D(-, \infty)$

$C(S, 0+5)$



Extracted
vertex

Vertices in Q

$C(S, 5)$

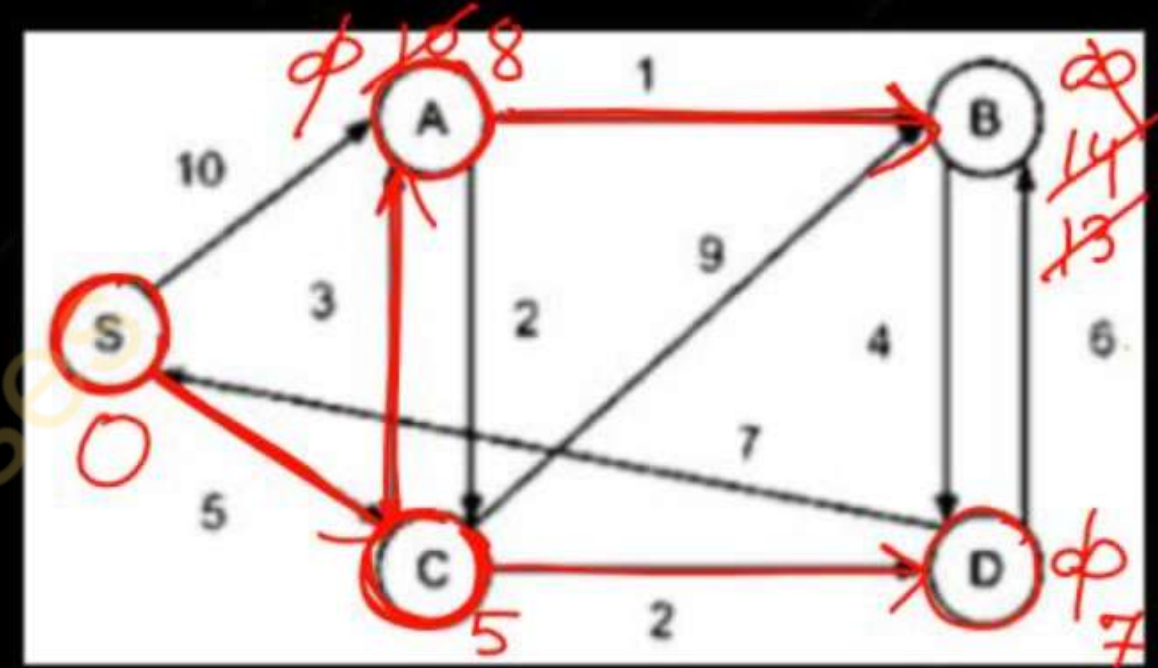
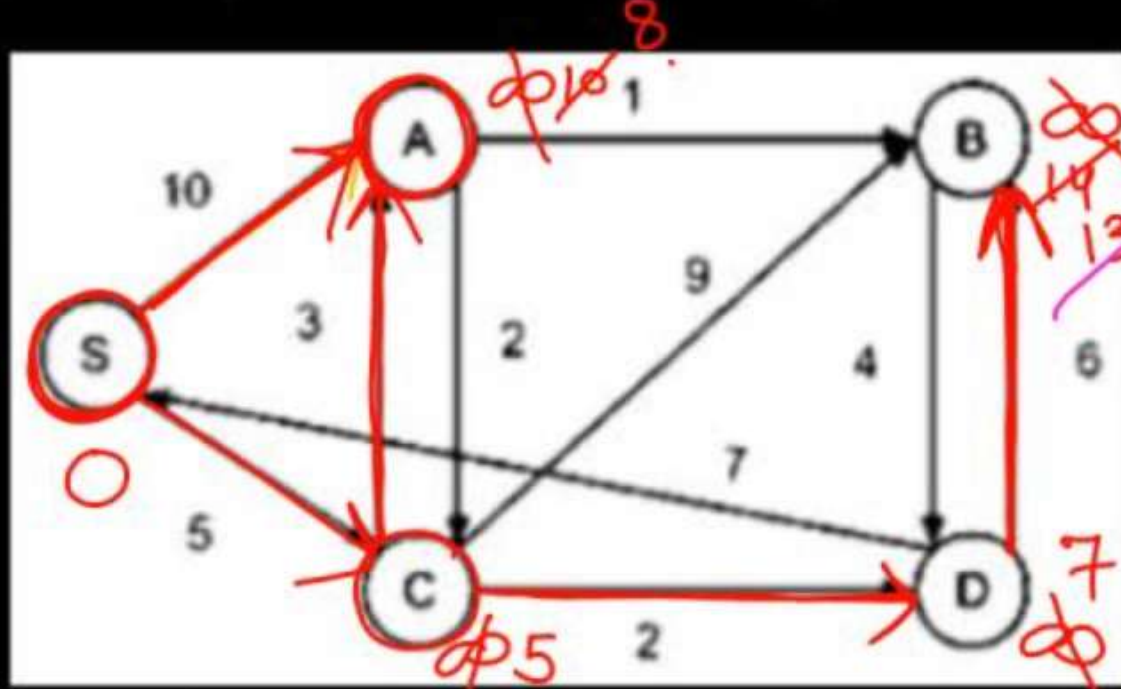
$A(C, 5+3)$ $B(C, 14)$ $D(C, 7)$

$D(C, 7)$

$A(C, 8)$ $B(D, 7+6)$

Q: Consider the following graph and using Dijkstra Algorithm find the shortest path.

(AKTU 2023-24, 2019-20)



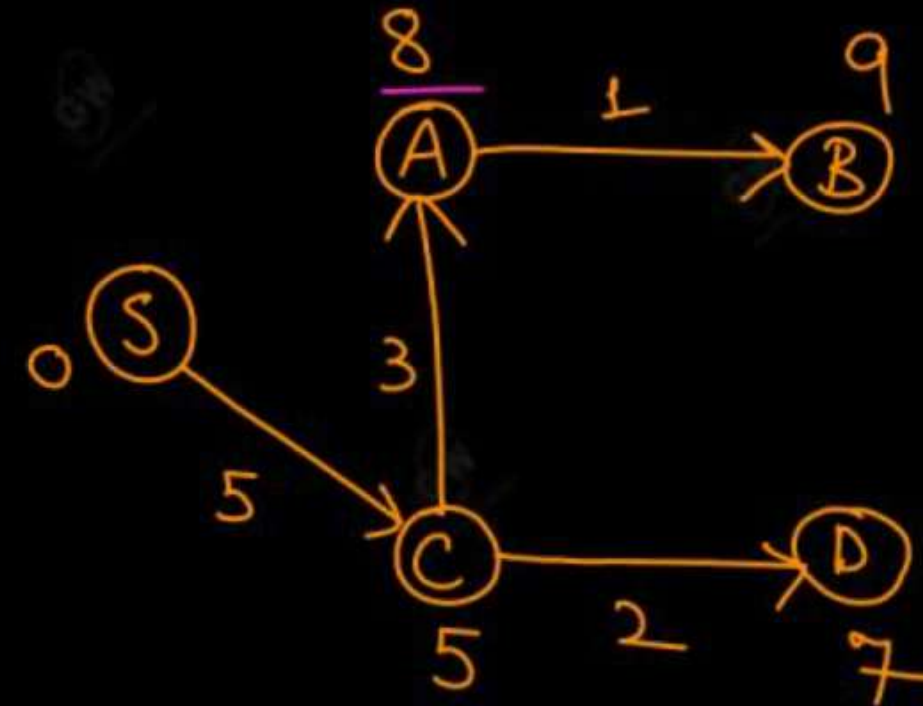
Extracted
vertex

Vertices in Q

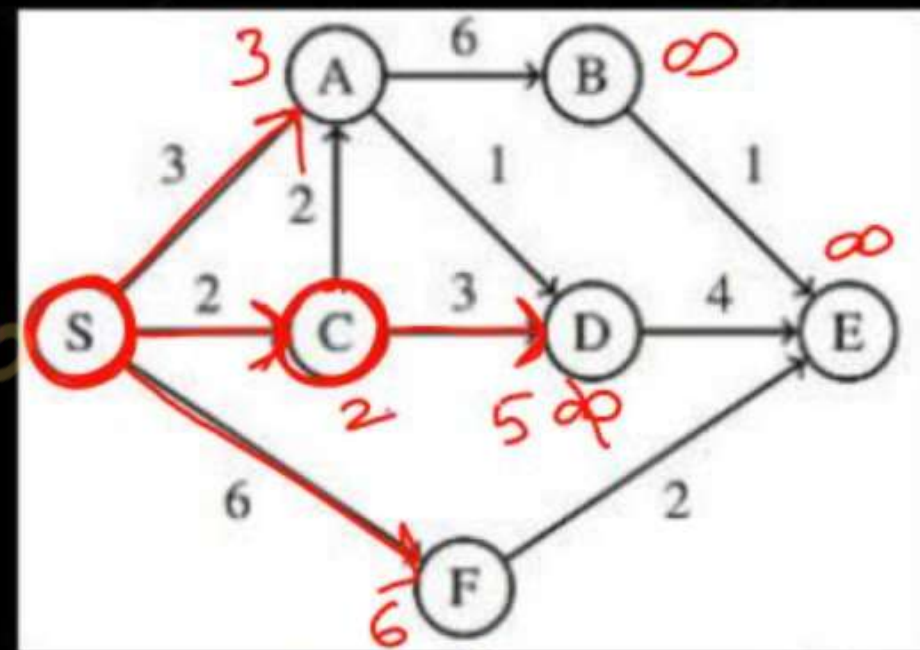
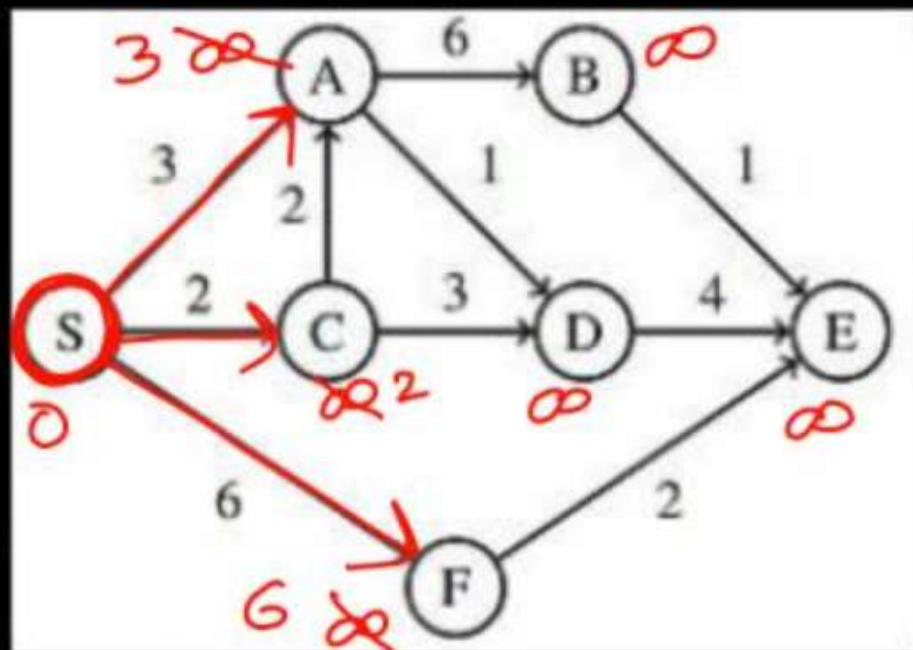
$A(C, 8)$

$B(A, 8+1)$

$B(A, 9)$ → final cost of B



Q: Find the shortest path in graph with vertex 'S' as source vertex. (AKTU 2022-23)



Extracted
Vertices

Vertices in Q

$S(-, 0)$ $A(-, \infty)$ $B(-, \infty)$

$C(-, \infty)$ $D(-, \infty)$ $E(-, \infty)$

$F(-, \infty)$

$S(-, 0)$

$A(S, 0+3)$ $B(-, \infty)$ $C(S, 0+2)$

$D(-, \infty)$ $E(-, \infty)$ $F(S, 0+6)$

Extracted
Vertex

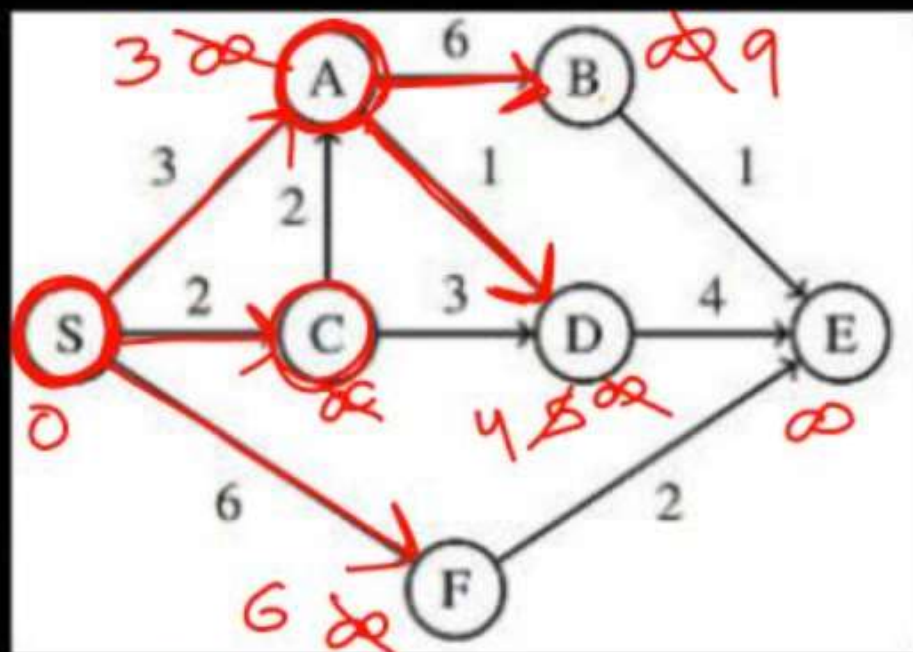
Vertices in Q

$C(S, 2)$

$A(S, 3)$ $D(C, 2+3)$ $B(-, \infty)$

$E(-, \infty)$ $F(S, 6)$

Q: Find the shortest path in graph with vertex 'S' as source vertex. (AKTU 2022-23)



Extracted
Vertices

Vertices in Q

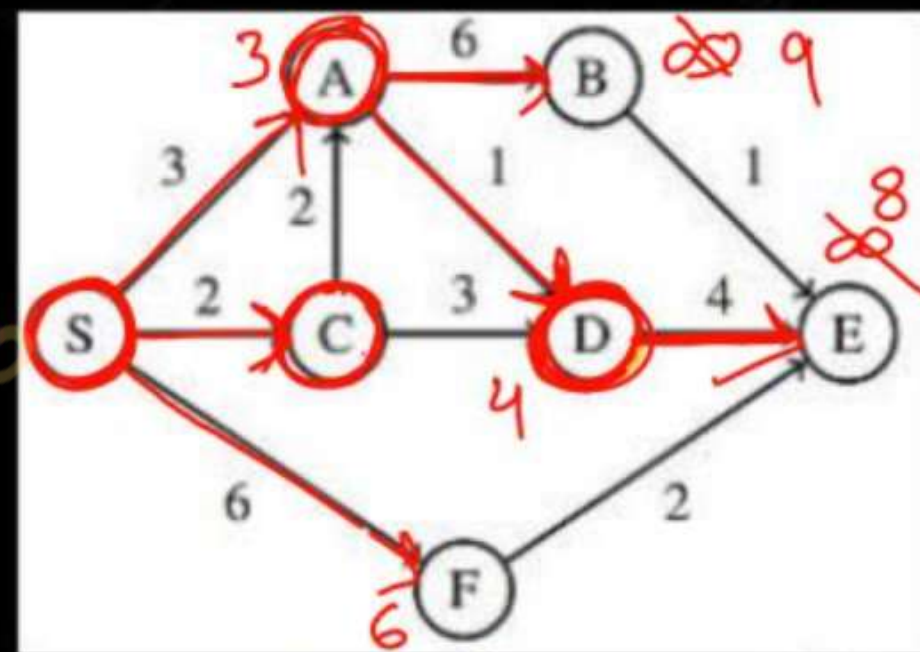
$A(S, 3)$

$B(A, 3+6)$

$D(A, 3+1)$

$E(-, \infty)$

$F(S, 6)$



Extracted
Vertex

Vertices in Q

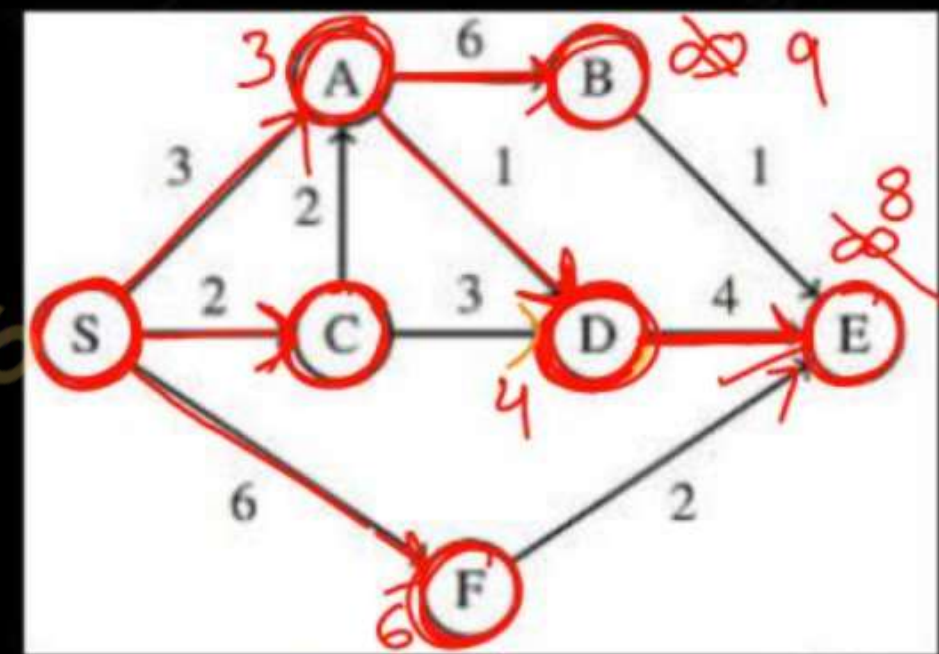
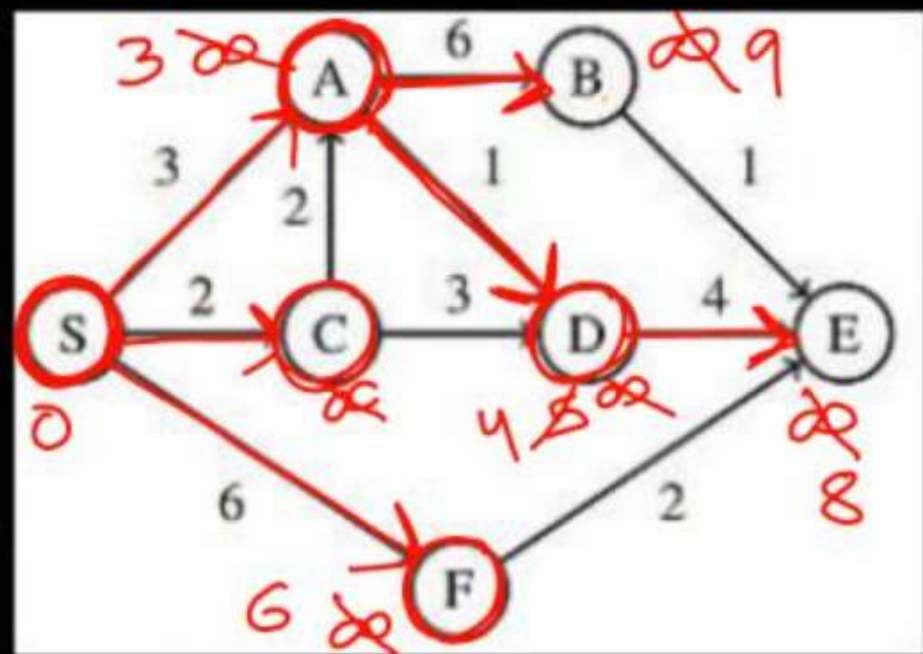
$D(A, 4)$

$B(A, 9)$

$E(D, 4+4)$

$F(S, 6)$

Q: Find the shortest path in graph with vertex 'S' as source vertex. (AKTU 2022-23)



Extracted
Vertices

Vertices in Q

F(S, 6)

B(A, 9) E(D, 8)

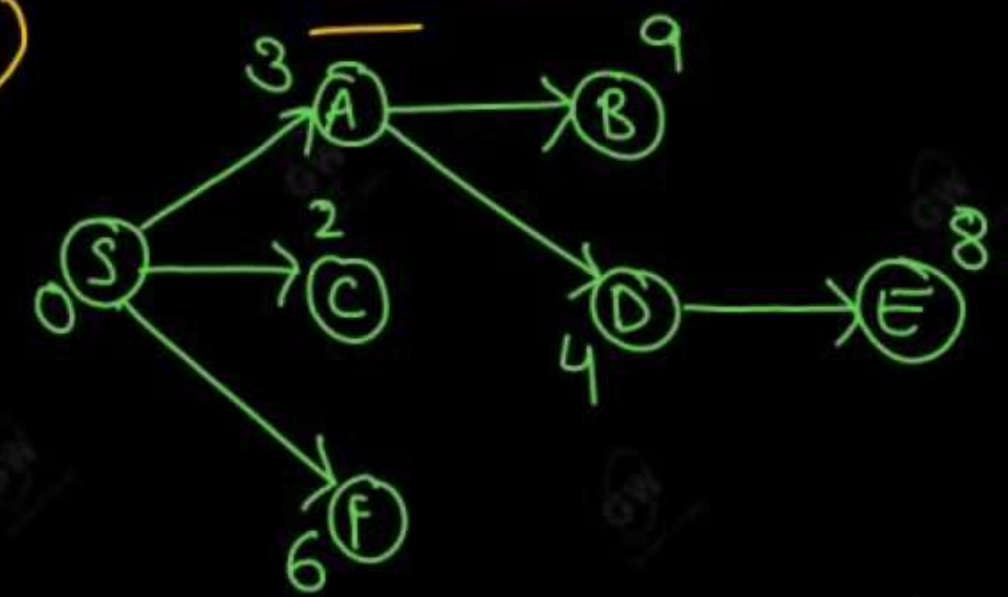
Extracted
Vertex

Vertices in Q

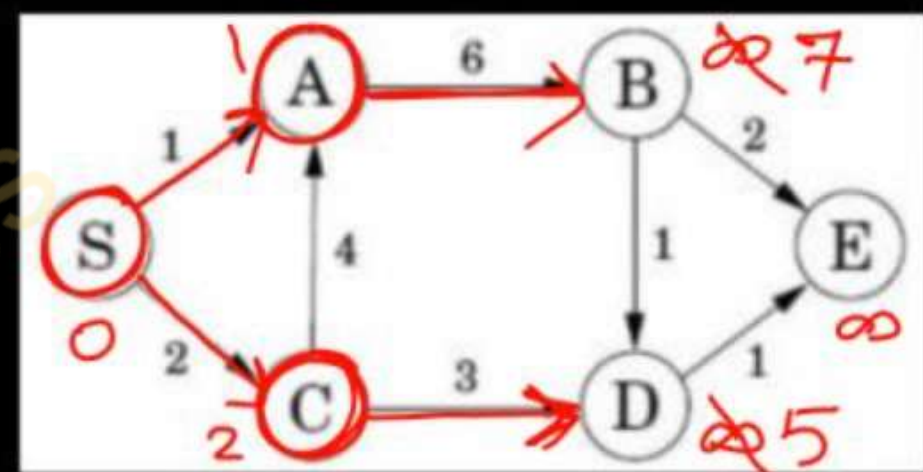
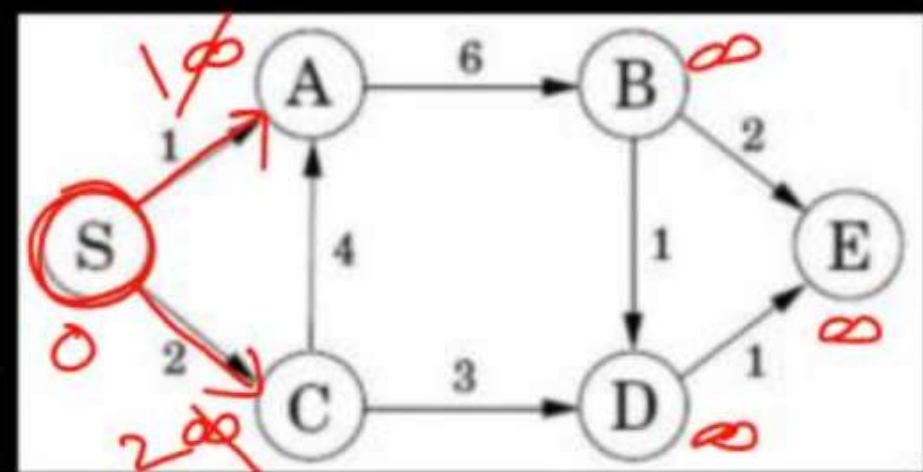
E(D, 8)

B(A, 9)

B(A, 9)



Q: Considering 'S' as source vertex, Apply the Dijkstra's shortest path algorithm on the following graph: (AKTU 2018-19)



Extracted vertex Vertices in Q

$S(-, 0)$ $A(-, \infty)$ $B(-, \infty)$
 $C(-, \infty)$ $D(-, \infty)$ $E(-, \infty)$

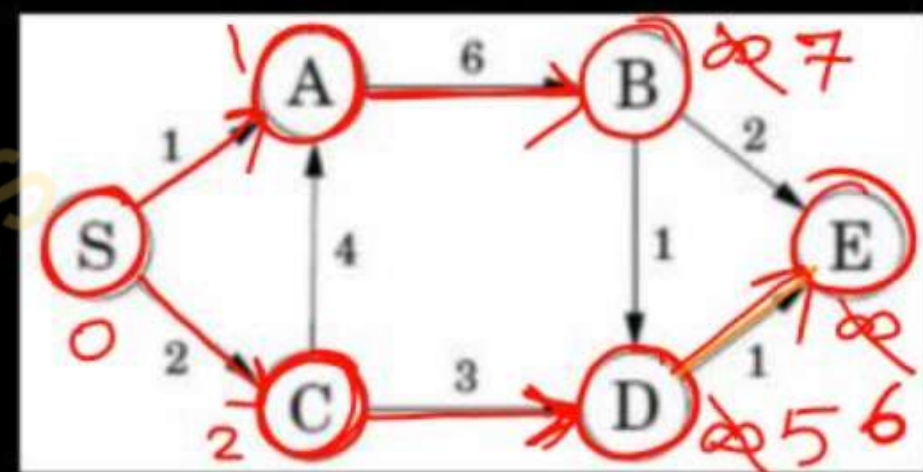
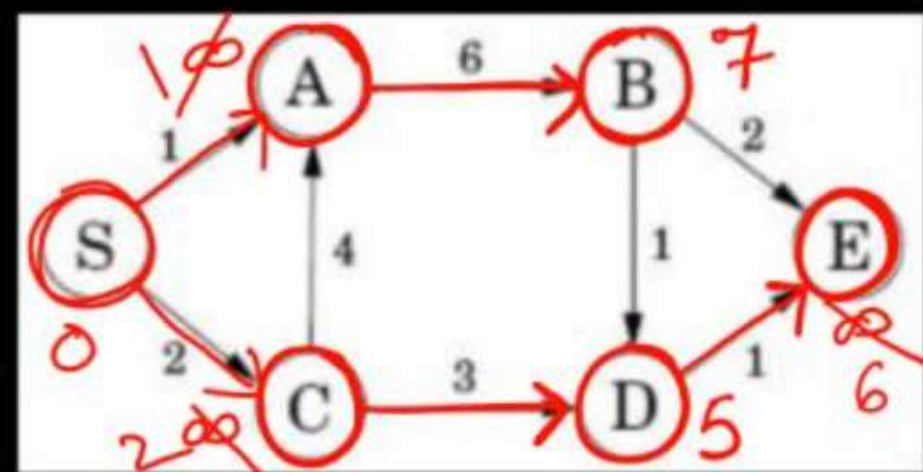
$S(-, 0)$ $A(S, 0+1)$ $B(-, \infty)$
 $C(S, 0+2)$ $D(-, \infty)$
 $E(-, \infty)$

Extracted vertex Vertices in Q

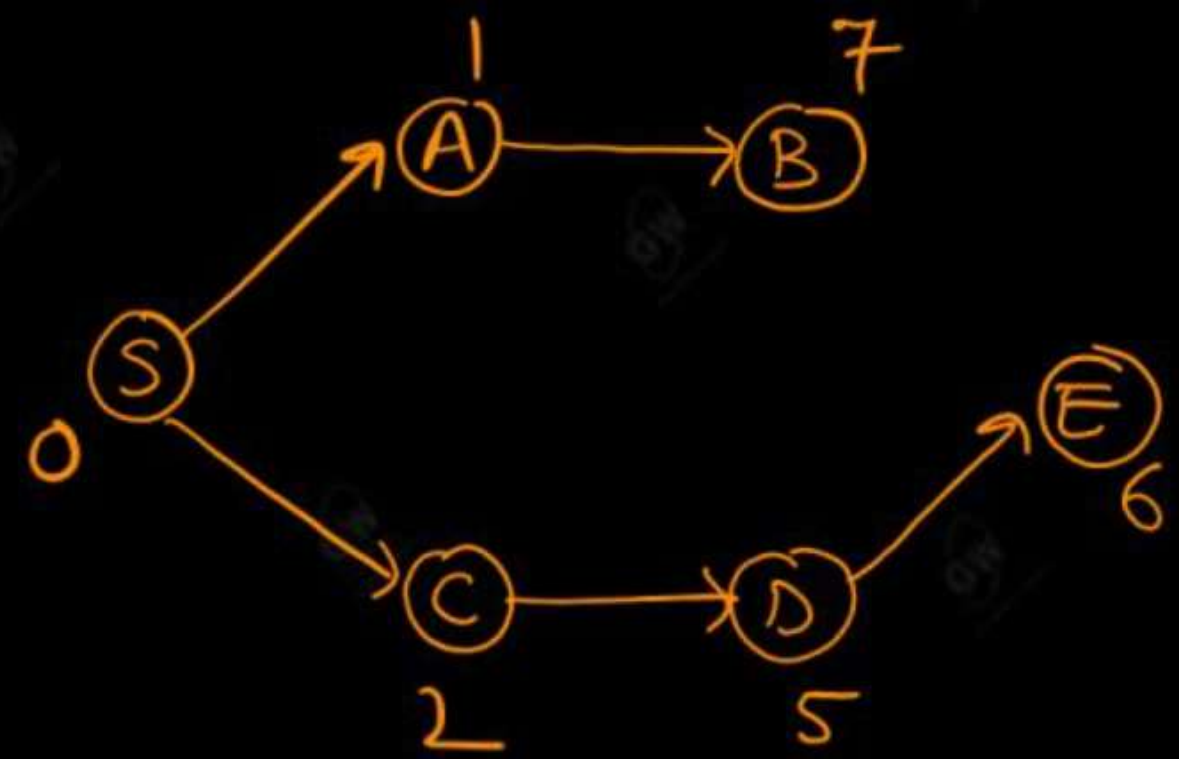
$A(S, 1)$ $B(A, 1+6)$ $C(S, 2)$
 $D(-, \infty)$ $E(-, \infty)$

$C(S, 2)$ $B(A, 7)$ $D(C, 2+5)$
 $E(-, \infty)$

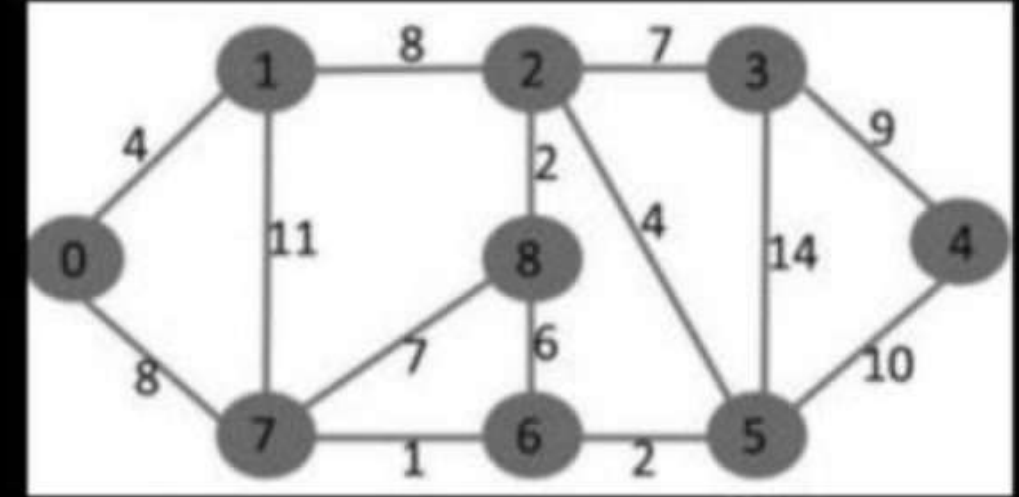
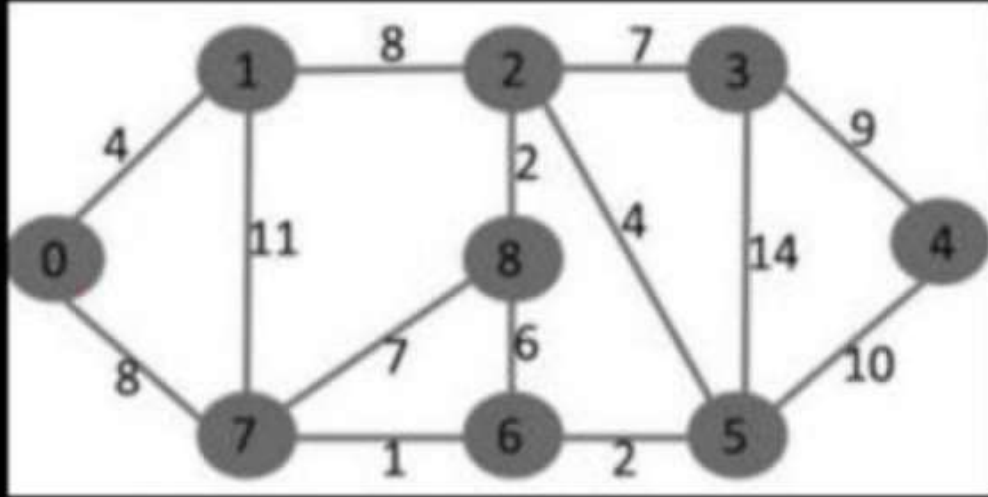
Q: Considering 'S' as source vertex, Apply the Dijkstra's shortest path algorithm on the following graph: (AKTU 2018-19)



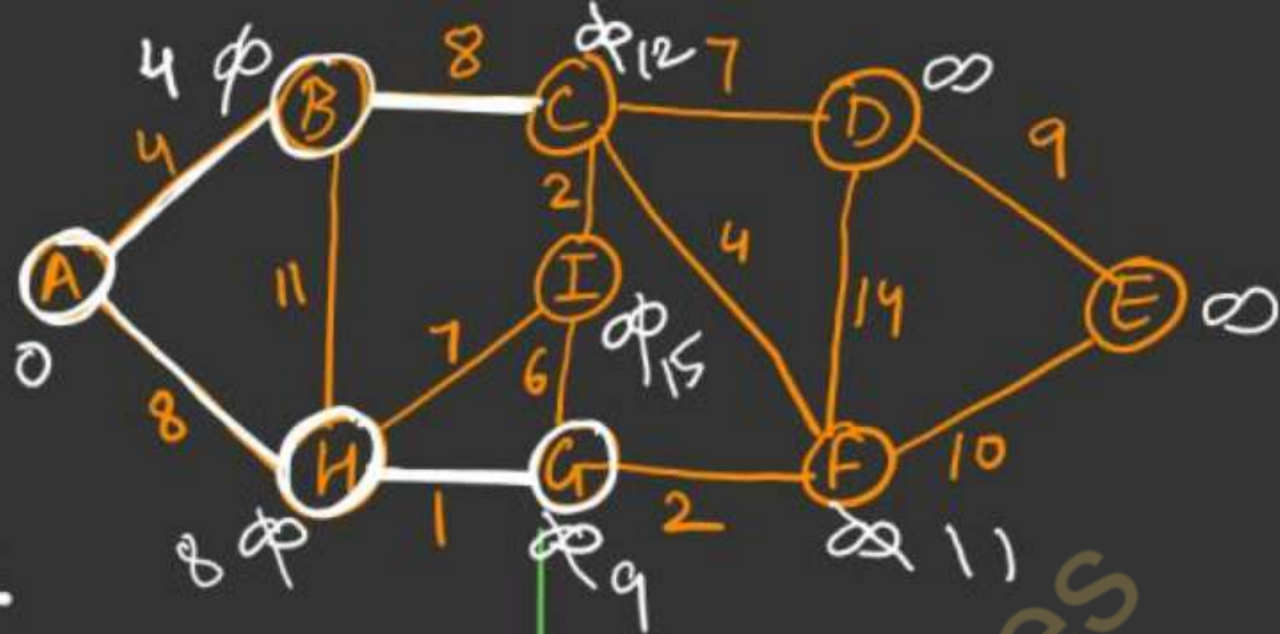
<u>Extracted vertex</u>	<u>Vertices in Q</u>
D(C, 5)	B(A, 7) E(D, 5+1)
E(D, 6)	B(A, 7)
B(A, 7)	—



Q: Use Dijkstra's algorithm to find the shortest paths from source to all other vertices in the following graph. (AKTU 2021-22)



Gateway Classes



Extracted
vertex

Vertices in Q

A(-, 0)

B(A, 0+4) C(-, ∞), D(-, ∞)

E(-, ∞) F(-, ∞) G(-, ∞)

H(A, 0+8) I(-, ∞)

B(A, 4)

C(B, 4+8) D(-, ∞) E(-, ∞)

F(-, ∞) G(-, ∞)

H(A, 8) I(-, ∞)

Extracted vertex

H(A, 8)

G(H, 9)

Vertices in Q

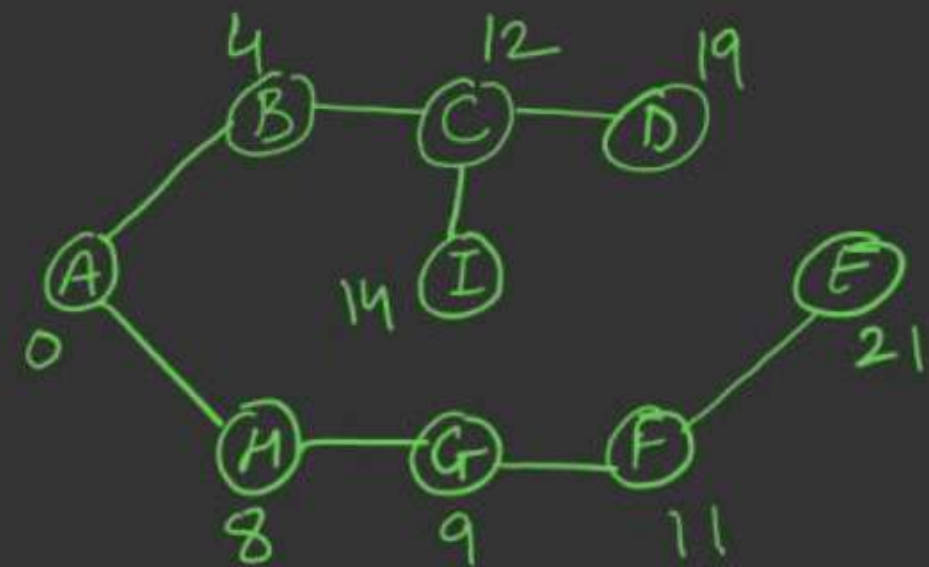
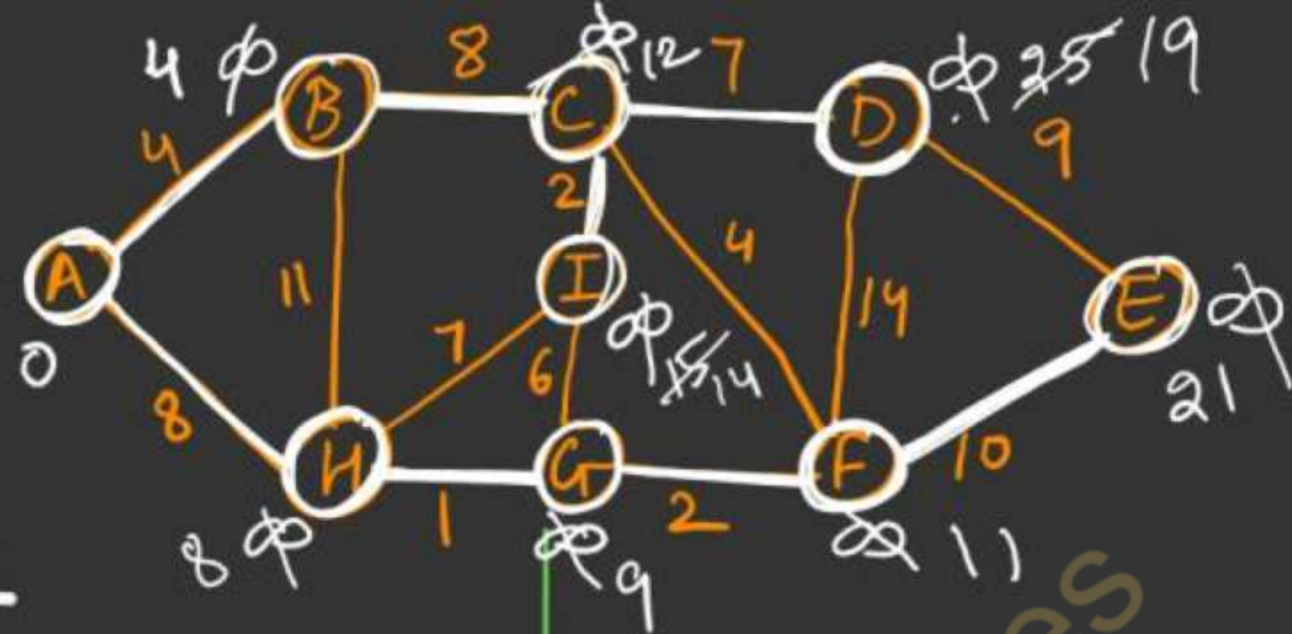
C(B, 12) D(-, ∞) E(-, ∞)

F(-, ∞) G(H, 8+1)

I(H, 8+7)

C(B, 12) D(-, ∞) E(-, ∞)

F(G, 9+2) I(H, 15)



Extracted vertex

Vertices in Q

F(G, 11)

$C(B, 12)$ $D(F, 11+14)$
 $E(F, 11+10)$ $I(H, 15)$

C(B, 12)

$D(C, 12+7)$ $E(F, 21)$
 $I(C, 12+2)$

Extracted vertex

I(C, 14)

D(C, 19)

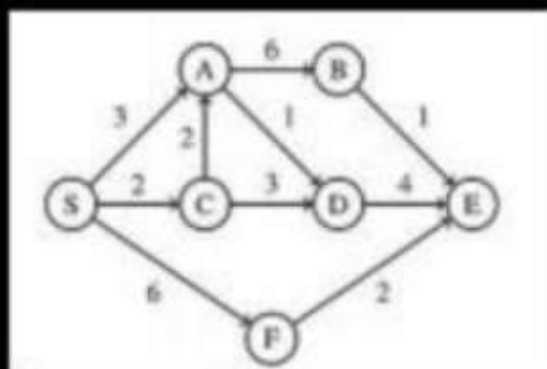
E(F, 21)

Vertices in Q

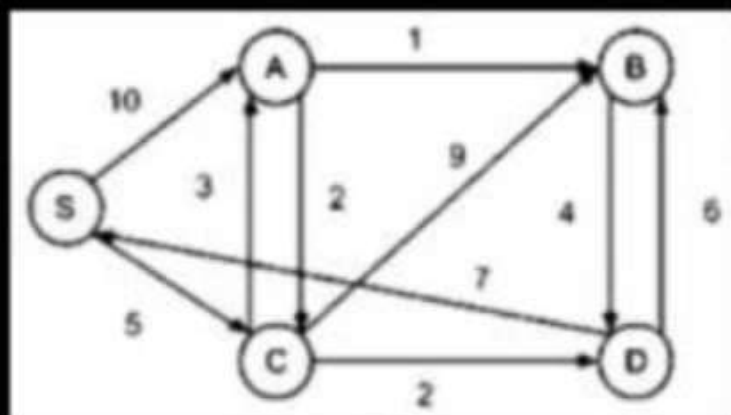
$D(C, 19)$ $E(F, 21)$

$E(F, 21)$

1. Write the Dijkstra algorithm for shortest path in a graph and also find the shortest path from 'S' to all remaining vertices of graph in the following graph: (AKTU 2022-23)



2. Describe the Dijkstra algorithm to find the shortest path. Find the the shortest path in the following graph with vertex 'S' as source vertex. (AKTU 2019-20)





AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-5 Lecture-6

Today's Target

- All Pair Shortest path problem ✓
 - Floyd Warshall Algorithm
- AKTU PYQs
- DPP

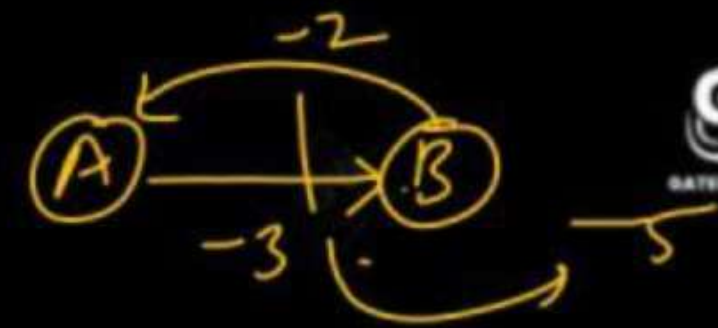


By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

Floyd Warshall Algorithm

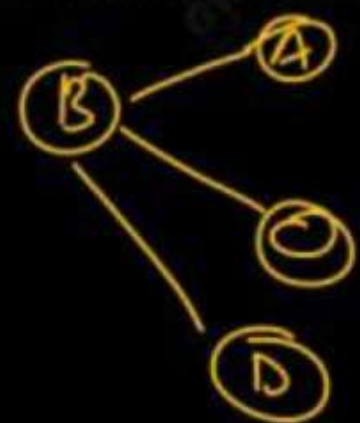
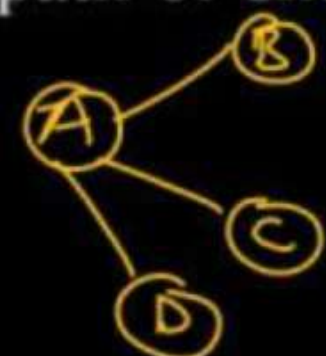
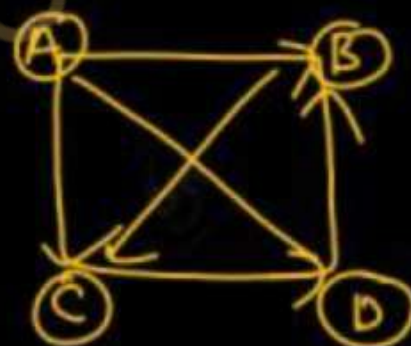


→ It is a dynamic programming based algorithm so it breaks the problem into Smaller subproblems and then solves each sub problem. After that it combines the result to solve bigger problem.

→ Also called as Floyd's Algo.



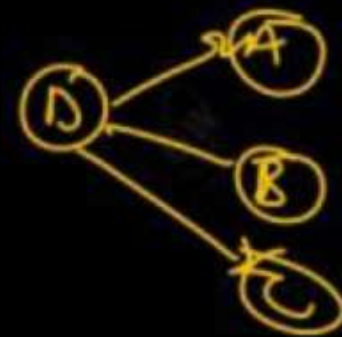
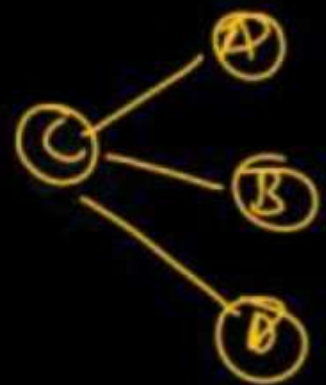
→ It is all - pair shortest path algorithm used to find the shortest path or shortest distance between every pair of Vertices in the given graph.



→ Applicable to both directed / undirected graph.

→ It fails if the graph have negative cost cycle.

→ It computes transitive closure of a graph.



- Make a matrix for the given input graphs.
- Create another solution matrix as the input graph Matrix.
- Update the solution Matrix while considering all vertices as an intermediate vertex.
- Choose each vertex one by one and update all the shortest paths that includes the selected vertex as an intermediate vertex in the as an intermediate vertex in the shortest path.

Floyd-Warshall(w)

n = no. of rows in w m=4

D⁰ = w

for k = 1 to n → intermediate vertex

for i = 1 to n // for rows

for j = 1 to n // for cols

$$D^K(i, j) = \min(D^{K-1}[i, k] + D^{K-1}[k, j], D^{K-1}(i, j))$$

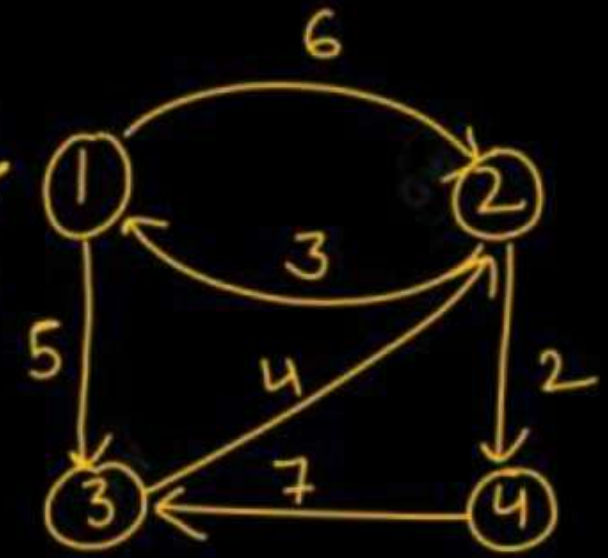
return Dⁿ

Time complexity = $O(n^3)$

n → no. of vertices in a graph.

Example -1

$$D^K(i, j) = \min \{ D^{K-1}(i, j), D^{K-1}(i, k) + D^{K-1}(k, j) \}$$



Step-1 → write D^0 matrix

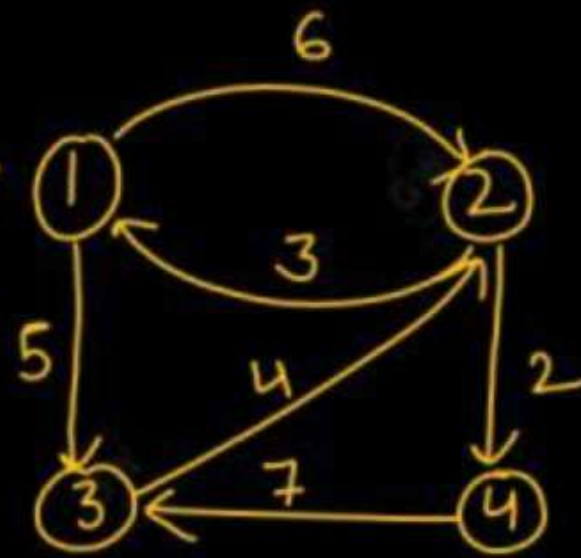
Step-2 → Calculate D^1 from D^0

$$D^1 = \begin{matrix} & \begin{matrix} \xrightarrow{j} \\ 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} \downarrow i \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 6 & 5 & \infty \\ 3 & 0 & - & - \\ \infty & - & 0 & - \\ \infty & - & - & 0 \end{bmatrix} \end{matrix}$$

$$D^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 6 & 5 & \infty \\ 3 & 0 & \infty & 2 \\ \infty & 4 & 0 & \infty \\ \infty & \infty & 7 & 0 \end{bmatrix} \end{matrix}$$

D^1 → considering node 1 as intermediate vertex
 D^2 → node 2 as intermediate vertex
 D^3
 D^4

Example - 1



$$D^K(i, j) = \min \{ D^{K-1}(i, j), D^{K-1}(i, k) + D^{K-1}(k, j) \}$$

$K=1, i=2, j=3$

$$D^1(2, 3) = \min \{ D^0(2, 3), D^0(2, 1) + D^0(1, 3) \}$$

$2 \xrightarrow{1} 3$

$$= \min \{ \infty, 3 + 5 \} = 8$$

$$D^1(2, 4) = \min \{ D^0(2, 4), D^0(2, 1) + D^0(1, 4) \}$$

$$= \min \{ 2, 3 + \infty \} = 2$$

$$D^1(3, 2) = \min \{ D^0(3, 2), D^0(3, 1) + D^0(1, 2) \} = \min \{ 4, \infty + 6 \} = 4$$

$$D^1(3, 4) = \min \{ D^0(3, 4), D^0(3, 1) + D^0(1, 4) \} = \min \{ \infty, \infty + \infty \} = \infty$$

$$D^1(4, 2) = \min \{ D^0(4, 2), D^0(4, 1) + D^0(1, 2) \} = \min \{ \infty, \infty + 6 \} = \infty$$

$$D^1(4, 3) = \min \{ D^0(4, 3), D^0(4, 1) + D^0(1, 3) \} = \min \{ 7, \infty + 5 \} = 7$$

$D^0 =$

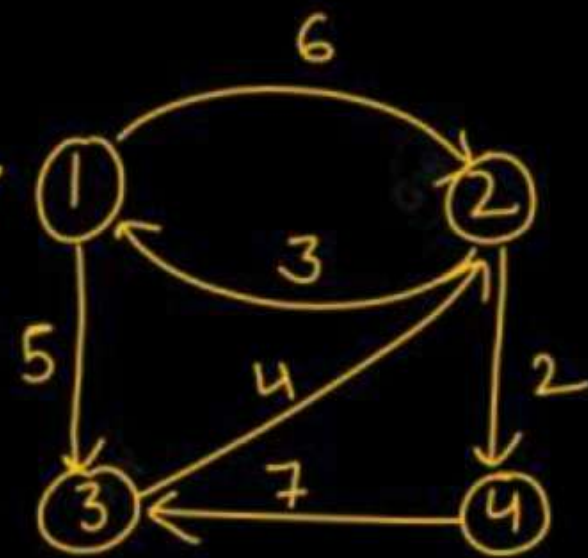
	1	2	3	4
1	0	6	5	∞
2	3	0	∞	2
3	∞	4	0	∞
4	∞	∞	7	0

$D^1 =$

	1	2	3	4
1	0	6	5	8
2	3	0	∞	2
3	∞	4	0	8
4	∞	8	7	0

Example - 1

$$D^K(i, j) = \min \{ D^{K-1}(i, j), D^{K-1}(i, k) + D^{K-1}(k, j) \}$$



$$D^2(1, 3) = \min \{ D^1(1, 3), D^1(1, 2) + D^1(2, 3) \} = \min \{ 5, 6 + 8 \}$$

$$D^2(1, 4) = \min \{ D^1(1, 4), D^1(1, 2) + D^1(2, 4) \} = \min \{ \infty, 6 + 2 \} = 8$$

$$D^2(3, 1) = \min \{ D^1(3, 1), D^1(3, 2) + D^1(2, 1) \} = \min \{ \infty, 4 + 3 \} = 7$$

$$D^2(3, 4) = \min \{ D^1(3, 4), D^1(3, 2) + D^1(2, 4) \} = \min \{ \infty, 4 + 2 \} = 6$$

$$D^2(4, 1) = \min \{ D^1(4, 1), D^1(4, 2) + D^1(2, 1) \} = \min \{ \infty, \infty + 3 \} = \infty$$

$$D^2(4, 3) = \min \{ D^1(4, 3), D^1(4, 2) + D^1(2, 3) \} = \min \{ 7, \infty + 8 \} = 7$$

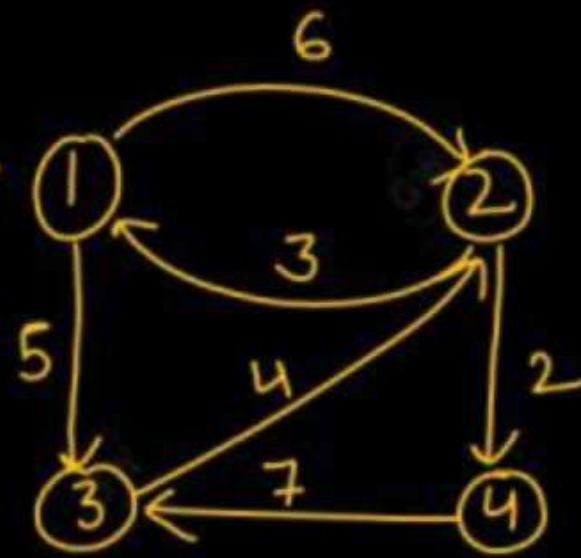
$$D^1 =$$

	1	2	3	4
1	0	6	5	∞
2	3	0	8	2
3	∞	4	0	∞
4	∞	∞	7	0

$$D^2 =$$

	1	2	3	4
1	0	6	5	8
2	3	0	8	2
3	7	4	0	6
4	8	∞	7	0

Example - 1



$$D^K(i, j) = \min \{ D^{K-1}(i, j), D^{K-1}(i, k) + D^{K-1}(k, j) \}$$

$$D^3(1, 2) = \min \{ D^2(1, 2), D^2(1, 3) + D^2(3, 2) \} = \min \{ 6, 5 + 4 \} = 6$$

$$D^3(1, 4) = \min \{ D^2(1, 4), D^2(1, 3) + D^2(3, 4) \} = \min \{ 8, 5 + 6 \} = 8$$

$$D^3(2, 1) = \min \{ D^2(2, 1), D^2(2, 3) + D^2(3, 1) \} = \min \{ 3, 8 + 7 \} = 3$$

$$D^3(2, 4) = \min \{ D^2(2, 4), D^2(2, 3) + D^2(3, 4) \} = \min \{ 2, 8 + 6 \} = 2$$

$$D^3(4, 1) = \min \{ D^2(4, 1), D^2(4, 3) + D^2(3, 1) \} = \min \{ \infty, 7 + 7 \} = 14$$

$$D^3(4, 2) = \min \{ D^2(4, 2), D^2(4, 3) + D^2(3, 2) \} = \min \{ \infty, 7 + 4 \} = 11$$

$$D^2 = \begin{bmatrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 6 & 5 & 8 \\ 2 & 3 & 0 & 8 & 2 \\ 3 & 7 & 4 & 0 & 6 \\ 4 & \infty & \infty & 7 & 0 \end{bmatrix}$$

$$D^3 = \begin{bmatrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 6 & 5 & 8 \\ 2 & 3 & 0 & 8 & 2 \\ 3 & 7 & 4 & 0 & 6 \\ 4 & 14 & 11 & 7 & 0 \end{bmatrix}$$

Example - 1

$$D^K(i, j) = \min \{ D^{K-1}(i, j), D^{K-1}(i, k) + D^{K-1}(k, j) \}$$

$$D^4(1, 2) = \min \{ D^3(1, 2), D^3(1, 4) + D^3(4, 2) \} = \min \{ 6, 8 + 11 \} = 6$$

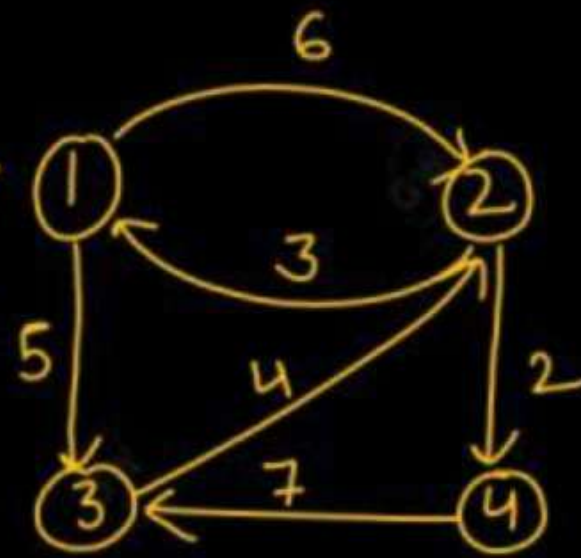
$$D^4(1, 3) = \min \{ D^3(1, 3), D^3(1, 4) + D^3(4, 3) \} = \min \{ 5, 8 + 7 \} = 5$$

$$D^4(2, 1) = \min \{ D^3(2, 1), D^3(2, 4) + D^3(4, 1) \} = \min \{ 3, 2 + 14 \} = 3$$

$$D^4(2, 3) = \min \{ D^3(2, 3), D^3(2, 4) + D^3(4, 3) \} = \min \{ 8, 2 + 7 \} = 8$$

$$D^4(3, 1) = \min \{ D^3(3, 1), D^3(3, 4) + D^3(4, 1) \} = \min \{ 7, 6 + 14 \} = 7$$

$$D^4(3, 2) = \min \{ D^3(3, 2), D^3(3, 4) + D^3(4, 2) \} = \min (4, 6 + 11) = 4$$



$$D^3 = \begin{bmatrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 6 & 5 & 8 \\ 2 & 3 & 0 & 8 & 2 \\ 3 & 7 & 4 & 0 & 6 \\ 4 & 14 & 11 & 7 & 0 \end{bmatrix}$$

$$D^4 = \begin{bmatrix} & 1 & 2 & 3 & 4 \\ 1 & 0 & 6 & 5 & 8 \\ 2 & 3 & 0 & 8 & 2 \\ 3 & 7 & 4 & 0 & 6 \\ 4 & 14 & 11 & 7 & 0 \end{bmatrix}$$

Ans ←

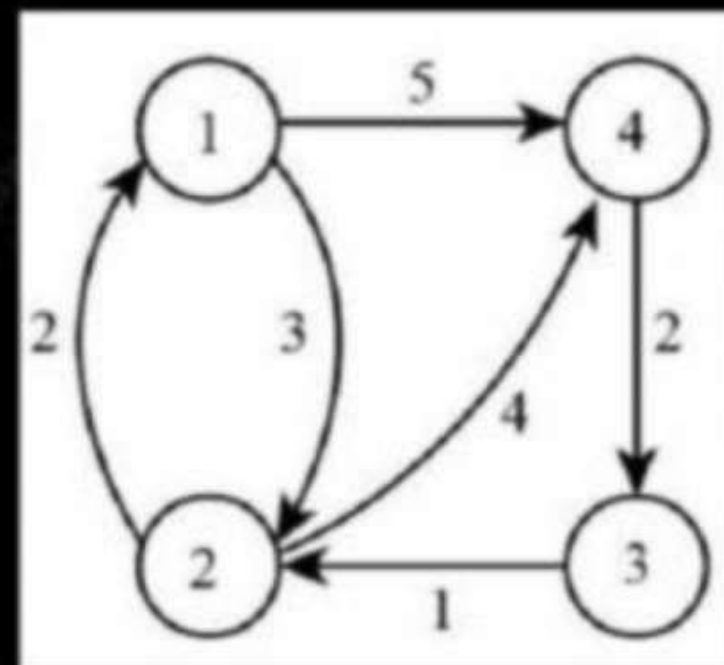
Q: Use the Floyd Warshall algorithm to find shortest path among all the vertices in the given graph:
(AKTU 2022-23)

$$D^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

$$D^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ 3 & 1 & 0 & 5 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

$$D^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

$$D^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$



$$D^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & 6 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

Q: Write a program in C language to compute the indegree and outdegree of every vertex of a directed graph when the graph is represented by an adjacency matrix. (AKTU 2023-24)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int graph[50][50]; // 2-D array to store all the edges
```

```
    int indegree[50], outdegree[50]; // Arrays to store in-degree
```

```
    int vertices, edges, i, j, from, to; and out-degree of vertices.
```

```
    printf("Enter the number of vertices: ");
```

```
    scanf("%d", &vertices); // 4
```

```
    printf("Enter the number of edges: ");
```

```
    scanf("%d", &edges); // 6
```

```
    for (i = 0; i < vertices; i++)
```

```
    {
```

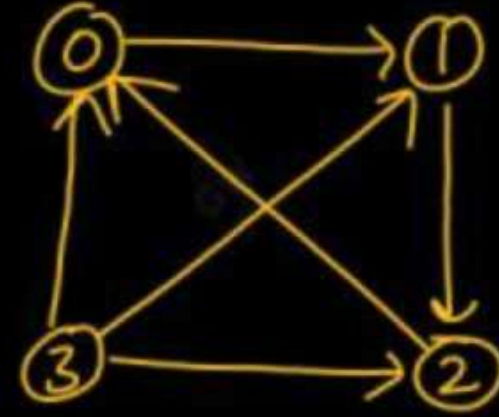
```
        for (j = 0; j < vertices; j++)
```

```
        {
```

```
            graph[i][j] = 0;
```

```
        }
```

```
    }
```



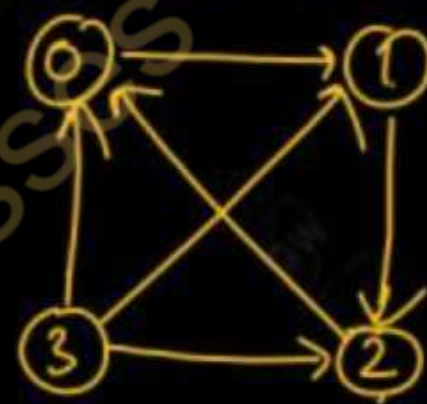
$$\text{graph} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

```

printf("Enter edges (format: from to):\n"); // Read all directed edges
for (i = 0; i < edges; i++)
{
    scanf("%d %d", &from, &to);
    graph[from][to] = 1; //Mark edge in adjacency matrix
}

// Calculate indegree and outdegree
for (i = 0; i < vertices; i++) from
{
    for (j = 0; j < vertices; j++) to
    {
        if (graph[i][j] == 1)
        {
            outdegree[i]++; // outgoing edge from i
            indegree[j]++; // incoming edge to j
        }
    }
}

```



graph =

	0	1	2	3
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

⇓

	0	1	2	3
0	0	1	0	0
1	0	0	1	0
2	1	0	0	0
3	1	1	1	0

```

printf("\nIndegree of vertices:\n");
for (i = 0; i < vertices; i++)
{
    printf("Vertex %d : %d\n", i, indegree[i]);
}

printf("\nOutdegree of vertices:\n");
for (i = 0; i < vertices; i++)
{
    printf("Vertex %d : %d\n", i, outdegree[i]);
}

return 0;
}
    
```

Indegree

0	1	2	3
2	2	2	0

outdegree

0	1	2	3
1	1	1	3

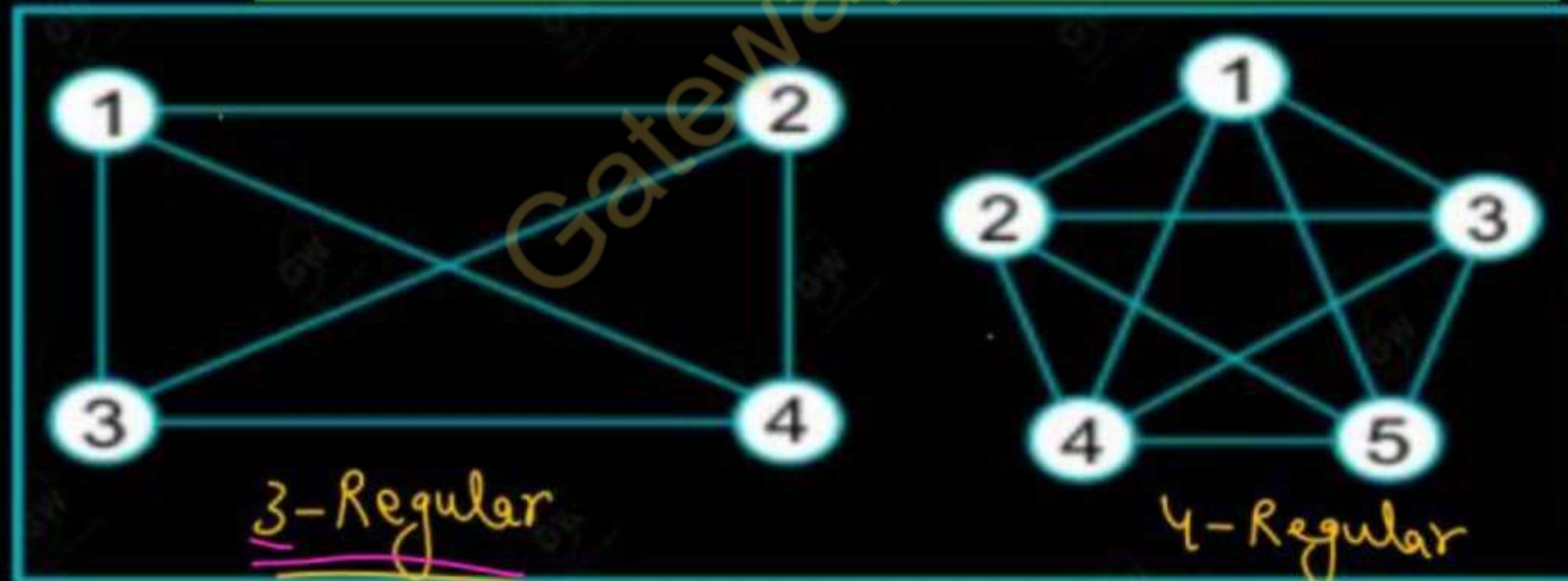
A **regular graph** is a **graph** where each vertex has the same number of neighbors; i.e. every vertex has the same degree. A graph is called **K regular** if degree of each vertex in the graph is K .

no. of vertices = n

degree of each vertex = d (no. of neighbor of each vertices)

$$\text{total edges} = n * (\text{no. of neighbor}) / 2 = (n * d) / 2$$

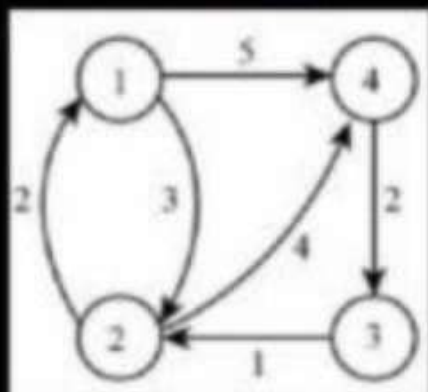
no. of nodes
degree of each node



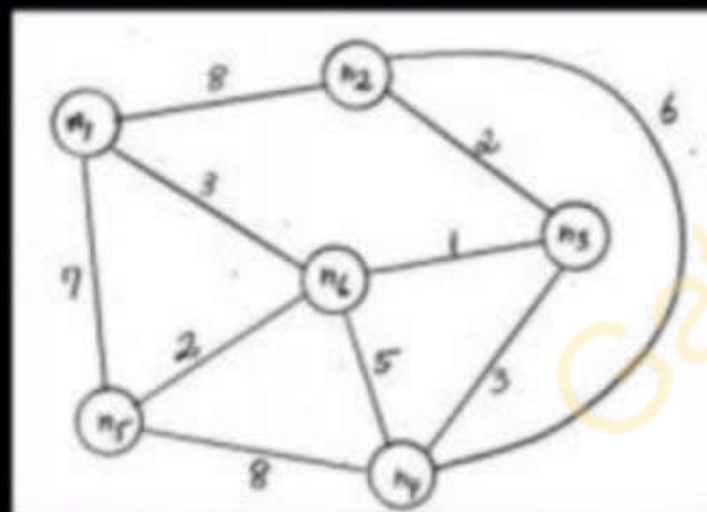
$$\begin{aligned} n &= 4, d = 3 \\ \text{edges} &= \frac{4 \times 3}{2} \\ &\Rightarrow 6 \end{aligned}$$

$$\text{edges} = \frac{5 \times 4}{2} = 10$$

1. Write and explain the Floyd Warshall algorithm to find the all pair shortest path. Use the Floyd Warshall algorithm to find shortest path among all the vertices in the given graph: (AKTU 2022-23)



2. Define Floyd Warshall Algorithm for all pair shortest path and apply the same on following graph: (AKTU 2018-19)



3. Write a program in C language to compute the indegree and outdegree of every vertex of a directed graph when the graph is represented by an adjacency matrix. (AKTU 2023-24)



AKTU



Gateway Classes



CS IT & CS Allied

Gateway Udaan Batch



Sem-IV

Validity Till
Sem Exam

COMBO of 5 Subjects

- ✓ Video Lectures
- ✓ Pdf Notes
- ✓ AKTU PYQs
- ✓ Most Imp. Questions



Helpline No-7819 0058 53





Gateway Classes



Empowering Learners, Transforming Futures

B.Tech Courses



GET IT ON
Google Play



Download on the
App Store

Helpline No- 7819 0058 53